The LisbOn KInetics Global Model User Guide

Current version: `LoKI-GM_26.07`

L L Alves[1], A Tejero-del-Caz[2] and N Pinhão[1]

[1]Instituto de Plasmas e Fusão Nuclear, Instituto Superior Técnico, Universidade de Lisboa, Portugal
[2]Departamento de Física, Facultad de Ciencias, Universidad de Córdoba, Campus de Rabanales, Spain
email: loki@tecnico.ulisboa.pt

October 14, 2026



®2018 I Lemos Alves.
All rights reserved.

The following is the user guide of the LisbOn KInetics Global Model (LoKI-GM) simulation tool, intended to model non-equilibrium low-temperature plasmas (LTPs), produced from different gases or gas mixtures under a wide range of working conditions.

LoKI-GM comprises two modules, that can run self-consistently coupled or as standalone tools:

- a Boltzmann solver (LoKI-B) for the electron Boltzmann equation [1, 2];
- a Chemical solver (LoKI-C) for the global kinetic model(s) of pure gases or gas mixtures.

LoKI-GM leverages on the scientific heritage in the field of non-equilibrium plasma kinetics of the Portuguese group N-Plasmas Reactive: Modeling and Engineering, N-PRiME [3] of *Instituto de Plasmas e Fusão Nuclear* with *Instituto Superior Técnico*, and it was developed resorting to well-grounded scientific foundations established years ago [4, 5]. Several numerical improvements and updates have recently been implemented in LoKI-GM, and further developments are ongoing, as part of a fruitful collaboration with the *Departamento de Física* of *Facultad de Ciencias* with *Universidad de Córdoba* (regarding LoKI-C) and the group Elementary Processes in Gas Discharges, EPG [6] of the Eindhoven University of Technology (regarding LoKI-B).

The LisbOn KInetics Global Model is an open-source tool, distributed under the *GNU general public license*, and developed with an ontology that prioritizes the clear separation between tool and data. Its implementation follows a flexible and extensible object-oriented design, using the following programming languages:

- MATLAB® [7] for LoKI-B and LoKI-C, leveraging its matrix-based architecture;
- C++ [8] for LoKI-B++, taking advantage of the higher performance of a compiled language, and linked with Eigen [9], a C++ template library for linear algebra.

LoKI-B++ is a C++ counterpart of MATLAB's LoKI-B, replicating essentially all of its original features. Unless otherwise specified, the acronym LoKI-B in this guide refers to either version of the code. For simplicity, and unless otherwise specified, the acronym LoKI in this guide refers to the code LoKI-GM.

LoKI-B, LoKI-B++ and LoKI-C are part of the LoKI tool suite and are freely available at `https://github.com/LoKI-suite`, for users to perform LTPs simulations, and for modelers / developers who are invited to continue testing the tool and/or to contribute for its development and improvement.

Contents

1 Introduction **3**

1.1 General features . . . 3

1.2 This user guide . . . 3

1.3 How to contact us . . . 4

1.4 How to reference the code . . . 5

1.5 Acknowledgments . . . 5

2 Getting started **6**

2.1 Requirements and installation . . . 6

2.1.1 GitHub . . . 6

2.2 Directory Structure and Data Organization . . . 6

2.3 Ontology . . . 7

2.3.1 Species description . . . 7

2.3.2 Gaseous system description . . . 8

2.4 Starting LoKI-GM . . . 10

2.5 Simulation types available in LoKI-GM . . . 13

3 The setup file **15**

3.1 Working conditions . . . 16

3.2 Electron kinetics . . . 20

3.3 Chemistry . . . 27

3.4 Graphical user interface . . . 33

3.5 Output files . . . 36

3.5.1 Datasets in text format . . . 37

3.5.2 Datasets in HDF5 format . . . 42

4 Auxiliary input files **46**

4.1 LXCat files . . . 46

4.2 Property files . . . 49

4.3 Chemistry files . . . 50

5 Auxiliary functions **53**

5.1 Property functions . . . 53

5.2 Rate coefficient functions . . . 58

5.3 Other auxiliary functions . . . 68

5.4 Utilities . . . 69

6 Changelog (until 2026/07/07) **70**

References **78**

1 Introduction

Overall, LoKI-GM provides the combined chemical and transport description of plasma charged/neutral species, both in volume and surface phases, for user-defined working conditions: mixture compositions, pressure, dimensions and excitation features (glow (DC/HF), post-discharge, pulsed).

1.1 General features

LoKI-GM [10, 11] comprises two modules, that can run self-consistently coupled or as standalone tools:

- a Boltzmann solver (LoKI-B) for the electron Boltzmann equation [1, 2];
- a Chemical solver (LoKI-C) for the global kinetic model(s) of pure gases or gas mixtures.

LoKI-B solves the space independent form of the two-term electron Boltzmann equation (EBE) to calculate the isotropic and the anisotropic parts of the electron distribution function (the former usually termed the *electron energy distribution function*, EEDF), and the associated electron macroscopic parameters. LoKI-B applies to non-magnetised non-equilibrium LTPs, excited by DC/HF electric fields or time-dependent (non-oscillatory) electric fields (a new feature introduced in LoKI-B_v2.0.0, when LoKI-B runs independently) [2] from different gases or gas mixtures. The tool uses a stationary description for DC fields, a Fourier time-expansion description for HF fields, and a time-dependent description for time-varying fields.

LoKI-B was developed as a response to the need of having an electron Boltzmann solver easily addressing the simulation of the electron kinetics in any complex gas mixture (of atomic / molecular species), describing first and second-kind electron collisions with any target state (electronic, vibrational and rotational), characterized by any user-prescribed population. LoKI-B includes electron-electron collisions, it handles rotational collisions adopting either a discrete formulation or a more convenient continuous approximation, and it accounts for variations in the electron population caused by non-conservative events (ionisation and attachment) by adopting either a space-homogeneous exponential temporal growth or a time-constant exponential spatial growth of the electron density.

LoKI-C solves the system of zero-dimensional (volume average) rate balance equations for the most relevant charged and neutral species in the plasma. LoKI-C receives as input data the kinetic schemes for the gas/plasma/surface system under study, via an intuitive csv-like input file, and gives as output the particle densities of the different gas/plasma/surface species, the corresponding creation/destruction reaction rates, and the reduced electric field (and any related quantity, such as the discharge current or the discharge power-density). The tool uses several modules to describe the mechanisms (collisional, radiative and transport) controlling the creation/destruction of species, namely various transport models for the charged particles (ambipolar diffusion, effective diffusion, and Liberman and Lee's low-pressure model) and for the neutral particles (multi-component diffusion of species following Wilke's approach, and considering wall recombination). LoKI-C includes also a gas/plasma thermal model, for the self-consistent calculation of the gas temperature, and supports multi-component mean-field microkinetic mesoscopic models to handle surface kinetics in a fully coupled way with volume kinetics. The inclusion of several transport models and support for surface kinetics models are distinguishing features of LoKI-C compared with other global chemistry models.

The class of problem addressed by LoKI-GM, the physical models adopted and the algorithms used are described in the Reference Manual [12].

1.2 This user guide

The distribution of LoKI-GM includes MATLAB® `*.m` files together with data files required to define the problem under study and the physical properties of the gases.

This user guide begins with essential information for quickly obtaining and running LoKI-GM (see section 2). It then describes the files included in the LoKI-GM distribution, provides guidance on preparing the input file, and explains how the output data are structured.

The main components of the distribution are summarized below.

- The `Setup file` (see section 3) is the main input to LoKI-GM and serves as the primary user interface (UI) for interacting with the code.

  It contains the specification of the `Working conditions` (see section 3.1), the configuration of the `Electron kinetics` (see section 3.2) and the `Chemistry` (see section 3.3), including numerical settings. It also defines the data displayed in the `Graphical user interface` (see section 3.4) and the information written to the `Output files` (see section 3.5). Setup files can be written either in plain text or JSON format. Currently, JSON input is supported only for *electron kinetics simulations* (using LoKI-B as standalone tool). Support for the remaining simulation types available in LoKI-GM is under development.

- `Auxiliary input files` (see section 4) provide the fundamental data required to run the simulations. These files are organized into three categories:

  - `LXCat files` (see section 4.1), which contain electron-scattering cross sections;
  - `Property files` (see section 4.2), which define the properties of gases and their states;
  - `Chemistry files` (see section 4.3), which list the electron and heavy-species reactions that compose a kinetic mechanism.

  These files are either plain text files or JSON files.

- `Auxiliary functions` (see section 5) are MATLAB® `*.m` functions used internally by LoKI-GM to perform the simulations. They are organized into:

  - `Property functions` (see section 5.1), which define the properties of gases and states;
  - `Rate coefficient functions` (see section 5.2), which compute the rate coefficients of reactions included in the kinetic mechanism;
  - `Other auxiliary functions` (see section 5.3), which provide additional utilities, including tools to calculate working conditions that may evolve during the simulations.

- `Utilities` (see section 5.4), which contains several scripts to support the pre- and post-processing of data used by LoKI-GM, including tools to modify LXCat text files, plot two different EEDFs on the same MATLAB® graph, or plot the errors associated with the convergence cycles.

An HTML version of this user guide is also available. The corresponding files are located in the folder `[repository folder]/LoKI-GM_26.07/Documentation/html/` and can be accessed directly by opening the page `Index.html`.

The HTML documentation can also be accessed from the GUI-based input through the `Help` button (see section 2.4) or from MATLAB® using the command

```
web("../Documentation/html/Index.html", "-browser")
```

which opens the corresponding page in the system's default web browser.

1.3 How to contact us

After downloading and installing LoKI-GM, and especially if you intend to interact with us, you are invited to send a short message
to loki@tecnico.ulisboa.pt
with **subject** LoKI-GM
just giving your **name** and **affiliation**.

Also, you are much welcome to report any problem or bug you may find using the code, or to send any feedback with suggestions for further developments.

1.4 How to reference the code

LoKI-GM is the result of the efforts of the Portuguese group N-Plasmas Reactive: Modeling and Engineering (N-PRiME) of *Instituto de Plasmas e Fusão Nuclear* with *Instituto Superior Técnico*, the *Departamento de Física* of *Facultad de Ciencias* with *Universidad de Córdoba* and the group Elementary Processes in Gas Discharges (EPG) of the Eindhoven University of Technology. These groups decided to share the outcome of its research on code development with the members of the Low-Temperature Plasmas community.

When using the code in your work, please give proper credits to the main developers, by adding the following citations.

For LoKI-B
- Tejero A *et al.* *The LisbOn KInetics Boltzmann solver* 2019 *Plasma Sources Sci. Technol.* **28** 043001
- Tejero A *et al.* *On the quasi-stationary approach to solve the electron Boltzmann equation in pulsed plasmas* 2021 *Plasma Sources Sci. Technol.* **30** 065008
- Tejero A *et al.* *The LisbOn Kinetics Boltzmann solver*, DOI: https://doi.org/10.57859/ulisboa-istipfn.000062, URL: https://github.com/LoKI-Suite/LoKI-GM

For LoKI-GM
- Alves L L *et al.* *LoKI-GM: a global model tool for plasma chemistry studies* 2026 arXiv preprint, arxiv:2607.27234. Manuscript submitted for publication to *Plasma Sources Sci. Technol.* (under review)
- Alves L L *et al.* *The LisbOn Kinetics Global Model*, DOI: https://doi.org/10.57859/ulisboa-istipfn.000062, URL: https://github.com/LoKI-Suite/LoKI-GM

1.5 Acknowledgments

This work was funded by Portuguese FCT - Fundação para a Ciência e a Tecnologia, initially under project PTDC/FISPLA/1243/2014 (KIT-PLASMEBA), and currently under projects PSI.COM, UID/50010/2025, UID/PRR/50010/2025, UID/PRR2/50010/2025, and LA/P/0061/2020. Part of this work is supported by the Dutch Science Foundation NWO, and by ASML. We also thank D Boer and J van Dijk (Department of Applied Physics and Science Education, Eindhoven University of Technology, The Netherlands) for helpful suggestions in producing this guide.

2 Getting started

This section provides basic information enabling a user to quickly obtain and run LoKI-GM for the first time[1]

2.1 Requirements and installation

LoKI-GM is developed under MATLAB®, to benefit from its matrix-based architecture. The minimum requirements to run the code is a computer with an installation of MATLAB® (oldest recommended version R2024b; we cannot ensure that all the features of LoKI will work properly under different versions). The hardware requirements for LoKI are not different than those for MATLAB®. However, for complex gas mixtures or very fine energy grids, a larger RAM is recommended.

If you are reading this, you probably already have LoKI-GM installed. Nevertheless, the following summarizes the recommended ways to obtain the code.

Alternative installation methods are discouraged, as they may result in incomplete setups or cause some parts of the code to function incorrectly.

2.1.1 GitHub

You can obtain the latest version of the code from the github URL
`https://github.com/LoKI-Suite/LoKI-GM`
by downloading the `LoKI-GM_26.07` zip-file and extracting it to your local repository.

You can also clone the repository using git [13], particularly if you want to take part in the development of the code.

2.2 Directory Structure and Data Organization

The `LoKI-GM_26.07` folder contains

1. A subfolder `Documentation`, which includes the user guide (available in both PDF and HTML formats), the reference manual and relevant publications.
2. A subfolder `Code` containing
   (a) Several `'*.m'` files corresponding to the MATLAB® code, of which `'loki.m'` is the main file of LoKI-GM.
   (b) Two files to launch the GUI-based input for LoKI-B: the MATLAB® file `'InputGUILokib.m'`, which provides a user-friendly graphical user interface for LoKI-B setup and run; and the batch file `'InputGUILokib.bat'`, which allows launching the GUI-based input form a `cmd` window (or by double-clicking the corresponding `*.bat` file).
   (c) A subfolder `Input`, containing the input files required for the simulations, organised as follows
      i. Default configuration files in text format `'default_lokibc_setup.in'`, `'default_lokic_pulse_setup.in'`, `'default_lokib_setup.in'` and `'default_lokib_pulse_setup.in'`, respectively, for LoKI-GM (the first two files, for *steady-state simulations* and *quasi-stationary simulations*, respectively) and for LoKI-B as standalone tool (the latter two, for *electron kinetics stationary simulations* and *electron kinetics time-dependent simulations*, respectively) (see sections 2.4, 2.5 and 3).
      ii. Default configuration files in JSON format (only for simulations using LoKI-B as standalone tool)
      `'default_lokib_setup.json'` and `'default_lokib_pulse_setup.json'`, for *electron kinetics stationary simulations* and *electron kinetics time-dependent simulations*, respectively (see sections 2.4, 2.5 and 3).

[1] This user guide refers to the MATLAB-based LoKI-GM, which integrates the LoKI-B and LoKI-C codes. Note that the LisbOn KInetics Boltzmann solver is also available as a standalone open-source tool, offered in two versions: the MATLAB® implementation LoKI-B, and the C++ implementation LoKI-B++.

iii. A subfolder `Databases` with `'*.txt'` files, containing different properties (masses, energies of states, atomic/molecular constants, ...) for the gases used in the simulations.

iv. Several subfolders `Argon`, `CO`, ... `Nitrogen`, `Oxygen` with `'*.txt'` files, containing
- the data needed to perform simulations in different gases or gaseous mixtures: electron-scattering cross sections, usually obtained from the open-access website LXCat [14]; kinetic schemes; etc (see section 4);
- setup input files to run swarm simulations;
- setup input files to run chemistry simulations.

Subfolders `Argon`, `CO`, `CO2`, `Nitrogen`, `Oxygen` contain also `'*.json'` files with electron-scattering cross sections (obtained from the *demo* version of LXCat 3.0 [15]) and input setups for swarm simulations.

(d) A subfolder `PropertyFunctions`, with several `'*.m'` auxiliary functions for calculating some predefined distribution of states (Boltzmann, Treanor, ...), the statistical weights of states due to their degeneracy, the energy of the states according to some models, transport parameters and thermal conductivities of different species, etc (see section 5.1).

(e) A subfolder `RateCoeffFunctions`, organized into one-level subfolders with self-explanatory names, with several `'*.m'` auxiliary functions for calculating the values of the different rate coefficients used in LoKI-C (see section 5.2).

(f) A subfolder `OtherAuxFunctions`, with several `'*.m'` auxiliary functions, *e.g.* to calculate some working conditions (see section 5.3).

(g) A subfolder `Utilities`, with several scripts to help process or parse data, etc. (see section 5.4).

(h) A subfolder `Output` (eventually), where LoKI will write the output files resulting from the simulations (see section 3.5).

2.3 Ontology

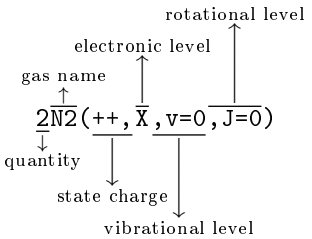
Before presenting LoKI simulations and detailing the syntax of LoKI input files, we describe the ontology and associated syntax rules used to define individual species and the gaseous systems considered in the simulations.

2.3.1 Species description

The ontology for species description in LoKI corresponds essentially to the one currently adopted in LXCat [14]. Note, however, that LoKI now supports the new ontology adopted in the *demo* version of LXCat 3.0 [15] for describing species and reactions when using JSON files. This parsing scheme is expected to continue evolving. Work is in progress to consolidate the transcription of LXCat data between versions 2.0 and 3.0, and the corresponding data parsing in numerical codes.

LoKI handles two different types of species: electrons and heavy species (atomic or molecular). All species are represented by a **string without any white-space characters** and when specified in an electron collision **they are separated by plus signs** (`+`).
The representation of these species follows the syntax rules below:

- **Electrons:** can be represented by either characters `e` or `E`, which can appear multiplied by an integer, *e.g.* `2e` which is equivalent to `e + e`.
- **Heavy species:** refer to the different atomic or molecular species used in the simulations. **For electron-collision target species, it is mandatory to provide information on its configuration (see below)**.
  The scheme below shows the general structure to adopt, when representing the configuration of a particular species:



where

- quantity: is an **optional** field composed only by integers. It represents the number of species to be considered.
- gas name: is a **mandatory** field composed by letters (English alphabet), integers and underscore characters (_). It can not start with an integer and is case sensitive.
- state charge: is an **optional** field composed by plus (+) and minus (-) characters. It represents the charge of an ionic state and has to terminate with a comma (,).
- electronic level: is a **mandatory** field composed by letters (English alphabet), integers, underscore characters (_), plus characters (+), minus characters (-), asterisk characters (*), apostrophe characters ('), square brackets ([]) and periods (.).
- vibrational level: is an **optional** field composed by letters (English alphabet), integers, underscore characters (_), plus characters (+), minus characters (-) and asterisk characters (*). It has to start with the string "`,v=`".
- rotational level: is an **optional** field composed by letters (English alphabet), integers, underscore characters (_), plus characters (+), minus characters (-) and asterisk characters (*). It has to start with the string "`,J=`" and it must necessarily come after the vibrational level field.

IMPORTANT NOTE:
The asterisk character ($\star$) is supported **only** in the setup file, **not in the LXCat auxiliary input files**.

- **Special species:** refer to species, different from **electrons** and **heavy-species**, that can also be included in the description of chemical reactions. The following lists the different special species recognised by LoKI:
  - `wall`: when found as a reactant of a chemical reaction, it identifies the reaction as a transport process, *i.e.* the diffusion of the other reactant species and its collision with the reactor wall;
  - `gas`: when found as both a reactant and a product of a chemical reaction, it identifies this reaction as a "gas stabilised process", *i.e.* a reaction where any atomic/molecular state present in the system can act as a stabilising body (example: `A + gas -> B + gas`);
  - **surface species:** refer to heavy species (atomic or molecular) adsorbed to the walls (physisorbed or chemisorbed), as well as to vacant sites on the surface, used in simulations when the chemistry scheme includes a surface kinetics. The representation of these species follows the same syntax rules as for the (gas) heavy species, adopting a gas name that **MUST** start with `wall_`.
    Some examples of surface-species description are given below.
    * `wall_empty(F)` $\cdots$ a physically vacant surface site;
    * `wall_empty(S)` $\cdots$ a chemically vacant surface site;
    * `wall_N(F)` $\cdots$ a nitrogen atom physically adsorbed on the surface;
    * `wall_O(S)` $\cdots$ an oxygen atom chemically adsorbed on the surface.

    This notation represents a *recommended* (though not mandatory) ontology for describing surface species, based on the general format
    `wall_species(state)`
    where the only strict requirement is the use of the prefix `wall_`.
    Alternative ontologies may also be adopted, for example
    `wall_state(species)`,
    but these can limit compatibility with utility scripts such as `checkChemFilesStoichiometry`, which performs stoichiometric consistency checks on chemistry files describing the kinetic scheme (see section 3.3).

2.3.2 Gaseous system description

LoKI handles simulations in any atomic / molecular gas mixture, considering collisions with target states featuring internal degrees of freedom (electronic, vibrational and rotational), organized as a system of "Russian puppets" [2]. The populations of these states are self-consistently calculated when using the LoKI simulation tool, and are prescribed by the user when running LoKI-B as standalone tool. Table 1 exemplifies the system of states for a N_2-N-He gas mixture. Here,

[2] LoKI handles also surface states, when the plasma kinetics is extended to include plasma-surface interactions.

Table 1: Example of a gas mixture (childless states are displayed in red).

	Gases	States		
System	**N_2**	**N_2(X)** Parent of → ← Childs of	**N_2(X,v=0)** Parent of → ← Childs of	**N_2(X,v=0,J=0)**
				N_2(X,v=0,J=1)
				N_2(X,v=0,J=2)
				⋮
			N_2(X,v=1)	
			N_2(X,v=2)	
			⋮	
		N_2(A)		
		N_2(B)		
		⋮		
	N	**N(^{4}S)**		
		N(^{2}D)		
		N(^{2}P)		
		⋮		
	He	**He(1^1S)**		
		He(2^3S)		
		He(2^1S)		
		⋮		
	⋮			

- **System:** is the ensemble of all the gases present in the simulation ($N_2 \cup N \cup He \cup \cdots$).
- **Gas:** refers to each type of atom or molecule present in the simulation (N_2, N, He, ...). Each gas k has mass M_k and individual density N_k, being composed by several electronic / vibrational / rotational levels.
- **State:** specific configurations of electronic (i), vibrational (v) and/or rotational (J) levels belonging to a given gas (e.g, N_2(X), N_2(X,v=0), N_2(X,v=0,J=2), N(^{4}S), He(2^3S), ...). Each gas k is composed by several electronic states k_i with absolute density N_{k_i}; each k_i-state can include several vibrational states k_{i_v} with absolute density $N_{k_{i_v}}$, each one composed by several rotational states $k_{i_{v_J}}$ with absolute density $N_{k_{i_{v_J}}}$.
- **Parent (state):** state of a certain gas comprising a set of child states. Parent states can be electronic or vibrational, but not rotational (*e.g.*, N_2(X) is the parent of N_2(X,v=0,1,2,...)). Each parent level comprises all of its children.
- **Child (states):** ensemble of states that share the same parent. Child states can be vibrational or rotational, but not electronic (*e.g.*, N_2(X,v=0,J=0,1,2,...) are the children of N_2(X,v=0)).
- **Sibling (states):** relation between the states that share the same parent (*e.g.*, N_2(X,v=0,1,2,...) are siblings).
- **Childless (states):** ensemble of the innermost states (without children) (*e.g.*, N_2(X,v=0,J=0) is childless).

The previous terminology, allows introducing several definitions and relations for the particle densities with the different species (gases / states) in the system.

- **Total gas density:** total particle density of the heavy species in the system, evaluated as:

$$N = \frac{p}{k_B T_g} \quad , \tag{1}$$

  where p is the total pressure, k_B is the Boltzmann constant and T_g is the gas temperature (assumed equal for all k-gases in the system).
  In the code, this quantity is stored in variables labeled with `totalGasDensity`.

- **Individual gas density:** total particle density of a gas in the system. For gas k this is given by N_k, with $\sum_k N_k = N$.
  In the code, this quantity is stored in variables labeled with `individualGasDensity`.

- **Gas fraction:** fraction of a gas in the system. For gas k this is given by $\chi_k \equiv N_k/N$.

- **Absolute density:** particle (absolute) density of a state. For a generic state i of gas k this is represented by N_{k_i}.
  In the code, this quantity is stored in variables labeled with `abs`.

- **Population:** density of a state relative to its siblings. For a generic state i of gas k this is represented by $\xi_{k_i} \equiv N_{k_i}/N_{\text{parent of } k_i} = N_{k_i}/(\sum_{i\in\text{siblings}} N_{k_i})$.

- **Density:** particle density of a state relative to the total gas density. For a generic state i of gas k this is represented by $\delta_{k_i} \equiv N_{k_i}/N$.
  In LoKI-B, this quantity is stored in variable `density`.

From the previous definitions, it is straightforward to obtain the following normalisation conditions

$$\sum_{k\in\text{system}} \chi_k = 1 \tag{2a}$$

$$\sum_{i\in\text{siblings}} \xi_i = 1 \tag{2b}$$

$$\sum_{i\in\text{childless}} \delta_i = 1 \quad , \tag{2c}$$

concluding also that the following relationships hold

$$\delta_i = \xi_i \delta_{i\text{'s parent}} \tag{3a}$$

$$\delta_{k_i} = \xi_{k_i} \chi_k \quad . \tag{3b}$$

LoKI uses SI units for all physical quantities, except the energies that are expressed in eV (electron-volt), the reduced fields that are expressed in Td (Townsend; 1 Td = 10^{-21} V m^2), and the flows that are expressed in sccm (standard cubic centimeter per minute).

2.4 Starting LoKI-GM

LoKI-GM runs upon calling the MATLAB® function `loki(setupFile)`. The end user interacts with the code by specifying a particular "setup" for the simulation. This setup is sent to the `loki()` function through the *required* input argument `setupFile`. As mentioned in section 2.2, the setup files should be located in `[repository folder]/LoKI-GM_26.07/Code/Input/` with

- `.in` extension (this is just a recommendation in order to keep the input folder organised; the `.in` setup files are just plain text files);

- `.json` extension (only for simulations using LoKI-B as standalone tool; in this case it is important to use this extension, for the parsing to work properly).

The distribution of LoKI-GM includes some default configuration files (`'default_lokibc_setup.in'`, `'default_lokib_setup.in'`, `'default_lokic_pulse_setup.in'`, `'default_lokib_pulse_setup.in'`, `'default_lokib_setup.json'` and `'default_lokib_pulse_setup.json'`), for the benefit of the user. By using one of these files, it is very easy to make a first run of the code, just following the sequence of steps below. For example, using the `'default_lokibc_setup.in'` setup file:

1. Open MATLAB®
2. Navigate to the `Code` folder of your local copy of the repository:
   `>> cd [repository folder]/LoKI-GM_26.07/Code/`
3. Execute the following command in the MATLAB® command window:
   `>> loki('default_lokibc_setup.in')`
4. The graphical user interface (GUI) should appear showing the solution(s) for the default setup file, represented in figure 1

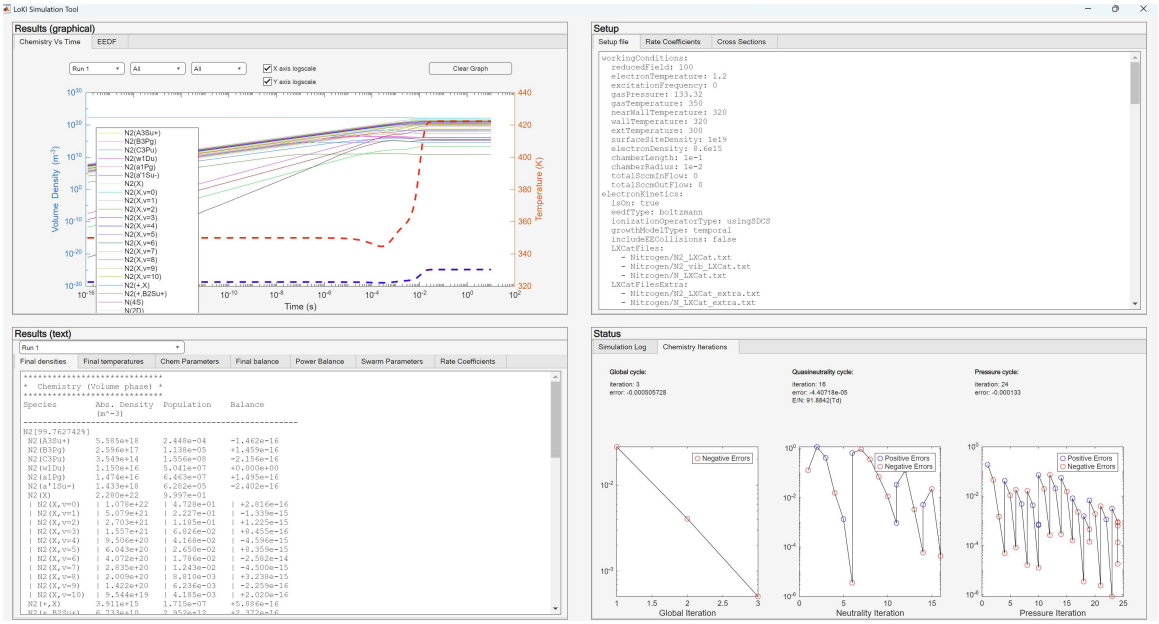


Figure 1: Graphical user interface output, for the default setup-file of LoKI-GM (stationary coupling between LoKI-B and LoKI-C).

Congratulations! You have now concluded your first LoKI-GM simulation.

To run LoKI-B, a similar procedure should be followed, for example using the `'default_lokib_setup.in'` setup file, to obtain the GUI represented in figure 2.

Additionally, LoKI-B can be **configured and executed** through a GUI-based input by using one of the following options:

- from the MATLAB® command window:
  `>> InputGUILokib`
- from a `cmd` window (or by double-clicking the corresponding `*.bat` file)
  `>> InputGUILokib.bat`

In both cases, the input-GUI opens as shown in figure 3.

Currently, GUI-based input is available only for LoKI-B; however, future developments will extend this capability to LoKI-GM.

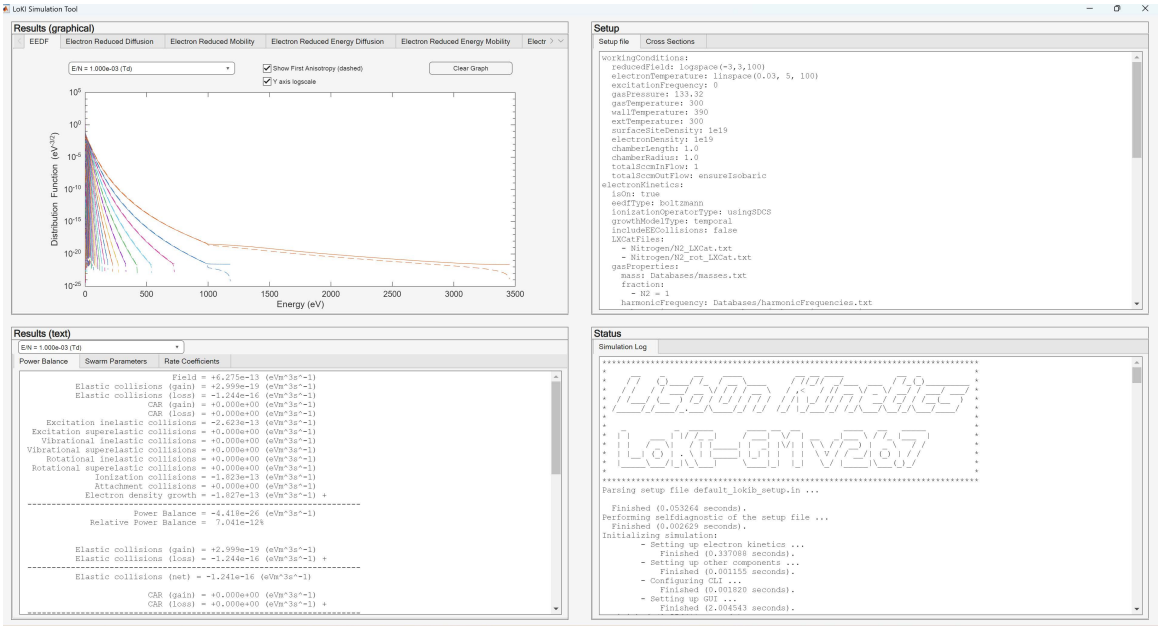


Figure 2: Graphical user interface output, for the default setup-file of LoKI-B (stationary simulations).

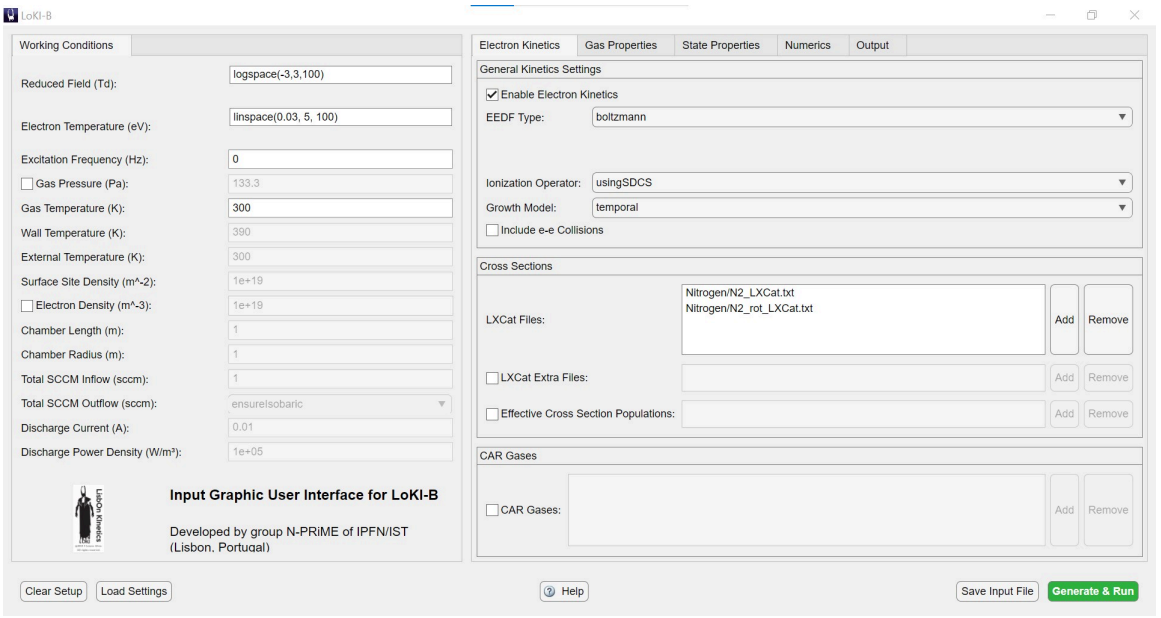


Figure 3: Graphical user interface to configure and execute LoKI-B.

In order to run the code for different simulations, users just have to call the `loki()` function with a different setup file, or a modified version of the default ones. The details on how to configure the setup file according to your needs are given in section 3.

2.5 Simulation types available in LoKI-GM

LoKI-GM supports the following types of simulations:

- *Steady-state simulations*, in which LoKI-C and LoKI-B are run in a coupled manner until steady-state is reached.
  In this case, the input file should be configured as follows:
  - `workingConditions`, specifying:
    - `reducedField` or `electronTemperature`
    - `gasPressure`, `gasTemperature`, `chamberLength`, `chamberRadius`, `totalSccmInFlow`, and `totalSccmOutFlow`
    - `electronDensity` and, optionally, `dischargeCurrent` or `dischargePowerDensity`
  - `electronKinetics`, specifying:
    - `isOn: true`
    - `eedfType: boltzmann` or `eedfType: prescribedEedf`, consistent with the choice in `workingConditions`
    - the relevant data for the selected gas or gas mixture
  - `chemistry`, specifying:
    - `isOn: true`
    - `convergenceParameter: electronDensity`, `dischargeCurrent`, or `dischargePowerDensity`, consistent with the choices in `workingConditions`
    - the relevant chemistry for the selected gas or gas mixture
    - a non-zero value for `dischargeTime`
    - `postDischargeTime: 0`
- *Post-discharge simulations*, which start with a coupled LoKI-C/LoKI-B steady-state calculation, followed by a standalone LoKI-C simulation at $E/N = 0$ to describe the time-dependent post-discharge.
  The input file should be configured as for *steady-state simulations*, with the following modification:
  - `chemistry`:
    - set a non-zero value for `postDischargeTime`
- *Quasi-stationary simulations*, in which LoKI-C runs as a standalone tool using precomputed lookup tables to obtain electron swarm parameters, rate coefficients, and power transfer as functions of the electric field (*Local Field Approximation*, LFA) or electron mean energy (*Local Mean Energy Approximation*, LEA).
  In this case, the input file should be configured as follows:
  - `workingConditions`, specifying:
    - a time-dependent function for `reducedField`
    - `gasPressure`, `gasTemperature`, `chamberLength`, `chamberRadius`, `totalSccmInFlow`, and `totalSccmOutFlow`
    - `electronDensity`
  - `electronKinetics`:
    - `isOn: false`
  - `chemistry`, specifying:
    - `isOn: true`
    - `lookupTables` with files for `swarm`, `rateCoeff`, and `power`, and `lookUpMethod: localField` or `lookUpMethod: localEnergy`
    - the relevant chemistry for the selected gas or gas mixture

    * `stateProperties`, with ion populations consistent with `electronDensity` (to ensure charge neutrality)

- *Electron kinetics simulations* (including swarm analysis), in which case LoKI-B runs as a standalone tool. In this case, the input file should be configured as follows:
  - `workingConditions`, specifying:
    * `reducedField` or `electronTemperature`, or a time-dependent function for `reducedField`
    * `gasTemperature` and, optionally, `gaspressure` and `electronDensity`
  - `electronKinetics`, specifying:
    * `isOn: true`
    * `eedfType: boltzmann` or `eedfType: prescribedEedf`, consistent with the choice in `workingConditions`
    * the relevant data for the selected gas or gas mixture
  - `chemistry`, specifying:
    * `isOn: false`

  Depending on whether `reducedField` is specified as discrete value(s) or as a time-dependent function, LoKI-B simulations are classified as *electron kinetics stationary simulations* or *electron kinetics time-dependent simulations*, respectively.

  Internally, LoKI-GM uses the following flags to configure the simulations (see `Chemistry.m` function):

  - `imposeQuasiNeutrality`, a Boolean property that determines whether quasi-neutrality is enforced during temporal integration.
    At present, `imposeQuasiNeutrality=true` is applied at every time-step in *quasi-stationary simulations* and *post-discharge simulations* (in the latter case, only between `dischargeTime` and `postDischargeTime`; see section 3.3). In contrast, quasi-neutrality is not enforced during the temporal integration of *steady-state simulations*; in this case, charge neutrality is only guaranteed upon convergence to steady-state (see Reference Manual [12]).
  - `electronKineticsDependence`, a string property that specifies how the electron kinetics is treated during temporal integration.
    The available options are `steadyState`, `quasiStationary` and `postDischarge`.
  - `solveEedf`, a Boolean property that specifies whether the EEDF is solved self-consistently when solving the rate balance equations during the temporal integration of *post-discharge simulations*.
    Currently, `solveEedf=true` is only supported when `electronKineticsDependence` = `postDischarge`.

3 The setup file

The setup file is the main input to LoKI-GM and serves as the primary user interface (UI) for interacting with the code. Setup files can be written either in plain text or JSON format. Currently, JSON input is supported only for *electron kinetics simulations* (using LoKI-B as standalone tool). Support for the remaining simulation types available in LoKI-GM is under development.

In this file, the user can provide / select relevant information, namely

- about physical parameters, *e.g.*
  - working conditions
  - kinetic data and schemes
  - atomic and molecular data
  - species populations
- about physical models, *e.g.*
  - mode of energy-sharing in ionisation collisions
  - type of growth model for the electron density
  - inclusion of electron-electron collisions
  - inclusion of thermal model
- about numerical details, *e.g.*
  - energy grid for LoKI-B
  - nonlinear algorithm(s) to adopt in the simulations
  - convergence criteria for iterative cycles
  - integration times

The setup **text** file is an **indentation structured file**, *i.e.* it follows the "Off-side rule" [16] like in some programming languages such as Python. This means that it is **mandatory** that all the subfields within a field share the same level of indentation, otherwise the behaviour of the code is undefined.

The **setup text file** is organised according to the following syntax rules.

- The setup file can include comments, signaled by the character `%`.
- Some of the fields in the setup file can accept either one or more values.
  When only one value is to be specified, it can be given in the same line of the field, right after the colon, for example:

  ```
  mass: Databases/masses.txt
  ```

  Alternatively, when more than one value are to be specified, one can use several lines starting with a dash, for example

  ```
  LXCatFiles:            % cross section files
      - Nitrogen/N2_LXCat.txt
      - Nitrogen/N2_rot_LXCat.txt
  ```

- The setup text file can call either `*.txt` or `*.json` auxiliary input files.

The **setup JSON file** (only for simulations using LoKI-B as standalone tool) is organised similarly to the setup text file, following the standard JSON syntax rules. The following should be noted:

- The calling of functions within the setup JSON file is passed with quotes "." .
- The setup JSON file can call either `*.txt` or `*.json` LXCat files.

Note that the value(s) of certain fields in the setup file can be defined specifying an "auxiliary input file" or a "property function".
All the "auxiliary input files" (*i.e.* LXCat files, property files, chemistry files) specified in the setup file should be located inside the input folder (`[repository folder]/LoKI-GM_26.07/Code/Input`), eventually organised in subfolders with self-explanatory names (*e.g.* `Nitrogen`, `Databases`; see section 2.2). In this case, the name of the subfolder should be specified in the setup file (see examples above for `mass` and for `LXCatFiles`). For more information about auxiliary input files see section 4.
All the "property functions" specified in the setup file should be located in the property functions folder (`[repository folder]/LoKI-GM_26.07 /Code/PropertyFunctions`; see section 2.2). For more information about property functions see section 5.1.

The setup file is organised into five different main fields, with self-explanatory designations:

- `workingConditions`
- `electronKinetics`
- `chemistry`
- `gui`
- `output`.

The next subsections present these fields in detail, using a **setup text file** for nitrogen as example.

3.1 Working conditions

In this field of the setup file the user can specify the values of the different working parameters for the simulation. Code 1 shows the section of the `'default_lokibc_setup.in'` file that handles this configuration.

```
% --- configuration of the working conditions ---
workingConditions:
  reducedField:  100                        % in Td
  electronTemperature: 1.2                  % in eV
  excitationFrequency: 0                    % in Hz
  gasPressure: 133.32                       % in Pa
  gasTemperature: 350                       % in K (average gas temperature)
  nearWallTemperature: 320                  % in K (near wall temperature)
  wallTemperature: 320                      % in K (wall temperature)
  extTemperature: 300                       % in K (external temperature)
  surfaceSiteDensity: 1e19                  % in m-2 (used for surface kinetics)
  electronDensity: 8.6e15                   % in m-3
  chamberLength: 1e-1                       % in m
  chamberRadius: 1e-2                       % in m
  % dischargeCurrent: 4e-2                    % in A
  % dischargePowerDensity: 2.7e5              % in W m-3
  totalSccmInFlow: 0                        % in sccm
  totalSccmOutFlow: 0                       % a number (in sccm) or a model: totalSccmInFlow, ensureIsobaric
```

Code 1: Configuration of the working conditions for a steady-state simulation

IMPORTANT NOTES:

- The `reducedField` corresponds to E/N, where
  - $E = E_p$ is the amplitude of the applied electric field, in a DC case;
  - $E = E_p/\sqrt{2}$ is the root-mean-square (RMS) amplitude of the applied electric field, in a HF case.

The `reducedField` is a mandatory parameter if `eedfType: boltzmann`. Otherwise this parameter can be omitted, but if defined it can never be empty.
The `reducedField` can correspond to

- **one value or an array of values**, as in the case of (see section 2.5)
  - *steady-state simulations* and *post-discharge simulations*, when running LoKI-GM.
    Here, `reducedField` corresponds to the *initial value* of E/N, to be updated during the *neutrality cycle*, when choosing `boltzmann` as `eedfType` (see section 3.2 and Reference Manual [12]);
  - *electron kinetics stationary simulations*, when running LoKI-B as standalone tool.
- **the name of a pulse function (to obtain E/N (in Td) *vs.* t)**, as in the case of (see section 2.5)
  - *quasi-stationary simulations*, when running LoKI-C as standalone tool;
  - *electron kinetics time-dependent simulations*, when running LoKI-B as standalone tool.

  Codes 2 and 3 present examples of configurations of the `reducedField` for an electric-field pulse, in the cases of *quasi-stationary simulations* and *electron kinetics time-dependent simulations*, respectively.

```
% --- configuration of the working conditions ---
workingConditions:
  reducedField: pulse@reducedFieldFromCurrentFile, 1e-12, 1e-6, logspace, 200, CO2/pulse/currentPulseFile.txt
    % pulse@functionName, firstStep, finalTime, samplingType, samplingPoints, additionalParameters
```

Code 2: Configuration of the reducedField for an electric-field pulse in a quasi-stationary simulation

```
% --- configuration of the working conditions ---
workingConditions:
     reducedField: pulse@reducedFieldPulse, 1e-18, 2e-6, logspace, 100, 1e-6, 50
    % reducedField: pulse@reducedFieldRamp, 1e-18, 2e-6, logspace, 100, 1e-6, 25
    % pulse@functionName, firstStep, finalTime, samplingType, samplingPoints, additionalParameters
```

Code 3: Configuration of the reducedField for an electric-field pulse in an electron kinetics time-dependent simulation

Here, the field parameters are, in order,

- the name of the pulse function (providing E/N (in Td) *vs.* t)
  - for *quasi-stationary simulations*, $E/N(t)$ is calculated from a time-dependent function of the reduced electric field or any related quantity, such as the discharge current or the discharge power-density
  - for *electron kinetics time-dependent simulations*, the current distribution of LoKI-B includes the following pulse functions (see section 5.3):
    - `pulse@reducedFieldPulse`, corresponding to

$$\frac{E(t)}{N}(\text{Td}) = \frac{E_0}{N}\sqrt{\frac{t}{\tau_{\text{rise}}}}\,\exp\left(-\frac{t}{\tau_{\text{rise}}}\right) \quad . \tag{4}$$

    - `pulse@reducedFieldRamp`, corresponding to

$$\frac{E(t)}{N}\ (\text{Td}) = \begin{cases} \frac{E_0/N}{\tau_{\text{rise}}}t\,, & 0 \le t \le \tau_{\text{rise}} \\ \frac{E_0}{N}\,, & t > \tau_{\text{rise}} \end{cases} \quad . \tag{5}$$

- the first time step (in s) for the simulation output
- the final time (in s) for the simulations and the simulation output
- the type of sampling for the time steps used in the output (`logspace` or `linspace`)
- the number of sampling points in the simulation output
- any additional parameters (optional) assisting the construction of the pulse function, *e.g.*
  - the values of the discharge current as a function of time, for *quasi-stationary simulations*;
  - the values of variables τ_{rise} and E_0/N in equations (4) and (5), for *electron kinetics time-dependent simulations*.

Note the following, **when adopting an electric-field pulse**:

- the `excitationFrequency` **MUST** be set to zero;
- the simulation runtimes can rapidly increase, especially for very long pulse-times, and/or high gas pressures, and/or a very large number of sampling points. User discretion is advised, when defining these parameters (the set of working conditions presented above for the `reducedField` is a good compromise example);
- users should be aware that keeping the graphical user interface activated (see section 3.4) can also degrade the simulation runtimes;
- it is recommended to adopt a constant energy grid (see section 3.2), chosen as to ensure the correct description of the EEDF for both the minimum $E(t)/N$ value in the simulations (considering low threshold mechanisms), and the maximum $E(t)/N$ value in the simulations (considering the bulk-to-tail drop in the EEDF);
- it is recommended to activate the optional parameters for the ode solver adopted in the `temporalIntegration` algorithm (see section 3.2 and Reference Manual [12]): absolute tolerance, relative tolerance, and maximum step-size. Users should understand the role of these parameters before activating them in the simulations.

- The `electronTemperature` corresponds to the electron temperature T_e.
  When running LoKI-GM for *steady-state simulations*, `electronTemperature` corresponds to the *initial value* of T_e, to be updated during the *neutrality cycle*, when choosing `prescribedEedf` as `eedfType` (see section 3.2 and Reference Manual [12]).
  The `electronTemperature` is a mandatory parameter if `eedfType: prescribedEedf`. Otherwise this parameter can be omitted, but if defined it can never be empty.

- The user can choose simulations that solve the EBE or that adopt a generalized Maxwellian EEDF (see section 3.2 and Reference Manual [12] for details on how to select the type of simulation for the EEDF):
  - the parameter `reducedField` is activated when the "boltzmann" simulation is selected;
  - the parameter `electronTemperature` is activated when the "prescribedEedf" simulation is selected.

- The `excitationFrequency` corresponds to the linear oscillation frequency $\omega/(2\pi)$.
  In the case of *electron kinetics simulations*, when running LoKI-B as standalone tool, `excitationFrequency` can be one value or an array of values.

  The `excitationFrequency` is a mandatory parameter.

- The `gasPressure` corresponds to the gas pressure p.
  In the case of *electron kinetics simulations*, when running LoKI-B as standalone tool, `gasPressure` can be one value or an array of values.

  For LoKI-GM and LoKI-C simulations, the `gasPressure` is a mandatory parameter.
  For LoKI-B simulations, the `gasPressure` is a mandatory parameter when `includeEECollisions` = `true` (see section 3.2) or/and `excitationFrequency` is different than zero.

- The `gasTemperature` T_g corresponds to (i) the **initial value of the average gas temperature**, if the thermal model is activated; (ii) the **value of the constant average gas temperature** prescribed by the user, if the thermal model is not activated (see section 3.3 and Reference Manual [12]).
  In the case of *electron kinetics simulations*, when running LoKI-B as standalone tool, `gasTemperature` can be one value or an array of values.

  The `gasTemperature` is a mandatory parameter.

- The `nearWallTemperature` corresponds to the **temperature near the wall** T_{nw}, to be adopted if the thermal model is not activated (see section 3.3 and Reference Manual [12]).

- The `wallTemperature` corresponds to the **temperature at the wall** T_w, to be used as *wall* boundary condition of the thermal model, if activated (see section 3.3 and Reference Manual [12]).

  The `wallTemperature` is a mandatory parameter if the thermal model is activated, adopting the `wall` boundary condition. In other cases, this parameter is optional.

- The `extTemperature` T_{ext} corresponds to the **external temperature**, to be used as *external* boundary condition of the thermal model, if activated (see section 3.3 and Reference Manual [12]).

  The `extTemperature` is a mandatory parameter if the thermal model is activated, adopting the `external` boundary condition. In other cases, this parameter is optional.

- The `surfaceSiteDensity` corresponds to the total density of surface sites ($[F]_{\text{surface}} + [S]_{\text{surface}}$ in m^{-2}; see Reference Manual [12], for use in a surface kinetics.

- The `electronDensity` corresponds to the average electron density n_e, taken as: (i) an **initial value** in the case of *steady-state simulations*, when the `convergenceParameter` is the `dischargeCurrent` or the `dischargePowerDensity` (see section 3.3 and Reference Manual [12]); (ii) an **initial value** in the case of *quasi-stationary simulations*, when the $E/N(t)$ data is obtained from time-dependent functions of the `dischargeCurrent` or the `dischargePowerDensity`; (iii) a **constant value** in other cases.
  In the case of *electron kinetics simulations*, when running LoKI-B as standalone tool, `electronDensity` can be one value or an array of values.

  For LoKI-GM and LoKI-C simulations, the `electronDensity` is a mandatory parameter.
  For LoKI-B simulations, the `electronDensity` is a mandatory parameter when `includeEECollisions` = `true` (see section 3.2).

- The `chamberLength` corresponds to the **full** length L of the discharge (for example between two electrodes) and the `chamberRadius` corresponds to the discharge radius R.

  The case of a pure cylindrical geometry ($L \gg R$) is chosen by setting `chamberLength` to zero.
  The case of a pure slab geometry ($R \gg L$) is chosen by setting `chamberRadius` to zero.

  The `chamberRadius` is a **mandatory** parameter if the thermal model is activated. Recall that this model considers a parabolic profile of the gas temperature in the radial direction only (see Reference Manual [12]), hence LoKI ignores the value of the parameter `chamberLength` in these calculations.

  Both, the `chamberRadius` and the `chamberLength` are **mandatory** parameters when using the rate coefficient functions `inFlow` and `outFlow`, to estimate the plasma volume (see section 5.2).

  At least one of the parameters `chamberRadius` or `chamberLength` are **mandatory** when defining a surface kinetics, to calculate the discharge area-to-volume ratio.

- The `dischargeCurrent` $I = en_e\mu_e E\pi R^2$ (in A; μ_e is electron mobility) is an optional input parameter to characterize one of the supported experimental conditions for the chemical kinetics.

- The `dischargePowerDensity` $P/V = e\Theta_E n_e$ (in W m^{-3}; Θ_E is the power transferred from the electric field to one electron) is an optional input parameter to characterize one of the supported experimental conditions for the chemical kinetics.

- The `totalSccmInFlow` is the total input flow of the gas mixture $Q = v_g\pi R^2 \times (10^{-6}/60) \times (p/p_{\text{ref}})(T_{\text{ref}}/T_g)$ (in sccm), where v_g is the gas velocity in m s^{-1}.

  The `totalSccmInFlow` is a mandatory parameter for LoKI-C.

- The `totalSccmOutFlow` is the total output flow of the gas mixture, and it can be a number (in sccm), or one of the following models: `totalSccmInFlow` or `ensureIsobaric` (see more details in Reference Manual [12]).

  The `totalSccmOutFlow` is a mandatory parameter for LoKI-C.

- When **running LoKI-B as standalone tool**, the parameters `reducedField`, `electronTemperature`, `excitationFrequency`, `gasPressure`, `gasTemperature` and `electronDensity` can be **specified either as a single value, corresponding to a single simulation, or as an array of values, allowing multiple simulations to be performed simultaneously for the prescribed parameter range**. Arrays of values can be defined using any expression supported by MATLAB® (e.g. `logspace(-4,1,100)`, `linspace(0,100,10)`, `[0 10 20 50]`). Expressions using the colon operator (`:`) are not supported (for example, `1:10`).
  Note that GUI support to multiple simulations in LoKI-B is currently limited to simultaneous variations of `reducedField` / `electronTemperature` and `gasTemperature` (see section 3.4).

Note further that HDF5 support to multiple simulations in LoKI-B is currently limited to *electron kinetics stationary simulations* (see section 3.5.2).

- When **running LoKI-B as standalone tool**, not all the parameters in Code 1 are used. Indeed, the `gasTemperature` T_g is needed to calculate the elastic and the CAR operators (see Reference Manual [12]) whereas the `gasPressure` p is required to calculate the total gas density only in the particular case of HF excitations (for obtaining the reduced frequency ω/N) and/or if electron-electron collisions are considered in the calculations (for obtaining the ionisation degree n_e/N, in which case it is also necessary to provide the `electronDensity` n_e). Moreover, the parameters `nearWallTemperature`, `wallTemperature`, `extTemperature`, `surfaceSiteDensity`, `chamberLength`, `chamberRadius`, `dischargeCurrent`, `dischargePowerDensity`, `totalSccmInFlow` and `totalSccmOutFlow` are not used in LoKI-B simulations.
  The user can comment the line(s) of Code 1 that specify the value(s) of parameter(s) not used in the simulations. Alternatively these lines can be kept uncommented, but in this case **it is mandatory to specify the values of the corresponding parameters**, regardless the fact that they are not used in the simulations (the values of these parameters will have no impact in the final results).

- Again, the code uses SI units for all physical quantities, except the energies that are expressed in eV (electron-volt), the reduced fields that are expressed in Td (Townsend; 1 Td = 10^{-21} V m^2) and the flows that are expressed in sccm (standard cubic centimeter per minute).

3.2 Electron kinetics

In this field of the setup file the user can define all the details related to the electron kinetics, *i.e.* with the specific configuration of LoKI-B. Code 4 shows the section of the `'default_lokibc_setup.in'` file that handles this configuration. Note that in *quasi-stationary simulations* this field is either absent or deactivated.

```
% --- configuration of the electron kinetics ---
electronKinetics:
  isOn: true              % true or false (to activate or deactivate the electron kinetics)
  eedfType: boltzmann     % boltzmann or prescribedEedf (generalized expression Maxwellian/Druyvesteyn)
%   shapeParameter: 1     % prescribedEedf shape parameter from 1 (Maxwellian) to 2 (Druyvesteyn)
  ionizationOperatorType: usingSDCS  % conservative, oneTakesAll, equalSharing or usingSDCS
  growthModelType: temporal          % temporal or spatial
  includeEECollisions: false         % true or false (to include / remove e-e collisions)
  LXCatFiles:                        % cross section files
    - Nitrogen/N2_LXCat.txt
    - Nitrogen/N2_vib_LXCat.txt
%    - Nitrogen/N2_rot_LXCat.txt
    - Nitrogen/N_LXCat.txt
  LXCatFilesExtra:                   % extra cross section files
    - Nitrogen/N2_LXCat_extra.txt
    - Nitrogen/N_LXCat_extra.txt
%  effectiveCrossSectionPopulations: % see doc
%    - Nitrogen/N2_effectivePop.txt
  CARgases:                         % gases for which CAR is activated
    - N2
  gasProperties:                % properties of the gases (S.I. Units)
    mass: Databases/masses.txt
    fraction:
      - N2 = 1
      - N = 0
    harmonicFrequency: Databases/harmonicFrequencies.txt
    anharmonicFrequency: Databases/anharmonicFrequencies.txt
    rotationalConstant: Databases/rotationalConstants.txt
    electricQuadrupoleMoment: Databases/quadrupoleMoment.txt
    OPBParameter: Databases/OPBParameter.txt
  stateProperties:              % properties of the states (S.I. Units except for the energy [eV])
    energy:
      - N2(X,v=*) = morseOscillatorEnergy
%      - N2(X,v=0,J=*) = rigidRotorEnergy
```

```
    statisticalWeight:
      - N2(X,v=*) = 1.0
%      - N2(X,v=0,J=*) = rotationalDegeneracy_N2
      - N2(A3Su+) = 3.0
      - N2(B3Pg) = 6.0
      - N2(C3Pu) = 6.0
      - N2(w1Du) = 1.0
      - N2(a1Pg) = 1.0
      - N2(a'1Su-) = 1.0
      - N2(+,X) = 2.0
      - N2(+,B2Su+) = 2.0
      - N(4S) = 4.0
      - N(2D) = 10.0
      - N(2P) = 6.0
    population:
      - N2(X) = 1.0
      - Nitrogen/N2_vibpop.txt
%      - N2(X,v=0,J=*) = boltzmannPopulation@gasTemperature
      - N(4S) = 1.0
  numerics:                    % configuration of numerical details of the simulation
    energyGrid:                % properties of the energy grid (in eV)
      maxEnergy: 100           % (use 18-20 for time-dependent simulations)
      cellNumber: 1500         % (use 1800-2000 for time-dependent simulations)
      smartGrid:             % configuration of the smart grid
        minEedfDecay: 20     % minimum number of decade-fall for the EEDF
        maxEedfDecay: 25     % maximum number of decade-fall for the EEDF
        updateFactor: 0.05   % factor used to increase or decrease the maximum value of the energy grid
    maxPowerBalanceRelError: 1e-8
    % threshold for the relative power balance warning message (use at least 100 for time-dependent simulations)
    nonLinearRoutines:
      algorithm: mixingDirectSolutions % mixingDirectSolutions or temporalIntegration
      mixingParameter: 0.7             % mixingDirectSolutions mixing parameter from 0 to 1
      maxEedfRelError: 1e-9 % maximum rel. variation for EEDF between two iterations (stop criterion)
%   odeSetParameters:       % optional parameters for ode solver of "temporalIntegration" algorithm
%       AbsTol: 1e-300
%       RelTol: 1e-6
%       MaxStep: 1e-7
```

Code 4: Configuration of the electron kinetics for a steady-state simulation

IMPORTANT NOTES:

- The `isOn` allows the user to activate (`true`) or deactivate (`false`) the simulations of LoKI-B.
- The `eedfType` allows the user to specify the type of simulation for the EEDF:
  - `boltzmann`, corresponding to the solution of the EBE under the classical two-term approximation;
  - `prescribedEedf`, corresponding to a prescribed generalised Maxwellian EEDF at given electron temperature T_e (see section 3.1 and Reference Manual [12]).
- The `shapeParameter` allows the user to specify the shape parameter s, when a `prescribedEedf` is selected (see Reference Manual [12]).
- The `ionizationOperatorType` allows the user to choose the type of ionisation operator with the EBE. The possible configurations are (see Reference Manual [12]):
  - `conservative`: the ionisation is treated as an excitation process;
  - `oneTakesAll`: the ionisation is treated as a non-conservative mechanism, with the new-born secondary electron introduced at zero kinetic-energy and the scattered electron taking the remaining energy after an ionisation event.
  - `equalSharing`: the ionisation is treated as a non-conservative mechanism, with the remaining energy after the event being equally shared between the scattered and the secondary electrons.

– `usingSDCS`: the ionisation is treated as a non-conservative mechanism, described by considering a Single Differential Cross Section (SDCS) that depends on the energies of the primary and the secondary electrons.

In general, the available experimental data for SDCS is scarce and often measured for only a few energy values of the primary electron. Here, we have adopted the comprehensive set of measurements of Opal *et al* [17], for the double differential cross section of different gases, resolved on both the energy and the angle of the secondary electron. This pioneering work proposes differential ionisation cross sections with a very reliable shape [18], which can be normalized from available experimental data for the corresponding integral cross sections (see Reference Manual [12]). Indeed, after integration over the angles, and by comparing the energy distributions of the secondary electrons for different energies of the primary electrons, these authors proposed a fitting expression for the SDCS

$$\sigma^{\text{sec}}_{i,\text{ion}}(u',u) = \frac{\sigma_{i,\text{ion}}(u')}{w\,\text{arctg}\left[\left(u'-V_{i,\text{ion}}\right)/(2w)\right]}\frac{1}{1+(u/w)^{\beta}} \quad , \tag{6}$$

where $\beta \simeq 2$ and w is a parameter close to the ionisation threshold $V_{i,\text{ion}}$, which is set for each gas according to the original estimates [17]. Note that (6) satisfies exactly the expression for the integral cross section $\sigma_{i,\text{ion}}$ (see Reference Manual [12]), and therefore it is used in LoKI-B to describe the energy distribution of the electrons involved in ionisation events, using available data and mitigating possible normalisation errors.

The value of w is introduced via the `OPBParameter`, in the `gasProperties` section (see below), and should be provided by the user if the choice of the `ionizationOperatorType` is `usingSDCS`. If no value is provided for this parameter, the code will take w for each state equal to the corresponding ionisation potential.

- The `growthModelType` allows the user to specify the configuration of the growth model, `spatial` or `temporal`, for the electron density (see Reference Manual [12]);

  The parameter `growthModelType` is **mandatory**, being used if the choice of the `ionizationOperatorType` is different than `conservative`, and/or in the case of electronegative gases (with non-conservative attachment mechanisms).
  Recall that, when choosing `prescribedEedf`, the electron power density transferred in collisional events is calculated assuming conservative mechanisms only, *i.e.* treating ionisation as an excitation mechanism and setting to zero the power-transfer due to attachment (see Reference Manual [12]). For this reason, **selecting a prescribed EEDF for studying electronegative gases is not recommended.**

  When choosing `growthModelType` equal to `spatial` the user **MUST** adopt a DC applied electric field (see Reference Manual [12]), hence `excitationFrequency` must be set equal to `0` in this case.

  Note that all non-conservative modes (`oneTakesAll`, `equalSharing`, `usingSDCS` and the presence of attachment collisions) activate non-linear operators for the creation / destruction of electrons, hence code performance is expected to decrease in these cases.

- The `includeEECollisions` allows the user to activate (`true`) or deactivate (`false`) the electron-electron collision operator in LoKI-B.

  The parameter `includeEECollisions` is **mandatory**.

  Note that the inclusion of electron-electron collisions activates a non-linear operator, hence code performance is expected to decrease in this case.

  When performing time-dependent simulations with LoKI-B as standalone tool, it is possible to handle electron-electron collisions considering a time-dependent electron-density $n_e(t)$, calculated according to the solution of $dn_e(t)/dt = \langle\nu_{\text{eff}}\rangle(t)n_e(t)$ (see Reference Manual [12]), for an initial value n_0 defined by the parameter `electronDensity` in the `workingConditions`.
  The calculation of $n_e(t)$ is activated through the expression
  `boltzmann.eDensIsTimeDependent = true`
  in function `obtainTimeDependentSolution(boltzmann)` with the `Boltzmann.m` class. To deactivate this calculation the user should select the logical parameter `false`, instead.
  Note that the previous expression only activates the calculation of $n_e(t)$. To use the time-evolution of the electron density in the electron-electron collisions operator, the user must further activate electron-electron collisions by setting `includeEECollisions: true` in the setup file.

- In `LXCatFiles` the user can specify the text files (along with the corresponding path) containing the electron-scattering cross sections for the different gases used in the simulations, usually obtained from the open-access website LXCat [14]. From these files, the code obtains also information about all the gases and all the states that are targets and/or products in the electron collisions.

  Note that not every cross section datafile obtained from LXCat is accepted, since it must comply with certain format requirements for LoKI-B to retrieve the relevant data; see section 4.1 for more information.

- When `LXCatFilesExtra` is activated, the user can specify the text files (along with the corresponding path) containing information on "extra" electron-scattering cross sections. The "extra" refers to **cross sections that are not used in the solution to the EBE**, but only in the evaluation of the corresponding rate coefficients by integration over the EEDF.

  Usually, the "extra" files are obtained from the open-access website LXCat [14], provided they comply with certain format requirements for LoKI-B to retrieve the relevant data; see section 4.1 for more information.

- The `effectiveCrossSectionPopulations` allows the user to specify some particular populations to evaluate an "elastic" momentum-transfer cross section from an "effective" one. The populations can be provided through one or more "property files", located in the input folder or in any subfolder within it (see section 4.2).

  For a gas k, the electron-neutral momentum-transfer cross section $\sigma_{k,c}(u)$ is constructed by weighting the contributions of the different types of collisional mechanisms to give

$$\sigma_{k,c}(u) = \sum_i \xi_{k_i} \sigma^{\text{el}}_{k_i,c}(u) + \sum_{i,j>i} \left[\xi_{k_i} \sigma_{k_{(i,j)},c}(u) + \xi_{k_j} \sigma_{k_{(j,i)},c}(u) \right] \quad , \tag{7}$$

  where $\sigma^{\text{el}}_{k_i,c}$, $\sigma_{k_{(i,j)},c}$ and $\sigma_{k_{(j,i)},c}$ are the electron-neutral momentum-transfer cross sections for the elastic scattering of state k_i, the inelastic excitation $k_i \rightarrow k_j$, and the superelastic de-excitation $k_j \rightarrow k_i$, respectively. The latter cross sections, whose magnitudes are usually much smaller than those of $\sigma^{\text{el}}_{k_i,c}$, are often taken assuming an isotropic scattering (also due to the lack of data), in which case one can identify the momentum-transfer cross section with the integral cross section (integrated over all scattering angles), *i.e.* $\sigma_{k_{(i,j)},c} \simeq \sigma_{k_{(i,j)}}$. Also, the elastic momentum-transfer cross section is usually identified with that of the highly-populated ground-state of the gas. Therefore

$$\sigma_{k,c}(u) \simeq \sigma^{\text{el}}_{k,c}(u) + \sum_{i,j>i} \left[\xi_{k_i} \sigma_{k_{(i,j)}}(u) + \xi_{k_j} \sigma_{k_{(j,i)}}(u) \right] \quad . \tag{8}$$

  Note that electron-neutral anisotropic scattering can indeed be handled in a two-term Boltzmann solver by using momentum-transfer cross sections in (7) and the EBE (see Reference Manual [12]), **for mechanisms with low excitation-thresholds** (such as rotational excitations / deexcitations) [19]. The IST-Lisbon database on LXCat provides momentum-transfer cross sections for these mechanisms in the case of some gases (see section 4.1).

  The solution of the EBE requires information on both the "elastic" and the "total" momentum-transfer cross sections for **all** the electronic target-states of the gas, but the way to obtain it depends on the momentum-transfer data available for each gas. If the "elastic" momentum-transfer cross section is available, equation (8) can be used directly to deduce the corresponding "total" $\sigma_{k,c}$, knowing the electron-scattering inelastic / superelastic cross sections for the transitions considered together with the relevant populations, to be self-consistently determined (if possible) within a kinetic model. Conversely, if the data available is for a "total" momentum-transfer cross section, which in databases is usually termed as "effective" $\sigma^{\text{eff}}_{k,c}$, then the elastic momentum-transfer cross section can be deduced from

$$\sigma^{\text{el}}_{k,c} = \sigma^{\text{eff}}_{k,c} - \sum_{i,j>i} \left[\xi^{\text{prescribed}}_{k_i} \sigma_{k_{(i,j)}} + \xi^{\text{prescribed}}_{k_j} \sigma_{k_{(j,i)}} \right] \quad , \tag{9}$$

  where the populations $\xi^{\text{prescribed}}_{k_i}$ and $\xi^{\text{prescribed}}_{k_j}$ correspond to prescribed distributions for the vibrational states with the electronic ground-state of the gas and for the rotational states with the vibrational ground-state of the electronic ground-state of the gas. Ideally, the distributions should be coherent with the measured / estimated data of $\sigma^{\text{eff}}_{k,c}$, allowing to obtain a *unique* result (density independent) for the elastic

cross section.

The prescription of populations $\xi_{k_i,j}^{\text{prescribed}}$ by the user is strongly discouraged and should be taken with extreme care, since the values of these populations are directly used in (9) without any checking or processing. In any case, this feature is to be used only when an "effective" momentum-transfer cross section is provided, instead of an "elastic". By default, LoKI-B adopts a Boltzmann distribution at 300 K for $\xi_{k_i,j}^{\text{prescribed}}$, but the user can modify this choice.

Note also the following:

- when the collisional data for a gas includes an electron-impact "effective" momentum-transfer cross section: : (i) it is assumed that it corresponds to an **electronic** momentum-transfer cross section; and (ii) the code evaluates an elastic momentum-transfer cross section and uses this **same** cross section for **all** the electronic target-states of the gas;
- when the collisional data for a gas includes an electron-impact "elastic" momentum-transfer cross section for a given state (electronic, vibrational or rotational), the user must provide **individual** elastic momentum-transfer cross sections for **each** target-sibling of that state.

  For example, if N_2(X,v=0-10) and N_2(A) are targets, the user can provide "elastic" momentum-transfer cross sections: (i) for the 11 vibrational states N_2(X,v=0-10) and for the electronic state N_2(A); or (ii) for the 2 electronic states N_2(X) and N_2(A). However, it cannot simply provide "elastic" momentum-transfer cross sections for N_2(X,v=0) and N_2(A).

- In `CARgases` the user can specify the gas or gases (indicated as a list of gas names) for which the continuous approximation for rotations (CAR) [20] should be activated in the simulations (see Reference Manual [12]). When the CAR is not to be used, this field must be removed or commented.

  Note that the CAR approach, to describe the effect of rotational inelastic/superelastic mechanisms, is valid only for some homonuclear diatomic molecules, such as N_2 and O_2 for which it was tested. In general, for rotations, the discrete approach of the collision operator should be adopted instead (see Reference Manual [12]).

- In `gasProperties` the user can specify the relevant properties of the different gases used in the simulations, such as

  ```
  mass
  fraction
  harmonicFrequency
  anharmonicFrequency
  rotationalConstant
  electricQuadrupoleMoment
  OPBParameter
  ```

  - It is **mandatory** to specify the values of `mass` and `fraction`. The latter should add to 1, when summed over all gases used in the simulations.
  - `harmonicFrequency` and `anharmonicFrequency` are optional, and are used to evaluate the energy of the different vibrational states by assuming an harmonic / anharmonic oscillator model.
  - `rotationalConstant` is optional, and is used to evaluate the energy of the different rotational states by assuming a rigid-rotor model.
  - `rotationalConstant` and `electricQuadrupoleMoment` are optional, and are used when adopting the CAR approach for homonuclear diatomic molecules. In this case the code returns an error message if these parameters are not specified.
  - `OPBParameter` is optional and corresponds to the value of the w parameter to be used in (6), when adopting the `usingSDCS` mode for the ionisation operator.

  The values of the different gas properties can be specified using one of the following methods:

  - "Direct input" (see example below).

    ```
    N2 = 1
    ```

```
N = 0
```

When using the "direct input" method, properties must be set only for those gases used in the simulations, *i.e.* those listed in the specified LXCat files.

– "Property file" (see example below; for more information about "property files" see section 4.2).

```
mass: Databases/masses.txt
```

When using the "property file" method, it is possible to specify the properties of gases that are not used in the simulations. LoKI-B handles input files as "databases", where it searches only for the information required.

- In `stateProperties` the user can specify the properties of the states of the different gases used in the simulations, in accordance with the information retrieved from the LXCat files. The properties available are

```
energy
statisticalWeight
population
```

– `energy` and `statisticalWeight` are optional, and can be used for the code to evaluate the `population` of some set of states, for example adopting a given distribution.

– `statisticalWeight` is **mandatory** for those states appearing in the LXCat files as involved in second-kind collisions (mechanisms tagged with a double-arrow ↔), in which case the Klein-Rosseland relation [21] is used to obtain the superelastic cross section from the corresponding inelastic.

In general, the `statisticalWeight` of a state corresponds to the product of the statistical weights g associated with the different partition functions (electronic, vibrational, rotation) of this state. For example, the statistical weight of states N_2(X,v=0,J) should be set as $g[N_2(X, v = 0, J)] = g_X \times g_{v=0} \times g_J = 1 \times 1 \times 3\left[1 + \frac{1}{2}\left(1 + (-1)^J\right)\right](2J + 1)$, where the rotational statistical weight g_J includes also the contribution of the hyperfine splitting resulting from the coupling between the electron and the nuclear spin. Note, however, that the statistical weights are used only to handle second-kind collisions (see Reference Manual [12]) and to define prescribed distributions of populations (e.g, see (18) and (22) in section (5.1)), where they always appear as ratios g_j/g_i with i, j corresponding to sibling states (i.e. both electronic, or both vibrational or both rotational). Therefore, the results are not changed if the `statisticalWeight` of a state is defined only for the partition function *intrinsic* to that state; for example, the statistical weight of *rotational* states N_2(X,v=0,J) can be defined simply as $g[N_2(X, v = 0, J)] = g_J = 3\left[1 + \frac{1}{2}\left(1 + (-1)^J\right)\right](2J + 1)$ (see also section (5.1)).

– It is **mandatory** to specify the values of `population` for all states considered as **targets of electron-scattering mechanisms**, *e.g.* the vibrational distribution of an electronic ground-state.
The user should **not** define populations for the states that are only involved in electron-scattering extra collisions (included in the `LXCatFilesExtra`), since the corresponding cross sections are not used in the calculation of the EEDF.

– The values of the different populations should be properly normalized to 1, when summed over all sibling states.

The values of the different properties of states can be specified using one of the following methods (note that the syntax supports the wildcard character `*`):

– "Direct input" (see example below)

```
N2(X,v=*) = 1.0
```

– "Property functions" (see example below; for more information about "property functions" see section 5.1).

```
N2(X,v=*) = morseOscillatorEnergy
```

When using the "direct input" method or the "property functions" method, properties

must be set only for those gases and states used in the simulations, *i.e.* **those listed in the specified LXCat files.**

– "Property file" (see example below; for more information about "property files" see section 4.2).

```
Nitrogen/N2_vibpop.txt
```

When using the "property file" method, it is possible to specify the properties of gases and states that are not used in the simulations. LoKI-B handles input files as "databases", where it searches only for the information required.

- In `numerics` the user can specify the numerical details of the LoKI-B simulations. The configuration parameters available are

```
energyGrid
maxPowerBalanceRelError
nonLinearRoutines
```

  – The `energyGrid` allows the user to specify the properties of the energy grid for simulations in LoKI-B (see Reference Manual [12]):
    * `maxEnergy`, corresponding to the value of $u_{\max}$ (in eV) (this is a **mandatory** parameter);
    * `cellNumber`, is an integer corresponding to the number of cells $\mathcal{N} > 0$ of the energy grid (this is a **mandatory** parameter);
    * `smartGrid`, activating a "smart grid" that will automatically adjust the maximum energy of the grid, as to ensure a user-prescribed number of decades in the fall of the EEDF, between f_1 and $f_{\mathcal{N}}$ (this is an optional parameter). This option is particularly relevant for simulations spanning different values of E/N or T_e. The configuration parameters are:
      · `minEedfDecay`, is an integer corresponding to the minimum number of decade-fall for the EEDF;
      · `maxEedfDecay`, is an integer corresponding to the maximum number of decade-fall for the EEDF (`maxEedfDecay` > `minEedfDecay`);
      · `updateFactor`, corresponding to the factor by which the `maxEnergy` is increased or decreased, in the automatic adjustment of the energy grid ($0 <$ `updateFactor` < 1).

    Note that the "smart grid" is as "smart" as the user that control its properties. So, if you have doubts on how this works, it is better to ignore this feature. However, if you decide to activate the "smart grid", you should consider the following:
    1. **When the smart grid is activated, the `maxEnergy` parameter is the initial value for the maximum energy of the grid. Please try to provide the best possible approximation for it, in order to avoid unnecessary iterations that will degrade the run time.**
    2. **Increasing $u_{\max}$ significantly without a corresponding increase in the number of grid points may reduce calculation accuracy. Users should verify the grid convergence of the simulation results.**
    3. **Do not specify very restrictive conditions for the EEDF, for example by setting `minEedfDecay` to 20 and `maxEedfDecay` to 21, since this condition can only be satisfied by imposing a very-small `updateFactor`, which will also degrade the run time. Typical values for the decade-fall are within 10 to 25, considering not only the use of double precision, but also the order of magnitude of the various quantities (kinetic energy, cross sections) multiplying the EEDF in the calculations.**
    4. **Use a "reasonable" value for the `updateFactor` parameter. Very-small values will decrease the performance of the code, whereas very-large values will make extremely difficult for the code to satisfy the decade-fall imposed for the EEDF. Recommended values are usually between 0.05 and 0.2.**

  – The `maxPowerBalanceRelError` allows the user to specify the threshold value for the error of the relative power balance, above which the code **returns a warning message**. The typical threshold value is $\simeq 10^{-9} - 10^{-8}$. (This is a **mandatory** parameter). Note that this parameter is not active in time-dependent simulations, in which case the "power balance" corresponds to the value of the intrinsic time-change of the electron mean energy, at unit gas density (see Reference Manual [12]).

– In `nonLinearRoutines` the user can specify several parameters for the configuration of the algorithms with the non-linear routines, dealing with the non-conservative collision operators (ionisation and attachment) and the electron-electron collisions operator (see Reference Manual [12]).

  * `algorithm`: the user can choose the nonlinear algorithm to adopt in the simulations. (This is a **mandatory** parameter).
    · `mixingDirectSolutions`, which uses two nested iterative algorithms, based on successive direct solutions.
    This algorithm is recommended in general.
    · `temporalIntegration`, which uses a relaxation method.
    This algorithm is recommended only in the case of strong nonlinear effects (*e.g.* high electronegativity and/or high ionisation degrees).
    This algorithm **MUST** be chosen for time-dependent simulations, *e.g.* when LoKI-B runs independently for electrons subjected to a pulsed electric-field.
  * `mixingParameter:` the user **MUST** define the value (between 0 and 1) of a mixing parameter, when choosing the `mixingDirectSolutions` algorithm.
    Note that the mixing parameter should not be zero, as the non-linear operators will not converge, in this case.
  * `maxEedfRelError:` the user can define the stop criterion for the `nonLinearRoutines`, by defining the maximum value for the relative variations of the EEDF, between two consecutive iterations. The typical value is 10^{-9}. (This is a **mandatory** parameter). Note that this parameter is not active in time-dependent simulations, in which case the convergence criterion is defined by the relative tolerance `RelTol`, imposed as parameter of the `temporalIntegration` solver (see Reference Manual [12]).
  * `odeSetParameters:` this is an optional field where the user can define specific options for the `temporalIntegration` solver. A detailed description of the different options can be found in the MATLAB® documentation [22]. Example of useful options are given below.
    · `AbsTol`: is the value of the absolute tolerance for the `temporalIntegration` algorithm. The typical value is 10^{-300}.
    · `RelTol`: is the value of the relative tolerance for the `temporalIntegration` algorithm. The typical value is 10^{-6}.
    · `MaxStep`: is the maximum value for the step of the advancing parameter with the `temporalIntegration` algorithm. The typical value is 10^{-7}.

3.3 Chemistry

In this field of the setup file the user can define all the details related with the kinetic scheme, *i.e.* with the configuration of the LoKI-C module. Code 5 shows the section of the `'default_lokibc_setup.in'` file that handles this configuration.

```
% --- configuration of the heavy-species for the kinetic scheme ---
chemistry:
  isOn: true
  convergenceParameter: electronDensity % electronDensity or dischargeCurrent or dischargePowerDensity
  thermalModel:
    isOn: true                % true or false (to activate or deactivate the thermal model)
    boundary: wall            % wall or external (location of the model boundary condition)
    wallFraction: 1           % fraction of energy released at the wall that returns
                              % to the gas/plasma volume (check fw parameter in the documentation)
    intConvCoeff: 120/1.6021766208e-19
                              % in eV s-1 m-2 K-1
                              % (internal convection coefficient of the cylindrical wall)
    extConvCoeff: 15/1.6021766208e-19
                              % in eV s-1 m-2 K-1
                              % (external convection coefficient of the cylindrical wall)
    includeJouleHeating: false
  chemFiles:
    - Nitrogen/nitrogen_mod.chem
```

```
checkChemFilesStoichiometry: true  % true or false
               % (to activate or deactivate stoichiometry check in chem files)
gasProperties: % inherited from the configuration of the electron kinetics
  mass: Databases/masses.txt
  heatCapacity:             % in eV K-1 mol-1 (heat capacity functions at constant pressure)
    - N2 = nitrogenHeatCapacity
    - N = nitrogenHeatCapacity
    - N3 = nitrogenHeatCapacity
    - N4 = nitrogenHeatCapacity
  thermalConductivity:     % in eV s-1 m-1 K-1 (thermal conductivity functions)
    - N2 = nitrogenThermalConductivity
    - N = nitrogenThermalConductivity
    - N3 = nitrogenThermalConductivity
    - N4 = nitrogenThermalConductivity
  inFlowFraction:
    - N2 = 1
stateProperties:
  energy:
    - N2(X,v=*) = morseOscillatorEnergy
  statisticalWeight:
    - N2(X,v=*) = 1.0
  population:
    - N2(X) = 1.0
    - N(4S) = 1.0
  inFlowPopulation:
    - N2(X) = 1
    - N2(X,v=0) = 1
  reducedDiffCoeff:
    - N2(A3Su+) = generalizedTemperatureDependentCoeff@5.0e20/sqrt(300),gasTemperature,0.5
    - N2(B3Pg) = generalizedTemperatureDependentCoeff@5.0e20/sqrt(300),gasTemperature,0.5
    - N2(C3Pu) = generalizedTemperatureDependentCoeff@5.0e20/sqrt(300),gasTemperature,0.5
    - N2(a1Pg) = generalizedTemperatureDependentCoeff@5.0e20/sqrt(300),gasTemperature,0.5
    - N2(a'1Su-) = generalizedTemperatureDependentCoeff@5.0e20/sqrt(300),gasTemperature,0.5
    - N2(w1Du) = generalizedTemperatureDependentCoeff@5.0e20/sqrt(300),gasTemperature,0.5
    - N2(+,X) = generalizedTemperatureDependentCoeff@1.7e20/sqrt(273),gasTemperature,0.5
    - N2(+,B2Su+) = generalizedTemperatureDependentCoeff@1.7e20/sqrt(273),gasTemperature,0.5
    - N3(+,X) = generalizedTemperatureDependentCoeff@1.7e20/sqrt(273),gasTemperature,0.5
    - N4(+,X) = generalizedTemperatureDependentCoeff@1.7e20/sqrt(273),gasTemperature,0.5
    - N(2D) = generalizedTemperatureDependentCoeff@6.4e20/sqrt(300),gasTemperature,0.5
    - N(2P) = generalizedTemperatureDependentCoeff@5.2e20/sqrt(300),gasTemperature,0.5
    - N(+,gnd) = generalizedTemperatureDependentCoeff@1.7e20/sqrt(273),gasTemperature,0.5
  reducedMobility:
    - N2(+,X) = 1.87e-4*101325/(273*1.38064852e-23)
    - N2(+,B2Su+) = 1.87e-4*101325/(273*1.38064852e-23)
    - N3(+,X) = 1.87e-4*101325/(273*1.38064852e-23)
    - N4(+,X) = 2.33e-4*101325/(273*1.38064852e-23)
    - N(+,gnd) = 1.87e-4*101325/(273*1.38064852e-23)
 % flowBarrier:       % flow barrier (factor between 0-1; by default is set = 1)
 %   - N2(A3Su+) = 0.05
 %   - N2(X,v=0) = 0.1
iterationSchemes:
  % pressureRelError: 1e-3
  pressureMaxIterations: 1000
  % neutralityRelError: 1e-3
  neutralityMaxIterations: 500
  % globalRelError: 1e-3
  globalMaxIterations: 100
timeIntegrationConf:
```

```
odeSolver: ode15s
dischargeTime: 10
postDischargeTime: 0
odeSetParameters:   % optional parameters that can be sent to the odeSolver
   MaxStep: 0.1
   RelTol: 5e-4
%   AbsTol: 1e-10
```

Code 5: Configuration of the chemistry for a steady-state simulation

IMPORTANT NOTES:

- The `isOn` allows the user to activate (`true`) or deactivate (`false`) the chemistry simulations of LoKI-C.
- LoKI-C supports three types of experimental input conditions for *steady-state simulations*, through `convergenceParameter`: `electronDensity`, `dischargeCurrent` and `dischargePowerDensity`.

  When selecting `electronDensity`, the quasi-neutrality cycle of the code iterates over E/N to satisfy the neutrality condition $\sum_{k=\text{ion}} n_k = n_e$, for given electron density n_e.
  When selecting `dischargeCurrent`, the quasi-neutrality cycle of the code iterates over E/N and n_e to satisfy the neutrality condition $\sum_{k=\text{ion}} n_k = n_e$, for given discharge current $I = en_e\mu_e E\pi R^2$.
  When selecting `dischargePowerDensity`, the quasi-neutrality cycle of the code iterates over E/N and n_e to satisfy the neutrality condition $\sum_{k=\text{ion}} n_k = n_e$, for given discharge power density $P/V = e\Theta_E n_e$.

  For *steady-state simulations*, the user **MUST** define in the `workingConditions` the values of one of the convergence parameters: `electronDensity`, `dischargeCurrent` or `dischargePowerDensity`.
  For *quasi-stationary simulations* and *post-discharge simulations*, the user **MUST** define in the `workingConditions` the value of the `electronDensity`, which is taken as an initial value in these time-dependent simulations. (see section 2.5)
  For more details, please check the Reference Manual [12]);
- In `thermalModel` the user can specify relevant details for the configuration of the thermal model.
  - `isOn` allows the user to activate (true) or deactivate (false) the thermal model calculations in LoKI-C. When the thermal model is activated the user **MUST** define in the `workingConditions` the `gasTemperature` and one of the boundary condition parameters: `wallTemperature` for the `wall` boundary condition or `extTemperature` for the `external` boundary condition (see section 3.1).
  - `boundary` allows the user to specify the boundary condition of the thermal model, `wall` or `external` (see Reference Manual [12]).
  - `wallFraction` corresponds to the parameter f_w (see Reference Manual [12]), representing the fraction of the enthalpy-exchange in wall-reactions that returns to the gas/plasma volume.
    Note that this parameter is automatically **SET TO ZERO** by LoKI, for the `external` boundary condition.
  - `intConvCoeff` corresponds to the convection parameter $h_{\text{int}} > 0$, in the inner region near the tube wall.
    To choose $h_{\text{int}} \gg 1$ the user should set `intConvCoeff = inf`.
  - `extConvCoeff` corresponds to the convection parameter $h_{\text{ext}} > 0$, in the region outside the cylindrical wall (used with the `external` boundary condition only).
    To choose $h_{\text{ext}} \gg 1$ the user should set `extConvCoeff = inf`.
  - The `includeJouleHeating` allows the user to activate (true) or deactivate (false) a thermal source term corresponding to the total power transferred from the applied electric field to the electrons, as calculated by LoKI-B.
    Note that when this option is enabled, all other source terms in the thermal balance equation are automatically **SET TO ZERO**.
- The `chemFiles` allows the user to specify the text files with the description of the kinetic scheme to adopt, namely the reactions, the gases and the states of those gases to be included in the simulations; see section 4.3 for more information.

- The `checkChemFilesStoichiometry` allows the user to activate (true) or deactivate (false) a stoichiometry check in the chemistry files with the description of the kinetic scheme.
- For *quasi-stationary simulations*, the `chemistry` configuration **MUST** include an extra field `lookUpTables`, with several subfields (see section 2.5)
  - `power`, `swarm`, `rateCoeff`, pointing to the corresponding lookup-table files;
  - `lookUpMethod`, which allows the user to choose the `localField` or `localEnergy` method to interpolate the electron parameters from the lookup tables (see more details in Reference Manual [12]).

  Code 6 shows an example of configuration for the parameter `lookUpTables`.

```
lookUpTables:
  lookUpMethod: localField      % localField or localEnergy
  power: CO2/pulse/lookUpTablePower.txt
  swarm: CO2/pulse/lookUpTableSwarm.txt
  rateCoeff: CO2/pulse/lookUpTableRateCoeff.txt
```

  Code 6: Configuration of the parameter `lookUpTables` for a quasi-stationary simulation

  Note the following:

  - the header names in the lookup tables **MUST** correspond exactly to
    - `RedField(Td)` in all lookup tables;
    - `EleTemp(eV)`, `RedMob((msV)^-1)`, `RedDiff((ms)^-1)` and `DriftVelocity(ms^-1)` in the `lookUpTableSwarm.txt`;
    - `RID_ine(m^3s^-1)` and `RID_sup(m^3s^-1)`, with `ID`$= 1, 2, 3, \cdots$ in the `lookUpTableRateCoeff.txt`;
    - `PowerField(eVm^3s^-1)`, `PwrElaNet(eVm^3s^-1)`, `PwrCARNet(eVm^3s^-1)`, `PwrVibNet(eVm^3s^-1)` and `PwrRotNet(eVm^3s^-1)` in the `lookUpTablePower.txt`;
  - the lookup table `lookUpTableRateCoeff.txt` begins with a header listing the `ID` and the `Description` of the reactions associated with the rate coefficients and extra rate coefficients provided in the table. The electron-impact reactions specified in this header **MUST** correspond exactly to those defined in the chemistry file.
  - the different lookup tables must be generated consistently, *i.e.*, using the same number and values of the reduced electric field.
- In `gasProperties` the user can specify the relevant properties of the different gases contributing with heavy species to the chemistry simulations, such as

```
mass
fraction
harmonicFrequency
anharmonicFrequency
...
```

  This field is equivalent, in every aspect, to the one described in section 3.2, hence all the considerations mentioned there apply here. Note, however, the existence of several specific properties of LoKI-C:

  - `heatCapacity` allows defining the molar specific heat at constant pressure (C_k; see Reference Manual [12]) of the different (atomic/molecular) gases involved in the simulations;
  - `thermalConductivity` allows defining the thermal conductivities (λ_k; see Reference Manual [12]) of the different (atomic/molecular) gases involved in the simulations;
  - `inFlowFraction` allows defining the inflow fractions of the different gases in the mixture. This parameter should add to 1, when summed over all gases used in the simulations.

  The first two properties (to be evaluated in SI units) are **mandatory** if the thermal model is activated. The last property is **mandatory** if the `totalSccmInFlow`, defined in the `workingConditions`, is different from zero.
  Please note that, when running both LoKI-B and LoKI-C, the species present in LoKI-C **may or may not** coincide with the species present in LoKI-B. Furthermore, when LoKI-B and LoKI-C share the same species, their properties **may or may not** coincide in both codes. For these reasons, the following rules apply to this field:

- When running both modules (LoKI-B and LoKI-C) the `gasProperties` subfield of the `chemistry` field is **optional**. By default, the properties of the gases present in the LoKI-C simulation are inherited from those defined in the `electronKinetics` field. However, the user **can** fill the `gasProperties` subfield of the `chemistry` field if they need to: (i) override the values provided in the `electronKinetics` configuration, for the LoKI-C calculations; and/or (ii) provide values for the properties of gases that are not present in the LoKI-B calculations (*e.g.*, `heatCapacity`, `thermalConductivity` and `inFlowFraction`).
- When running LoKI-C as standalone tool the `gasProperties` subfield of the `chemistry` field is **mandatory**. In particular, it is mandatory to provide the `fraction` property for each gas present in the simulations.

- In `stateProperties` the user can specify the properties of the states of the different gases contributing with heavy species to the chemistry simulations. The properties available are

```
energy
statisticalWeight
population
inFlowPopulation
reducedDiffCoeff
reducedMobility
flowBarrier
```

This field is equivalent, in every aspect, to the one described in section 3.2, hence all the considerations mentioned there apply here. Note, however, the existence of several new properties that are important for the implementation in LoKI-C of the transport losses. In particular

- `inFlowPopulation` allows defining the inflow populations of the different parent-states of the inflow gases in the mixture. The values of the different inflow populations should be properly normalized to 1, when summed over all sibling states.
  This property is **mandatory** if the `totalSccmInFlow`, defined in the `workingConditions`, is different from zero;
- `flowBarrier` is a parameter between 0-1 (with default value 1), which allows controlling the outflow of each state of the different gases in the mixture (see Reference Manual [12]).
  This property is used only if the `totalSccmOutFlow` is different from zero, as defined in the `workingConditions`;
- `reducedDiffCoeff` allows defining the reduced diffusion coefficients ($D_{k_i,\text{parent of k}_\text{i}}$ for neutrals and D_{i_p} for ions; see Reference Manual [12]) of the different (atomic/molecular) states involved in the simulations;
- `reducedMobility` allows defining the reduced mobilities (μ_{i_p} for ions; see Reference Manual [12]) of the different (atomic/molecular) states involved in the simulations.

The last two properties (to be evaluated in SI units) are **mandatory** for those states exhibiting transport losses, and can be specified by using the "direct input" method

```
N2(+,X) = 1.87e-4*101325/(273*1.38064852e-23)
```

or the "property function" method

```
N2(+,X) = generalizedTemperatureDependentCoeff@1.7e20/sqrt(273),gasTemperature,0.5
```

Note further that, when running both LoKI-B and LoKI-C, the species present in LoKI-C **may or may not** coincide with the species present in LoKI-B. Furthermore, when LoKI-B and LoKI-C share the same species, their properties **may or may not** coincide in both codes. For these reasons, the following rules apply to this field:

- When running both modules (LoKI-B and LoKI-C) the `stateProperties` subfield of the `chemistry` field is **optional**. By default, the properties of the gases present in the LoKI-C simulation are inherited from those defined in the `electronKinetics` field. However, the user **can** fill the `stateProperties` subfield of the `chemistry` field if they need to: (i) override the values provided in the `electronKinetics` configuration, for the LoKI-C calculations; and/or (ii) provide values for the properties of gases that are not present in the LoKI-B calculations (*e.g.*, `reducedDiffCoeff`, `reducedMobility` and `inFlowPopulation`).

– When running LoKI-C as standalone tool the `stateProperties` subfield of the `chemistry` field is **mandatory**. In particular, it is mandatory to provide the `population` property for each state present in the simulations.

Note finally that, in *quasi-stationary simulations* the user **MUST** define the `population` of the different ion states consistent with the value used for `electronDensity` in the `workingConditions`, such that $(\sum_k \chi_k \sum_{i=\text{ion}} \delta_{k_i})\, N = n_e$ (see section 2.5).

In simulations using LoKI-GM, the user can define **different initial populations** for LoKI-B (on the `electronKinetics`) and LoKI-C (on the `chemistry`, see section 3.3). In this case, the following should be noted. At the beginning of the simulation, LoKI checks the normalisation of the populations of both, the states considered as targets of electron-scattering mechanisms (termed *LoKI-B states*) and the states included in the chemistry scheme (termed *LoKI-C states*). After the evaluation of the densities by the chemistry model, and before returning to the electron kinetics calculations, LoKI renormalizes the populations of *LoKI-B states* to the total density of the states that are simultaneously *LoKI-B states* and *LoKI-C states*. Therefore:

– in the `electronKinetics`, all *LoKI-B states* with non-zero populations should be also *LoKI-C states*. The exception are *complete distributions of sibling states* (*e.g.* the rotational distribution of a vibrational ground-state), which can be targets of electron-scattering mechanisms, being absent from the chemistry scheme;

– in the `electronKinetics`, non-zero populations are allowed for *LoKI-B states* only. In response to this requirement, the distribution of LoKI includes some property functions with truncated distributions (*e.g.* `boltzmannPopulationRotationalCutoff` and `boltzmannPopulationVibrationalCutoff`, see section 5.1), which allow the user to define normalized populations for a subset of the states involved in the complete distribution.

If the previous items are not taken into account by the user, the code produces an error.

The user is advised to initialize the populations of *LoKI-B states* and *LoKI-C states* adopting distributions collapsed to the corresponding ground-states.

- In `iterationSchemes` the user can specify the numerical details for the different cycles implemented in LoKI-C (*pressure cycle*, *neutrality cycle* and *global cycle*). For each cycle, the user can specify the following parameters

  – `pressureRelError`, which defines the convergence threshold for the pressure cycle. This is an optional parameter. By default, its value is set to 2× `RelTol`, with typical value 10^{-3};

  – `pressureMaxIterations`, corresponding to the maximum number of iterations of the pressure cycle. This is a **mandatory** parameter when the pressure cycle is active, *i.e.* if the `totalSccmOutFlow` is different from the model `ensureIsobaric`, as defined in the `workingConditions` (see more details in Reference Manual [12]). The typical value is 1000;

  – `neutralityRelError`, which defines the convergence threshold for the neutrality cycle. This is an optional parameter. By default, its value is set to 2× `RelTol`, with typical value 10^{-3};

  – `neutralityMaxIterations`, corresponding to the maximum number of iterations of the neutrality cycle. This is a **mandatory** parameter for *steady-state simulations* (see section 2.5; for more details, see Reference Manual [12]). The typical value is 500;

  – `globalRelError`, which defines the convergence threshold for the global cycle. This is an optional parameter. By default, its value is set to 2× `RelTol`, with typical value 10^{-3};

  – `globalMaxIterations`, corresponding to the maximum number of iterations of the global cycle. This is a **mandatory** parameter for *steady-state simulations* (see section 2.5; for more details, see Reference Manual [12]). The typical value is 100.

  The user can deactivate any of the cycles by setting its maximum number of iterations to 1.
  Moreover, the neutrality cycle and the global cycle are deactivated when running LoKI-C as standalone tool (this behaviour might change in a future release of the code).

- In `timeIntegrationConf` the user can specify the numerical details of the time integration for the rate balance equations with LoKI-C. The available subfields for the configuration of this parameter are:

– `odeSolver`, specifying the ordinary differential equation (ODE) solver [22] to adopt in the time integration. This is a **mandatory** parameter. Presently, we strongly recommend the solver `ode15s`, because it has provided the best results in terms of accuracy and performance for the cases tested.

– `dischargeTime`, corresponding to the final integration time of the rate balance equations with LoKI-C, for active discharges. This is a **mandatory** parameter for *steady-state simulations*, and also for *post-discharge simulations* that begin by running a steady-state case (see section 2.5; for more details, see Reference Manual [12]).
Note that, in order to avoid numerical errors and preserve good performance, the `dischargeTime` should not be much longer than the time needed for the system to reach its steady-state situation. The typical value is 10 s.
The `dischargeTime` can be set to zero provided that `postDischargeTime` is non-zero (see below). In this case, the discharge phase is skipped and LoKI-GM performs only the *post-discharge simulation*.

– `postDischargeTime`, corresponding to the final integration time of the rate balance equations with LoKI-C, for a post-discharge. This is a **mandatory** parameter for *post-discharge simulations*.
This parameter should be set to zero, to deactivate the study of a post-discharge (see section 2.5).
Note that, in order to avoid numerical errors and preserve good performance, the `postDischargeTime` should not be too long. A typical value is 10^{-3} s.

– `odeSetParameters` is an optional field where the user can define specific options for the solver selected in the field `odeSolver`. A detailed description of the different options can be found in the MATLAB® documentation [22]. Some examples of useful options are given below

  * `MaxStep`: is the maximum value for the step of the advancing parameter with the `odeSolver` chosen. The typical value is 0.1.
  * `RelTol`: is the relative tolerance adopted in the `odeSolver` chosen. The typical value is 5×10^{-4} (the default MATLAB® value is 10^{-3}).
  * `AbsTol`: is the absolute tolerance adopted in the `odeSolver` chosen. The typical value is 10^{-10} .

  Inexperience users should **not change the values chosen for these parameters**, since their wrong definition will **degrade the performance of the code and the quality of the calculations**.

3.4 Graphical user interface

In this field of the setup file the user can set the configuration of the Graphical User Interface (GUI). Code 7 shows the section of the `'default_lokibc_setup.in'` file that handles this configuration.

```
% --- configuration of the grafical user interface ---
gui:
  isOn: true
  refreshFrequency: 1
```

Code 7: Configuration of the GUI for a simulation

IMPORTANT NOTES:

- The `isOn` parameter allows the user to activate (`true`) or deactivate (`false`) the GUI.
- In `refreshFrequency` the user can control the frequency with which the GUI is refreshed. The value of this parameter corresponds to the number of required simulations for the GUI to be updated, hence the larger this value the less frequently the GUI is updated.

  Note that a very frequent update of the GUI can considerably degrade the run time.
  Note further that the GUI is optimized for Full HD screen resolution. Adopting different resolutions may result in anomalous displays.

When **running LoKI-GM** with the GUI activated (`isOn: true`), the graphic interface appears as in figure 1.

- On the left-hand upper corner, for each run and depending on the selected tab, the GUI displays the **Results (graphical)** of the time evolution of the densities of species (in m^{-3} for volume densities and

m^{-2} for surface densities) and the gas-related temperatures (*Chemistry Vs Time*), or the EEDF and the corresponding first anisotropy (*EEDF*), calculated for the self-consistent E/N value obtained from the simulations.
The user can select/unselect a *X axis logscale* in the *Chemistry Vs Time* graphs, and/or a *Y axis logscale* in all the graphs (*Chemistry Vs Time* and *EEDF*).
The user can select to display the results of a specific Run (first tab), and/or the results for a specific gas (second tab in *Chemistry Vs Time*), and/or the results for a specific species (third tab in *Chemistry Vs Time*). Each of the previous selections adds one or more curves to the graphs (in addition to the gas-related temperatures in *Chemistry Vs Time*).

- On the left-hand lower corner, for each run and depending on the selected tab, the GUI displays a summary of the **Results (text)** that can be found in the output files (see section 3.5), namely
  - in *Final densities*, quantitative information about the final values of the particle balance in the *Chemistry*, including the absolute density N_{k_i}, the population ξ_{k_i} and the relative error of the rate balance equation, for each species i of "gas" k, separating the *Volume Chemistry* and the *Surface Chemistry*; quantitative information about the population ξ_{k_i} of each species i of gas k used in the calculation of the *Electron Kinetics*, by solving the EBE. The information is organized by gases and their corresponding states (see Reference Manual [12]). The final percentage $100\chi_k\%$ of each gas in the mixture is indicated in square brackets;
  - in *Final temperatures*, quantitative information about the final values of the gas-related temperatures that are present in the `workingConditions` object;
  - in *Chem Parameters*, a summary of results relevant for the chemistry calculations, *e.g.* the final values of E/N, p, T_g, N, Q_{out}, n_e, I and/or P/V, and the corresponding relative errors (when applicable);
  - in *Final balance*, details about the particle balance for each species, as calculated by the Chemistry solver. Note that the same gain/loss rate may appear in different places with different values and units, if it relates to a reaction involving both volume and surface species;
  - in *Power balance*, details about the electron power balance, as calculated by the Boltzmann solver for the self-consistent E/N value obtained from the simulations;
  - in *Swarm Parameters*, a summary of the electron swarm parameters, calculated for the self-consistent E/N value obtained from the simulations;
  - in *Rate Coefficients*, a summary of the inelastic/superelastic electron rate coefficients, calculated for the self-consistent E/N value obtained from the simulations, using both the cross sections adopted for solving the EBE and the *extra* set of cross sections defined by the user as additional input data of LoKI-B (see section 3). It also displays the description of each mechanism.
- On the right-hand upper corner, depending on the selected tab, the GUI displays a summary of the input **Setup**, namely
  - in *Setup file*, the uncommented lines of the setup file;
  - in *Rate Coefficients*, an interactive list of the reactions included in the `chemFiles`. By clicking on each reaction, the GUI shows the main properties related with the reaction;
  - in *Cross Sections*, the graphs of the cross sections uploaded for the simulations (including both those adopted for solving the EBE and the *extra* cross sections defined by the user as additional input data of LoKI-B). Here, the user can select in the appropriated tab which cross section(s) to display.
- On the right-hand lower corner, depending on the selected tab, the GUI displays a summary of the **Status** of the simulations, namely
  - in *Simulation Log*, information documenting events relevant to the code execution;
  - in *Chemistry Iterations*, graphical and quantitative information on the progress of the relative error of the simulations (identifying positive and negative error values), as a function of the iteration numbers, for the different nested calculation cycles (see Reference Manual [12]), namely (from right to left), the *Pressure cycle*, the *Quasineutrality cycle* and the *Global cycle*.

When **running LoKI-B as standalone tool for *electron kinetics stationary simulations*** (DC or HF cases) with the GUI activated (`isOn: true`), the graphic interface appears as in figure 2.

- On the left-hand upper corner, for each value of E/N (or T_e) adopted in the simulations and depending on the selected tab, the GUI displays the **Results (graphical)** that can be found in the output files (see Reference Manual [12]), namely
  - in *EEDF*, the plot of the EEDF and the corresponding first anisotropy;
  - in *Electron Reduced Diffusion*, the plot of D_eN as a function of E/N (or T_e);
  - in *Electron Reduced Mobility*, the plot of μ_eN (and also $\Re[\mu_{HF}N]$ and $-\Im[\mu_{HF}N]$ in a HF case), as a function of E/N (or T_e);
  - in *Electron Reduced Energy Diffusion*, the plot of $D_\varepsilon N$ as a function of E/N (or T_e);
  - in *Electron Reduced Energy Mobility*, the plot of $\mu_\varepsilon N$ as a function of E/N (or T_e);
  - in *Electron Energies*, the plot of $T_e \equiv (2/3)\varepsilon$ and u_k, as a function of E/N (or T_e);
  - in *Townsend Coefficient*, the plot of α/N as a function of E/N (or T_e);
  - in *Attachment Coefficient*, the plot of η/N as a function of E/N (or T_e);
  - in *Power Channels*, the plot of the electron energy transferred in different channels, relative to the reference power, $\Theta_x/\Theta_{\text{ref}}$, as a function of E/N (or T_e) (see Reference Manual [12]).

  In all the graphs (except for *EEDF* and *Power Channels*), the user can select/unselect a *X axis logscale* and/or a *Y axis logscale*.
  The GUI supports graphs for multiple values of E/N (or T_e) and T_g. In this case: (i) each curve in the graph *EEDF* corresponds to a pair of values (E/N, T_g) or (T_e, T_g); (ii) all other graphs plot points/curves for the different T_g values considered.
  GUI support to multiple *electron kinetics stationary simulations* (with LoKI-B) is currently limited to simultaneous variations of these parameters.

- On the left-hand lower corner, for each value of E/N (or T_e) adopted in the simulations and depending on the selected tab, the GUI displays a summary of the **Results (text)** that can be found in the output files (see section 3.5), namely
  - in *Power balance*, details about the electron power balance;
  - in *Swarm Parameters*, a summary of the electron swarm parameters (the reduced-ionisation Townsend coefficient, the reduced-attachment Townsend coefficient, and the drift velocity are calculated and displayed only in a DC case);
  - in *Rate Coefficients*, a summary of the inelastic/superelastic electron rate coefficients, calculated using both the cross sections adopted for solving the EBE and the *extra* set of cross sections defined by the user as additional input data of LoKI-B (see section 3). It also displays the description of each mechanism.

- On the right-hand upper corner, depending on the selected tab, the GUI displays a summary of the input **Setup**, namely
  - in *Setup file*, the uncommented lines of the setup file;
  - in *Cross Sections*, the graphs of the cross sections uploaded for the simulations (including both those adopted for solving the EBE and the *extra* cross sections defined by the user as additional input data of LoKI-B). Here, the user can select in the appropriated tab which cross section(s) to display.

- On the right-hand lower corner, the GUI will display a summary of the **Status** of the *Simulation Log*, with information documenting events relevant to the code execution.

When **running LoKI-B as standalone tool for *electron kinetics time-dependent simulations*** (e.g. considering a time-dependent, non-oscillatory ($\omega = 0$) electric field) with the GUI activated (`isOn: true`), the graphic interface also looks similar to that of figure 2. In this case

- the **Results (graphical)** and the **Results (text)** are displayed as a function of time t (instead of E/N or T_e);
- the GUI displays also a *Pulse temporal integration progress bar* that depicts the progress of the simulation as a function of the *Integration time*, indicating also the current *Computational time*.

In this case, the GUI also supports graphs for multiple values of t and T_g, depicting: (i) curves for pairs of values (t, T_g), in the graph *EEDF*; (ii) points/curves for the different T_g values considered, in all other graphs. GUI support to multiple *electron kinetics time-dependent simulations* (with LoKI-B) is currently limited to simultaneous variations of $E/N(t)$ and T_g.

When **running LoKI-C as standalone tool for *quasi-stationary simulations*** (e.g. considering a time-dependent electric field pulse) with the GUI activated (`isOn: true`), the graphic interface looks similar to that of figure 1. In this case

- **Results (text)** are displayed for the different simulation times (`samplingPoints`) defined in the `working Conditions` (see section 3.1).
- The graphs in *Chemistry Iterations* under **Status** are empty, since no calculation cycles are executed.

3.5 Output files

In this field of the setup file the user can define the details of the simulation output. Code 8 shows the section of the `'default_lokibc_setup.in'` file that handles this configuration.

```
% ---  configuration of the output files ---
output:
  isOn: false
  dataFormat: hdf5+txt           % txt or hdf5 or txt+hdf5
  folder: simulation_chem_1
  dataSets:
%    - inputs
    - log
    - eedf
    - swarmParameters
    - rateCoefficients
    - powerBalance
%    - lookUpTables
    - finalDensities
    - finalTemperatures
    - finalParticleBalance
    - finalThermalBalance
    - chemSolutionTime
    - chemParameters
```

Code 8: Configuration of the output for a simulation

IMPORTANT NOTES:

- LoKI-GM provides `log` information documenting events relevant to the code execution, that are displayed in the command window. This information can also be displayed in the GUI (if activated) and/or saved as text file (see below).
- The `isOn` parameter allows the user to activate (`true`) or deactivate (`false`) the output.
- The `dataFormat` parameter allows the user to select the output format: text files (`txt`), a single HDF5 file gathering all the information from the text files (`hdf5`), or both text and HDF5 files (`hdf5+txt`).

  The log and setup information are always written as text files.

  MATLAB® has used HDF5 C library version 1.14.4.3 since release R2024b. Running older MATLAB® versions may therefore prevent the use of the HDF5 format.

- In `folder` the user can specify the folder where the datasets of the simulation output will be saved. This is an optional parameter; if `folder` is removed, the code will generate a generic name for the output folder, with a time stamp.
  The output folder(s) will be located inside `[repository folder]/LoKI-GM_26.07/Code/Output/`.

- When `isOn` is `true`, `dataSets` is a **mandatory** parameter, where the user can specify the information to be saved after each simulation.
  Note that in previous versions of LoKI this parameter was called `dataFiles`.

3.5.1 Datasets in text format

When **running LoKI-B as standalone tool**, the text-format output (`txt`) is characterized by

- different E/N values, for `eedfType = boltzmann` and *electron kinetics stationary simulations* (see section 2.5). In this case the code generates as many output folders as the number of E/N values defined as `reducedField` in the `workingConditions`. In each folder, depending on the selected `dataSets` (see Code 8 and the description below), the user will find files containing the simulation results for each E/N value;

- different t values, for `eedfType = boltzmann` and *electron kinetics time-dependent simulations* (see section 2.5). In this case the code generates as many output folders as the number of "sampling points" defined within `reducedField` in the `workingConditions`. In each folder, depending on the selected `dataSets` (see Code 8 and the description below), the user will find files containing the simulation results at each sampled output time;

- different T_e values, for `eedfType = prescribedEedf`. In this case the code generates as many output folders as the number of T_e values defined as `electronTemperature` in the `workingConditions`. In each folder, depending on the selected `dataSets` (see Code 8 and the description below), the user will find files containing the simulation results for each T_e value;

- all combinations of the different parameters: E/N or T_e (for *electron kinetics stationary simulations*, when `eedfType = boltzmann` or `eedfType = prescribedEedf`, respectively), or t (for *electron kinetics time-dependent simulations* with `eedfType = boltzmann`), along with p, T_g, n_e and ω/N values. In this case, the code generates an output folder for each unique combination of parameters. The resulting directory structure is organized hierarchically, beginning with the different ω/N values and ending with the different E/N values.

The available options for `dataSets` are:

- `inputs`
  It copies all the input datafiles (*e.g* LXCat files; not functions) called from the setup file (and also an exact copy of the original setup file itself, including all commented / uncommented lines), using the corresponding structure of folders.

- `log`
  It writes the datafile `log.txt`, displaying the log information of the code execution.

- `eedf`
  It writes, for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation, the datafile `eedf.txt`, with three columns displaying the kinetic energy u, the EEDF f and the first anisotropy f^1 (only in the case of `boltzmann` simulations), respectively.

- `swarmParameters`
  It writes, for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation, the datafile `swarmParameters.txt`, displaying the values of the following quantities (see Reference Manual [12]):

  t (only for time-dependent simulations)

  E/N

  D_eN

$\mu_e N$

v_d (only in DC case)

$\Re[\mu_{\rm HF} N] + \Im[\mu_{\rm HF} N]\, i$ (only in HF case)

α/N (only in DC case)

η/N (only in DC case)

$D_\varepsilon N$

$\mu_\varepsilon N$

ε

u_k

$T_e \equiv (2/3)\varepsilon$.

- `rateCoefficients`
  It writes, for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation, the datafile `rateCoefficients.txt` displaying information on rate coefficients (see Reference Manual [12]), organized into the following groups:
  - e-Kinetics Rate Coefficients, including the inelastic / superelastic rate coefficients $C_{i,j}$ and $C_{j,i}$ calculated from the electron-impact cross sections between states i and $j > i$ provided as input data, as well as the threshold of the cross sections and the description of the reactions;
  - e-Kinetics Extra Rate Coefficients, including the inelastic / superelastic rate coefficients $C_{i,j}$ and $C_{j,i}$ calculated from the electron-impact extra cross sections between states i and $j > i$ provided as input data, as well as the threshold of the cross sections and the description of the reactions.

- `powerBalance`
  It writes, for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation, the datafile `powerBalance.txt`, displaying the values of the following electron power-density gains/losses, per electron at unit gas density (see Reference Manual [12]):

  Θ_E/N (for Joule power)

  $\Theta_{\rm el}^{\rm gain}/N$, $\Theta_{\rm el}^{\rm loss}/N$, $\Theta_{\rm el}/N \equiv \Theta_{\rm el}^{\rm gain}/N + \Theta_{\rm el}^{\rm loss}/N$ (for elastic mechanisms)

  $\Theta_{\rm CAR}^{\rm gain}/N$, $\Theta_{\rm CAR}^{\rm loss}/N$, $\Theta_{\rm CAR}/N \equiv \Theta_{\rm CAR}^{\rm gain}/N + \Theta_{\rm CAR}^{\rm loss}/N$ (for rotational mechanisms)

  $\Theta_{\rm sup}^{\rm gain}/N$, $\Theta_{\rm inel}^{\rm loss}/N$, $\Theta_{\rm ele,vib,rot}/N \equiv \Theta_{\rm sup}^{\rm gain}/N + \Theta_{\rm inel}^{\rm loss}/N$ (for electronic, vibrational, rotational mechanisms)

  $\Theta_{\rm ion}/N$ (for ionization mechanisms)

  $\Theta_{\rm att}/N$ (for attachment mechanisms)

  $\Theta_{\rm growth}/N$ (for growth mechanisms).

  It also writes

  the power balance $\Theta_{\rm growth}/N + \Theta_E/N + \Theta_{\rm coll}^{\rm gain}/N + \Theta_{\rm coll}^{\rm loss}/N$ (with "coll" referring to all collision types). Recall that, in time-dependent simulations, the "power balance" corresponds to the value of the intrinsic time-change of the electron mean energy, at unit gas density (see Reference Manual [12]);

  the relative power balance (in %) $\left[\Theta_{\rm growth}/N + \Theta_E/N + \Theta_{\rm coll}^{\rm gain}/N + \Theta_{\rm coll}^{\rm loss}/N\right] / [\Theta_{\rm ref}/N] \times 100$. Recall that, in time-dependent simulations, the "relative power balance" corresponds to the value of the intrinsic time-change of the electron mean energy divided by the reference power-density per electron (see Reference Manual [12]);

  It also writes, for each gas in the mixture, the gain/loss/net results for the electron power-density associated with the different types of collisions.

- `lookUpTables`
  It writes the following dataSets
  - `lookUpTableSwarm.txt`, containing a table with all the data in the file(s) `swarmParameters.txt` presented in columns, as a function of E/N, or t and E/N, or T_e (according to the type of simulations);

– `lookUpTableRateCoeff.txt`, containing a table with all the data in the file(s) `rateCoefficients.txt` presented in columns, as a function of E/N, or t and E/N, or T_e (according to the type of simulations);

– `lookUpTablePower.txt`, containing a table with all the data in the file(s) `powerBalance.txt` presented in columns, as a function of E/N, or t and E/N, or T_e (according to the type of simulations);

– `lookUpTableEedf.txt` **(only for *electron kinetics time-dependent simulations*)** containing a table with all the data in the file(s) `eedf.txt`, where the first column displays the values of t, the first row displays the values of u, and the remainder rows display the values of f for the different simulation times. Table 2 presents an extract of a typical `lookUpTableEedf.txt` file.

Table 2: Extract of a typical `lookUpTableEedf.txt` file for a time-dependent simulation

	0.005	0.015	0.025	0.035	...
0.0000E+00	220.49	149.76	101.72	69.09	...
1.0000E-18	220.49	149.76	101.72	69.09	...
1.0233E-18	220.49	149.76	101.72	69.09	...
1.0471E-18	220.49	149.76	101.72	69.09	...
...	...	...	...	...	...

– `electronDensityEvolution.txt`, containing a table with the values of the electron density n_e and the electron density variation dn_e/dt presented in columns, as a function of t. This file is written **only in the case of *electron kinetics time-dependent simulations*** in the presence of electron-electron collisions, considering a time-dependent electron density $n_e(t)$ (see section 3.2).

The `lookUpTables` is only written with `txt` output.
Note that in previous versions of LoKI this parameter was called `lookUpTable` (with no "s").

When **running LoKI-GM for a single simulation**, the text-format output (txt) is characterized by

- a self-consistent E/N value, for `eedfType = boltzmann` and *steady-state simulations* (see section 2.5). In this case LoKI-C and LoKI-B are run in a coupled manner, as described in the Reference Manual [12]. The code generates one output folder with text files corresponding to the chosen `dataSets` (see Code 8 and description below);
- a self-consistent T_e value, for `eedfType = prescribedEedf` and *steady-state simulations* (see section 2.5). In this case LoKI-C and LoKI-B are run in a coupled manner, as described in the Reference Manual [12]. The code generates one output folder with text files corresponding to the chosen `dataSets` (see Code 8 and description below);
- different t values, for *quasi-stationary simulations* (see section 2.5). In this case LoKI-C runs as standalone tool, and therefore the `dataSets` regarding the specific output of LoKI-B (`eedf.txt`, `swarmParameters.txt`, `powerBalance.txt` and `lookUpTables.txt`) are not written. The code generates as many output folders as the number of sampling points defined within `reducedField` in the `workingConditions`. In each folder, depending on the selected `dataSets` (see Code 8 and the description below), the user will find files containing the simulation results at each sampled output time;
- a self-consistent E/N value, for `eedfType = boltzmann` and *steady-state simulations*, in which case LoKI-C and LoKI-B are run in a coupled manner, followed by time-dependent *post-discharge simulations* at $E/N = 0$, in which case LoKI-C runs as standalone tool (see section 2.5); see also Reference Manual [12]). In this case, the code generates one output folder with text files corresponding to the chosen `dataSets` (see Code 8 and description below). The information in these files regards the *steady-state simulations* for the self-consistent E/N, with the following exceptions: `chemSolutionTime` presents results for calculations times until `dischargeTime` + `postDischargeTime`; `chemParameters` and `finalTemperatures` and present results for the final calculation time `dischargeTime` + `postDischargeTime`.

The available options in `dataSets` are (see Reference Manual [12]):

- `inputs`
  It copies all the input datafiles (*e.g* LXCat files, .chem files; not functions) called from the setup file (and also an exact copy of the original setup file itself, including all commented / uncommented lines), using the corresponding structure of folders.

- `log`
  It writes the datafile `log.txt`, displaying the log information of the code execution.

- `eedf`
  It writes the datafile `eedf.txt`, with three columns displaying the kinetic energy u, the EEDF f and the first anisotropy f^1 (only in the case of `boltzmann` simulations), respectively.

- `swarmParameters`
  It writes the datafile `swarmParameters.txt`, displaying the values of the following quantities:

  E/N

  $D_e N$

  $\mu_e N$

  v_d (only in DC case)

  $\Re[\mu_{\rm HF} N] + \Im[\mu_{\rm HF} N]\, i$ (only in HF case)

  α/N (only in DC case)

  η/N (only in DC case)

  $D_\varepsilon N$

  $\mu_\varepsilon N$

  ε

  u_k

  $T_e \equiv (2/3)\varepsilon$.

- `rateCoefficients`
  It writes, for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation, the datafile `rateCoefficients.txt` displaying information on rate coefficients, organized into the following groups:

  - e-Kinetics Rate Coefficients, including the inelastic / superelastic rate coefficients $C_{i,j}$ and $C_{j,i}$ calculated from the electron-impact cross sections between states i and $j > i$ provided as input data, as well as the threshold of the cross sections and the description of the reactions;
  - e-Kinetics Extra Rate Coefficients, including the inelastic / superelastic rate coefficients $C_{i,j}$ and $C_{j,i}$ calculated from the electron-impact extra cross sections between states i and $j > i$ provided as input data, as well as the threshold of the cross sections and the description of the reactions;
  - Chemistry Rate Coefficients, including the direct / inverse rates R_r and R_r^{-1}, the enthalpies ΔH_r, the net rates $R_r - R_r^{-1}$, and the description of the reactions r.

- `powerBalance`
  It writes the datafile `powerBalance.txt`, displaying the values of the following electron power-density gains/losses (per electron at unit gas density):

  Θ_E/N (for Joule power)

  $\Theta_{\rm el}^{\rm gain}/N$, $\Theta_{\rm el}^{\rm loss}/N$, $\Theta_{\rm el}/N \equiv \Theta_{\rm el}^{\rm gain}/N + \Theta_{\rm el}^{\rm loss}/N$ (for elastic mechanisms)

  $\Theta_{\rm CAR}^{\rm gain}/N$, $\Theta_{\rm CAR}^{\rm loss}/N$, $\Theta_{\rm CAR}/N \equiv \Theta_{\rm CAR}^{\rm gain}/N + \Theta_{\rm CAR}^{\rm loss}/N$ (for rotational mechanisms)

  $\Theta_{\rm sup}^{\rm gain}/N$, $\Theta_{\rm inel}^{\rm loss}/N$, $\Theta_{\rm ele,vib,rot}/N \equiv \Theta_{\rm sup}^{\rm gain}/N + \Theta_{\rm inel}^{\rm loss}/N$ (for electronic, vibrational, rotational mechanisms)

  $\Theta_{\rm ion}/N$ (for ionization mechanisms)

  $\Theta_{\rm att}/N$ (for attachment mechanisms)

  $\Theta_{\rm growth}/N$ (for growth mechanisms).

It also writes (see Reference Manual [12])

the power balance $\Theta_{\text{growth}}/N + \Theta_E/N + \Theta_{\text{coll}}^{\text{gain}}/N + \Theta_{\text{coll}}^{\text{loss}}/N$ (with "coll" referring to all collision types);

the relative power balance (in %) $\left[\Theta_{\text{growth}}/N + \Theta_E/N + \Theta_{\text{coll}}^{\text{gain}}/N + \Theta_{\text{coll}}^{\text{loss}}/N\right] / \left[\Theta_{\text{ref}}/N\right] \times 100$;

It also writes, for each gas in the mixture, the gain/loss/net results for the electron power-density associated with the different types of collisions.

- `finalDensities`
  It writes the datafile `chemFinalDensities.txt` displaying, for each state intervening in the chemistry calculations, the steady-state values of its absolute density, population, and relative creation/destruction rate-balance (see Reference Manual [12]). In this file, it also writes the relative percentage of the different parent states in the final mixture.
  It also writes the datafile `electronKineticsFinalPopulations.txt`, displaying the steady-state populations of each state intervening in the electron kinetics calculations. In this file, it also writes the final relative percentages of the different parent states used in the electron kinetics calculations.

- `finalTemperatures`
  It writes the datafile `finalTemperatures.txt` displaying the final values of the gas-related temperatures (e.g. T_g, T_w, T_{ext}, ...) that can be present in the `workingConditions` object.
  Note that, depending on the configuration of the thermal model, the values of the temperatures displayed in `finalTemperatures.txt` may differ from those defined by the user as working conditions in the `setup.in` file (in general, used as initial conditions).

- `finalParticleBalance`
  It writes the datafile `finalParticleBalance.txt` displaying, for each state intervening in the chemistry calculations (separated in groups according to the parent states), the steady-state values of the rates, fractional contribution (in percentage), and relative rate-balance (in percentage) for the different creation/destruction reactions affecting this state (see Reference Manual [12]).

- `finalThermalBalance`
  It writes the datafile `finalThermalBalance.txt` displaying the values of the following terms of the thermal balance equation (see Reference Manual [12]):

  conduction

  electron elastic collisions

  volume source

  wall source

  The option `finalThermalBalance` is supported only if the thermal model is activated.

- `chemSolutionTime`
  It writes the datafile `chemSolutionTime.txt`, with several columns displaying the calculation time t, the reduced electric field and the electron temperature (at time t), the relevant gas-related temperatures (at time t) if the thermal model is activated (or the gas temperature if the thermal model is not activated), and the absolute densities (at time t) of the different states intervening in the chemistry calculations, respectively.

- `chemParameters`
  It writes the datafile `chemParameters.txt`, displaying the values of the following quantities:

  Reduced electric field (last value used in the simulations)

  Target gas pressure (as defined in the `workingConditions`)

  Final gas pressure (last value obtained in the simulations)

  Final gas temperature (last value obtained in the simulations)

  Final gas density (as obtained using the final values of the gas pressure and gas temperature)

  Total input flow (as defined in the `workingConditions`)

  Total output flow (as defined in the `workingConditions` or obtained in the simulations)

Target electron density

(for steady-state simulations, the value defined in the `workingConditions`, with the corresponding relative error, if `convergenceParameter` is `electronDensity`)

(for time-dependent simulations, the last value obtained in the simulations; in this case the relative error is zero, as outcome of imposing charge neutrality)

Final electron density (for steady-state simulations, the last value obtained in the simulations, with the corresponding relative error, if `convergenceParameter` is `dischargeCurrent` or `dischargePowerDensity`)

Target discharge current (for steady-state simulations, as defined in the `workingConditions`, if `convergenceParameter` is `dischargeCurrent`)

Final discharge current (for steady-state simulations, the last value obtained in the simulations, with the corresponding relative error, if `convergenceParameter` is `dischargeCurrent`)

Target discharge power density (for steady-state simulations, as defined in the `workingConditions`, if `convergenceParameter` is `dischargePowerDensity`)

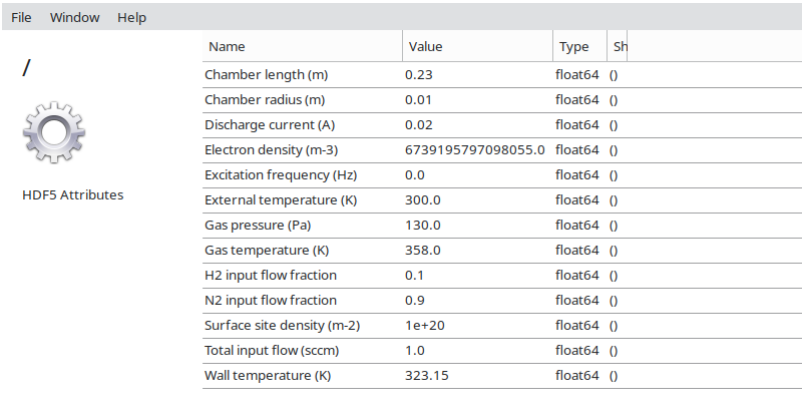
Final discharge power density (for steady-state simulations, the last value obtained in the simulations, with the corresponding relative error, if `convergenceParameter` is `dischargePowerDensity`)

3.5.2 Datasets in HDF5 format

In HDF5 format (`h5`) [23], the output results are organized in a single file. The access to the data can be done through Python scripts, using one of the several interfaces available [24], or adopting filters on tools like MATLAB® or Origin®.
Currently, HDF5 output is supported only for *electron kinetics simulations* (using LoKI-B as standalone tool) and *steady-state simulations* (in which LoKI-C and LoKI-B are run in a coupled manner). Support for the remaining simulation types available in LoKI-GM is under development.

In the following, the description of an HDF5 file organization is based on its visualization using HDFCompass [25].

The electron kinetics results (from LoKI-B) and the chemical results (from LoKI-C) are organized in the main groups `electronKinetics` and `chemistry`, respectively, under a root group ('/'). A *group* is conceptually equivalent to a folder with certain *attributes*, within which the *datasets* are organized. The attributes of the root group correspond to the relevant parameters of the working conditions (see figure 3.5.2).



Name	Value	Type	Sh
Chamber length (m)	0.23	float64	()
Chamber radius (m)	0.01	float64	()
Discharge current (A)	0.02	float64	()
Electron density (m-3)	6739195797098055.0	float64	()
Excitation frequency (Hz)	0.0	float64	()
External temperature (K)	300.0	float64	()
Gas pressure (Pa)	130.0	float64	()
Gas temperature (K)	358.0	float64	()
H2 input flow fraction	0.1	float64	()
N2 input flow fraction	0.9	float64	()
Surface site density (m-2)	1e+20	float64	()
Total input flow (sccm)	1.0	float64	()
Wall temperature (K)	323.15	float64	()

Figure 4: Attributes for a LoKI-C simulation on a N_2/H_2 mixture.

The `electronKinetics` group contains six or seven datasets:

- `reducedField`
  A table with the value(s) of the reduced electric field (or the corresponding electron temperature, see below). This table is linked to other datasets, corresponding to one of their axes. The shape and dimensions of the table depends on the problem under study.

When **running LoKI-B as standalone tool**, for `eedfType = boltzmann` and *electron kinetics stationary simulations*, the table is composed by a single row with the different E/N values used in the simulations. In this case the row contains as many cells (columns) as the number of E/N values defined as `reducedField` in the `workingConditions`.
When **running LoKI-B as standalone tool**, for `eedfType = boltzmann` and *electron kinetics time-dependent simulations*, the table has two rows with the t values and the $E/N(t)$ values. In this case the rows contain as many columns as the number of "sampling points" defined within `reducedField` in the `workingConditions`.
When **running LoKI-B as standalone tool**, for `eedfType = prescribedEedf`, the table is composed by a single row with the different T_e values used in the simulations. In this case the row contains as many cells (columns) as the number of T_e values defined as `electronTemperature` in the `workingConditions`.
When **running LoKI-GM**, for `eedfType = boltzmann` or `eedfType = prescribedEedf` and a single *steady-state simulation*, the table has a single cell with the final self-consistent value of E/N or T_e, respectively.
When **running LoKI-GM**, for a single *post-discharge simulation*, the table has a single cell with the final value $E/N = 0$ considered.
The units of each column are indicated in the dataset attributes.

IMPORTANT NOTE REGARDING SIMULATIONS FOR MULTIPLE PARAMETERS
As mentioned, LoKI-B can be **run as a standalone tool** for multiple simulations by specifying arrays of values for parameters such as `reducedField` or `electronTemperature`, as well as `excitationFrequency`, and/or `electronDensity`, and/or `gasPressure`, and/or `gasTemperature`.
In such cases, the `electronKinetics` group may additionally contain datasets describing these parameters, alphabetically ordered:

- `electronDensity`
- `excitationFrequency`
- `gasPressure`
- `gasTemperature`

Each dataset consists of a single-row table whose cells (columns) contain the values of the corresponding parameter for each simulation, as specified in `workingConditions`.
When LoKI-B is run for multiple simulations involving N_ω values of `excitationFrequency`, N_{n_e} values of `electronDensity`, N_p values of `gasPressure`, and N_{T_g} values of `gasTemperature`, the size of each output dataset is increased by a factor of $N_\omega \times N_{n_e} \times N_p \times N_{T_g}$. Note that the additional tables (pages) containing the results follow the order indicated here: `excitationFrequency`, `electronDensity`, `gasPressure`, `gasTemperature`. This information allows the user to identify the simulation conditions associated with each page of a given dataset. Note further that this order is not alphabetic, so it differs from that used for the corresponding parameter values as described above.
In the remainder of this section, we assume $N_\omega = N_{n_e} = N_p = N_{T_g} = 1$ when describing the structure and contents of the output datasets.

- `eedf`
  A 3d dataset, gathering the $\mathcal{N}$ values of the the kinetic energy u, the EEDF f and the first anisotropy f^1 (only in the case of `boltzmann` simulations), respectively, written in a three-column table (page) for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation.
  The dataset's size is $N_{E/N} \times \mathcal{N} \times 3$, with $N_{E/N}$ the number of E/N or T_e values considered in the output.
  The units of each column are indicated in the dataset attributes.

- `swarmParameters`
  A 2d table with the values of the swarm parameters, written for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation, in the following order:
  ε, u_k, T_e or E/N, v_d, $\mu_e N$, $D_e N$, $\mu_\varepsilon N$, $D_\varepsilon N$, α/N, η/N.
  T_e or E/N are written for `boltzmann` or `prescribedEedf` simulations, respectively.
  The dataset's size is $N_{E/N} \times 10$ for a DC case or $N_{E/N} \times 8$ for an HF case, where v_d, α/N and η/N are not written.
  The units of each column are indicated in the dataset attributes.

- `rateCoefficients`
  A 3d dataset with information on the rate coefficients of the collisional processes considered in the electron

kinetics, written in a table (page) for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation. The table has five columns displaying the process id, the inelastic rate coefficient, the corresponding superelastic rate coefficient, the cross section threshold, and the description of the collisional process.
The dataset's size is $N_{\text{proc}} \times N_{E/N} \times 5$, with N_{proc} the number of processes in the electron kinetics.
The units of each column are indicated in the dataset attributes.

- `extraRateCoefficients`
  A 3d dataset with information on the rate coefficients of the extra collisional processes considered (although not used in the electron kinetics), written in a table (page) for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation. The table has five columns displaying the process id, the inelastic rate coefficient, the corresponding superelastic rate coefficient, the cross section threshold, and the description of the collisional process.
  The dataset's size is $N_{\text{proc}} \times N_{E/N} \times 5$, with N_{proc} the number of extra processes considered.
  The units of each column are indicated in the dataset attributes.

- `powerBalanceSummary`
  A 3d dataset with the electron power-density gains/losses, per electron at unit gas density (see Reference Manual [12]), written for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation, in the following order (ten-column table):
  `Field`, `Elastic`, `CAR`, `Rotational`, `Vibrational`, `Electronic`, `Ionisation`, `Attachment`, `eDensGrowth`, and `Balance`.
  The dataset has three tables (pages) displaying, for each process (if applicable): the net power-density (gain+loss), the power-density gain, and the power-density loss, respectively. The power balance is displayed in only two pages, showing absolute and relative values, respectively.
  The dataset's size is $3 \times N_{E/N} \times 10$.
  The units of each column are indicated in the dataset attributes.

- `powerBalanceGases`
  A multidimensional dataset with the electron power-density gains/losses, per electron at unit gas density transferred to each gas in the mixture, written for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation, for the following collisional processes (five columns):
  `rotColl`, `vibColl`, `eleColl`, `ionColl`, and `attColl`.
  The dataset has $N_{\text{gas}} \times 3$ tables (pages) displaying, for each gas (in a total of N_{gas}) and for each process (if applicable): the net power-density (gain+loss), the power-density gain, and the power-density loss, respectively.
  The dataset's total size is $N_{\text{gas}} \times 3 \times N_{E/N} \times 5$.
  The units of each column are indicated in the dataset attributes.

The `chemistry` group is created exclusively for LoKI-GM simulations. It contains two datasets and three groups.

The datasets are the following:

- `chemSolutionTime`
  A 3d dataset with information on the solution of the chemistry calculations, written in a table (page) for each `boltzmann` (E/N) or `prescribedEedf` (T_e) simulation. The table has a variable number of columns displaying the simulation `Time`, `GasTemp`, `NearWallTemp`, and the absolute densities of the $N_{\text{vol}} + N_{\text{wall}}$ volume and surface states intervening in the chemistry calculations.
  The dataset's size is $N_{E/N} \times N_{\text{time}} \times (3 + N_{\text{vol}} + N_{\text{wall}})$, where N_{time} is the number of time-steps considered in the output.
  The units of each column are indicated in the dataset attributes.

- `rateCoefficients`
  A 3d dataset with information on the rate coefficients of the kinetic reactions considered in the chemical model, written in a table (page) for each `boltzmann` (E/N) or `prescribedEedf` (T_e) simulation. The table has six columns displaying, for each reaction, the id (`rate_id`), the direct rate coefficient (`dir_coeff`), the reverse rate coefficient (`inv_coeff`), the enthalpy (`enthalpy`), the net rate (`net_rate`) and the description (`description`).
  The dataset's size is $N_{E/N} \times N_{\text{reac}} \times 6$, with N_{reac} the number of kinetic reactions in the chemical model.
  The units of each column are indicated in the dataset attributes.

The groups are the following:

- `electronKineticsPopulations`
  This group includes, for each gas intervening in the electron kinetics calculations, a dataset with the steady-state populations of the excited neutral and ion states with the gas (parent state). Every 3d dataset is written in a table (page) for each `boltzmann` (E/N) or `prescribedEedf` (T_e) simulation. The table has two columns displaying the name of the `Species` and the values of the `Population`. The attribute of each dataset is the final relative percentage of the parent state used in the electron kinetics calculations.
  The size of each dataset is $N_{E/N} \times N_{\text{states}} \times 2$, with N_{states} the corresponding number of states.

- `finalDensities`
  This group includes, for each gas intervening in the electron kinetics calculations, a dataset with the densities and populations of the excited neutral and ion states with the gas (parent state). Every 3d dataset is written in a table (page) for each `boltzmann` (E/N or t) or `prescribedEedf` (T_e) simulation. The table has four columns displaying the name of the state (`Species`) and the values of the absolute density (`Abs.Density`), population (`Population`) and relative creation/destruction rate-balance (`Balance`).
  The size of each dataset is $N_{E/N} \times N_{\text{states}} \times 4$, with N_{states} the corresponding number of states.
  The units of each column are indicated in the dataset attributes.

- `particleBalance`
  This group includes, for each gas intervening in the chemistry calculations and for each `boltzmann` (E/N) or `prescribedEedf` (T_e) simulation, a dataset with the steady-state values of the rates and the fractional contribution (in percentage) for the different creation/destruction reactions affecting the excited neutral and ion states of the gas (parent state). Every multidimensional dataset has $N_{\text{states}} \times 2 \times N_{\text{reac}} \times 3$ tables (pages) displaying, for each state of the gas (in a total of N_{states}) and for each creation/destruction reaction (in a total of $2 \times N_{\text{reac}}$): the description of the reaction responsible for the creation/destruction of the state (`Reaction`), the creation/destruction rate (`Rate`) and the fractional contribution of the reaction (in percentage) for the creation/destruction of the state (`Contribution`).
  The dataset's total size is $N_{E/N} \times N_{\text{states}} \times 2 \times N_{\text{reac}} \times 3$.
  The units of each column are indicated in the dataset attributes.

4 Auxiliary input files

This section gives detailed information on the configuration of the different auxiliary input files used by LoKI, namely on the notation to be adopted in the description of the species, the electron-scattering mechanisms, and the heavy-species kinetics involved in the simulations.

The auxiliary input files should be located in the input folder `[repository folder]/LoKI-GM_26.07/Code/Input/` or in any sub-folder within it. They are either plain text files or JSON files, although some can display extensions different than `*.txt` and `*.json`, for the sake of folder organisation.

4.1 LXCat files

By default, LoKI-B uses electron-scattering cross sections obtained from the LXCat open-access website [14], in the solution to the EBE or as *extra* data for integration over the calculated EEDF, to obtain electron macroscopic parameters. The cross sections can be **assembled into a single file or given from multiple files**, and if/when **appearing duplicated in the datafiles they are considered only once by LoKI**.

LXCat is organised to provide "data required for modeling low temperature plasmas" [14] in the most effective way. In the case of electron-neutral scattering cross sections, LXCat provides easy access to *complete sets* of cross sections, defined as those giving a good description of all electron energy and momentum losses [26], yielding electron swarm parameters in agreement with available measured data (within experimental uncertainties), when used in a two-term Boltzmann solver [26, 27]. Each *complete set* assembles cross sections for electron collisions with different neutral targets, usually associated with the electronic ground-state of each gas. To date, on LXCat these data are tagged with the name of the corresponding gas (*e.g.* `Ar`, `N2`, `O2`, `CO2`, $\cdots$) and are grouped under the category `Ground states`, where one can find cross sections for the collisions of electrons with the electronic / vibrational / rotational ground-states of a neutral gas. For example, the datasets `N2` may contain (i) the elastic / effective momentum-transfer cross section for N_2; (ii) excitation cross sections for different electronic states (including the ionisation state), from the electronic ground-state $N_2(X)$; (iii) excitation cross sections for different vibrational states $N_2(X,v)$, from the vibrational ground-state $N_2(X,v=0)$; (iv) a global rotational excitation cross section for $N_2(X)$.

This practical organisation of LXCat can create some difficulties when a detailed discrimination and handling of data is intended. The reason is of technical nature, and it relates with the fact that, to date, electron collisions with the `Ground state` of a gas are described as a collection of mechanisms `e + gas name ->` $\cdots$, with no possibility to specify the internal structure of the electronic ground-state of the gas. Meaning that the information, if any, about the different species involved in each electron collision is scattered across the LXCat files, depending on the strategy adopted by each particular LXCat contributor. Obviously, this makes impossible to develop a general parser to collect the full list of species (with internal structure), for any file downloaded from LXCat.

The various issues mentioned above, regarding the organisation of LXCat, as well as the detailed classification and handling of data, are being addressed by the platform's developers, as already demonstrated in the *demo* version of LXCat 3.0 [15].

Meanwhile users can modify any cross section **text-file** downloaded from LXCat, to make it compliant with the LoKI-B parser. For this, it is important to understand how LXCat structures the metadata associated with each cross section. In an LXCat text-file, each cross-section-table comes with a header including the following fields:

- `SPECIES`
- `PROCESS`
- `PARAM.`
- `COMMENT` (multiple lines)
- `UPDATED`
- `COLUMNS`.

In principle, LoKI-B should check the `PROCESS` field to obtain details on the full structure of reactants and products, for each electron-impact mechanism, but again this is not available in the current structure of LXCat metadata. To circumvent this limitation, the LoKI-B parser looks for this information in the first line of the free-field `COMMENT`, which is modified to meet the following structure:

```
COMMENT:␣[<electron>␣+␣<target state>␣<directionality>␣<products states>,␣<collision
                        type>,␣<collision subtype>] ...
```

where the square brackets and spaces are mandatory characters, and where the different elements in the expression have the following features:

- `electron:` can be represented by either characters `e` or `E` (see section 2.3.1 for more information about the description of the species involved in electron-scattering mechanisms).
- `target state:` can detail the electronic/vibrational/rotational state of the target species (see section 2.3.1 for more information about the description of the species involved in electron-scattering mechanisms).
- `directionality:` can be a forward arrow (`->`) for inelastic collisions, or a double arrow (`<->`) for inelastic/superelastic collisions.
- `product states:` sum of all the product(s) resulting from an electron collision, including electron(s) and heavy-species, eventually with indication of the electronic/vibrational/rotational level (see section 2.3.1 for more information about the description of the species involved in electron-scattering mechanisms).
- `collision type:`[3] one of the following collision types, supported by the LXCat database: `Elastic`, `Effective`, `Excitation`, `Vibrational`, `Rotational`, `Ionization` or `Attachment`. Note the **mandatory** capital letters, since the parser is case sensitive.
- `collision subtype:` (optional field) either empty for integral cross sections, or `momentum-transfer` for momentum-transfer cross sections.
  Note the following exception: the collision types `Elastic` and `Effective` are automatically handled as `momentum-transfer`, even if the collision subtype if left empty.

The workaround described above is the practical solution to prepare any cross section **text-file** obtained from LXCat for compliance with the LoKI-B parser, and it is already implemented in the following files:

- the LXCat **text-files** distributed with the code, and located in `[repository folder]/LoKI-GM_26.07/Code/Input/` (see section 2.2).

  Before modifying any other LXCat file, the user should check the text files distributed with the code, to become familiarised with the modifications needed.

- cross section **text-files**, **directly downloadable only from the IST-Lisbon database with the website LXCat, and to date only for the gases indicated below**[4]
  - Ammonia: `NH3` for electron excitations from the electronic ground-state $NH_3(X)$; `NH3-rot` for electron excitations from rotational state $NH_3(X,v=0,J=0)$ $\Delta J = 1 - 5$;
  - Argon: `Ar`, for electron excitations from the ground-state $Ar(1S_0)$;
  - Carbon monoxide: `CO`, for electron excitations from the electronic ground-state CO(X) and the vibrational ground-state CO(X,v=0); `CO-rot` for electron excitations from rotational states CO(X,v=0,J=0-17) $\Delta J = 1$;
  - Carbon monoxide (with anisotropic scattering)[5]: `CO_anis`, for electron excitations from the electronic ground-state CO(X) and the vibrational ground-state CO(X,v=0), to be adopted when anisotropic scattering for rotational collisions is also considered; `CO_dipint-rot` delivering dipole-integral cross

[3] LXCat 3.0 will extend the collision types supported, (i) by adding: `MomentumTransfer`, `Electronic`, `Dissociative`, `Recombination`, `Dettachment` (to be introduced); and (ii) by allowing combinations between different collision types / subtypes, *e.g.* `Elastic MomentumTransfer`; `Electronic Excitation MomentumTransfer`.

[4] LoKI is distributed with only part of the data available on the IST-Lisbon database at LXCat.

[5] This complete set of cross sections is the result of the work of L. Vialetto *et al* [19], which proposes a model of anisotropic scattering for rotational collisions, to be used in a two-term Boltzmann solver.

sections for electron excitations from rotational states CO(X,v=0,J=0-25) $\Delta J = 1$; `CO_dipmt-rot` delivering dipole-momentum-transfer cross sections for electron excitations from rotational states CO(X,v=0,J=0-25) $\Delta J = 1$; `CO_quadint-rot` delivering quadrupole-integral cross sections for electron excitations from rotational states CO(X,v=0,J=0-24) $\Delta J = 2$.
These datagroups are clustered under `CO_anis` in `State-specific and gas mixtures` at LXCat;

- Carbon dioxide: `CO2`, for electron excitations from the electronic ground-state CO_2(X) and the vibrational ground-state CO_2(X,v=000);
- Helium: `He`, for electron excitations from the ground-state He(1^1S);
- Hydrogen: `H2`, for electron excitations from the electronic ground-state H_2(X) and the vibrational ground-state H_2(X,v=0); `H2-vib` for electron excitations from vibrational states H_2(X,v= $1-15$); `H2-rot` for electron excitations from rotational states H_2(X,v=0,J=0-5) $\Delta J = 2$; `H2-elec` for electron recombination of ionic states H_2^+ and H_3^+.
The latter three datagroups are clustered under `Hydrogen` in `State-specific and gas mixtures` at LXCat;
- Atomic Hydrogen: `H`, for electron excitations from the ground-state H(1s); `H-elec`, for electron excitations from the excited states H(2s), H(2p), H(3), H(4) and H(5).
These datagroups are clustered under `Hydrogen` in `State-specific and gas mixtures` at LXCat;
- Nitrogen: `N2`, for electron excitations from the electronic ground-state N_2(X) and the vibrational ground-state N_2(X,v=0); `N2-vib` for electron excitations from vibrational states N_2(X,v= $1-10$); `N2-rot` for electron excitations from rotational states N_2(X,v=0,J=0-30) $\Delta J = 2$.
The latter two datagroups are clustered under `Nitrogen` in `State-specific and gas mixtures` at LXCat;
- Atomic Nitrogen: `N`, for electron excitations from the ground-state N(^{4}S); `N-elec`, for electron excitations from the excited states N(^{2}D) and N(^{2}P).
These datagroups are clustered under `Nitrogen` in `State-specific and gas mixtures` at LXCat;
- Oxygen: `O2`, for electron excitations from the electronic ground-state O_2(X) and the vibrational ground-state O_2(X,v=0); `O2-vib` for electron excitations from vibrational states O_2(X,v= $0-41$); `O2-rot` for electron excitations from rotational states O_2(X,v=0,J=1-29) $\Delta J = 2$; `O2-elec` for electron excitations from excited electronic states of O_2, namely $O_2(a^1\Delta_g, b^1\Sigma_g^+)$.
The latter three datagroups are clustered under `Oxygen` in `State-specific and gas mixtures` at LXCat;
- Atomic Oxygen: `O`, for electron excitations from the ground-state O(^{3}P); `O-elec` for electron excitations from excited electronic states of O, namely O^-.
These datagroups are clustered under `Oxygen` in `State-specific and gas mixtures` at LXCat;
- Ozone: `O3`, for electron excitations from the ground-state O_3(X).
This datagroup is under `Oxygen` in `State-specific and gas mixtures` at LXCat.

In the folder `[repository folder]/LoKI-GM_26.07/Code/OtherAuxFunctions` the user can find the script `LXCat2LoKI.m` that interactively assists the modification of LXCat **text-files**, for compliance with the LoKI-B parser. Code 9 shows an example of a modified header, for a cross-section-table extracted from the `N2_LXCat.txt` file distributed with LoKI.

```
EXCITATION
N2 -> N2 (v=0 - v=1)
 3.000000e-1
SPECIES: e / N2
PROCESS: E + N2 -> E + N2 (v=0 - v=1), Excitation
PARAM.:  E = 0.3 eV, complete set
COMMENT: [e + N2(X,v=0) <-> e + N2(X,v=1), Vibrational] Pitchford L C and Phelps A V 1982
COMMENT: Bull. Am. Phys. Soc. 27 109 Tachibana K and Phelps A V 1979 JCP 71 3544 Schulz G J 1962
COMMENT: Phys. Rev. 125 229 Schulz G J 1964 Phys. Rev. 135 A988 Schulz G J 1973 Rev. Mod. Phys.
COMMENT: 45 423 Engelhardt A G, Phelps A V and Risk C G 1964 Phys. Rev. 135 A1566 Pavlovic Z,
COMMENT: Boness M J W, Herzenberg A and Schulz G J 1972 Phys. Rev. A 6 676.
UPDATED: 2017-09-03 10:54:40
COLUMNS: Energy (eV) | Cross section (m2)
-----------------------------
 3.000000e-1  0.000000e+0
 4.000000e-1  3.000000e-23
```

```
 5.000000e-1   4.000000e-23
 ...
 9.653000e+2   1.238000e-24
 9.654000e+2   0.000000e+0
 1.000000e+3   0.000000e+0
-----------------------------
```

Code 9: Extract from the `N2_LXCat.txt` file distributed with LoKI (cf the modified `COMMENT` field)

Note that the parsing of electron-scattering cross sections is a component of LoKI-B. When LoKI-C runs independently, the parsing of electron-scattering cross sections with LoKI-B is not available.

LXCat 3.0 enables downloading cross-section files in both text and JSON formats, both of which are now supported by `LoKI-B_25.10`. The LXCat **JSON-files** are fully compatible with the LoKI-B parser, which no longer uses the information in the first line of the free-field `COMMENT`, reading instead the following fields:

- `state`, `reversible` and `typeTags`, within `processes > reaction`;
- `threshold` and `values > data`, within `processes > info`.

Currently, collision type `Electronic` (supported by LXCat 3.0) is automatically converted by `LoKI-B_25.10` into `Excitation`, as supported by current version of LXCat. Work is in progress to consolidate the transcription of LXCat data between versions 2.0 and 3.0, and the corresponding data parsing in numerical codes.

4.2 Property files

Property files are text files with the properties of gases and states used in the simulations. Some property files are distributed with LoKI, but in general these files can be prepared by the user, considering the following rules:

- for organisation sake, all "property files" specified in the setup file should be located inside the input folder (`[repository folder]/LoKI-GM_26.07/Code/Input`), eventually organised in subfolders with self-explanatory names (*e.g.* `Nitrogen`, `Databases`; see section 2.2);
- property files can include comments, signaled by the character `%`;
- property files **MUST** be two-column-formatted, separated by any white-space character.
  The first column should include the gas/state name (see section 2.3.1 for more information about the description of the species used in the simulations), and the second column should include the value of the property for that particular gas/state;
- property files accept basic mathematical operations (such as addition, subtraction, multiplication and division), in the second column with the value of the property (see Code 10 as an example);
- property files can include the properties of gases/states not used in the simulations. LoKI handles input files as "databases", where it searches only for the information required.

Codes 10 and 11 show two examples of property files distributed with LoKI.

```
% masses of different gases expressed in SI units (Kg)
N3    3*14.007*1.660539040e-27
N4    4*14.007*1.660539040e-27
N2    2*14.007*1.660539040e-27
N     14.007*1.660539040e-27
O2    2*15.999*1.660539040e-27
O     15.999*1.660539040e-27
O3    3*15.999*1.660539040e-27
H2    2*1.008*1.660539040e-27
H     1.008*1.660539040e-27
H3    3*1.008*1.660539040e-27
CO2   (12.0107+2*15.999)*1.660539040e-27
```

```
CO    (12.0107+15.999)*1.660539040e-27
Ar    39.948*1.660539040e-27
He    4.002602*1.660539040e-27
He2   8.005204*1.660539040e-27
H2O   (2*1.008+15.999)*1.660539040e-27
CH4   16.0427*1.660539040e-27
C     12.0107*1.660539040e-27
N2H   (2*14.007+1.008)*1.660539040e-27
NH4   (14.007+4*1.008)*1.660539040e-27
NH3   (14.007+3*1.008)*1.660539040e-27
NH2   (14.007+2*1.008)*1.660539040e-27
NH    (14.007+1.008)*1.660539040e-27
N2H4  (2*14.007+4*1.008)*1.660539040e-27
N2H3  (2*14.007+3*1.008)*1.660539040e-27
N2H2  (2*14.007+2*1.008)*1.660539040e-27
```

Code 10: Property file with the masses of different gases (`Databases/masses.txt`)

```
% populations of the different vibrational states of N2
N2(X,v=0)   1
N2(X,v=1)   0
N2(X,v=2)   0
N2(X,v=3)   0
N2(X,v=4)   0
N2(X,v=5)   0
N2(X,v=6)   0
N2(X,v=7)   0
N2(X,v=8)   0
N2(X,v=9)   0
N2(X,v=10)  0
```

Code 11: Property file with a vibrational distribution of N_2 (`Nitrogen/N2_vibpop.txt`)

4.3 Chemistry files

Chemistry files are text files listing the different reactions to be included in the LoKI-GM and LoKI-C simulations. Some chemistry files are distributed with LoKI, but in general these files can be prepared by the user, considering the following rules:

- for aesthetic and organisation purposes, "chemistry files" adopt the file extension `.chem`.
- for organisation sake, all "chemistry files" specified in the setup file should be located inside the input folder (`[repository folder]/LoKI-GM_26.07/Code/Input`), eventually organised in subfolders with self-explanatory names (*e.g.* `Nitrogen`, `Oxygen`; see section 2.2);
- chemistry files can include comments, signaled by the character `%`;
- each entry in a chemistry file **MUST** have the following structure:

  `<reactants> <directionality> <products>` | `<rate coefficient>` | `<parameters>` | `<enthalpy>`

  (Reaction description | Type of rate coefficient | Parameters needed | Reaction enthalpy)

  where

  - `<reactants>` is the field with the reactant or the sum of reactant species involved in the reaction. **Note that this field should not be empty, in which case the code returns an error.** For more information about the description of the species involved in a reaction see section 2.3.1;
  - `<directionality>` can be a forward arrow (`->`), when only the direct reaction is considered, or a double arrow (`<->`), when both the direct and the reverse (detailed balance) reactions are considered. For electron-impact reactions with `eedf` rate coefficients and for two-body reactions between volume heavy species, when a double arrow is detected the `detailedBalance` function evaluates and returns the reverse rate coefficient. For other double-arrow reactions, the `detailedBalance` function checks

if the collision property `backRateCoeff` is not empty, in which case it returns the reverse rate coefficient.
When using double-arrow in a reaction, users must ensure that the corresponding rate coefficient function evaluates both the direct and the reverse rate coefficients (see `eedfcopyv0` as an example);

- `<products>` is the field with the product or the sum of product species resulting from the reaction. **Note that this field should not be empty, in which case the code returns an error.** For more information about the description of the species involved in a reaction see section 2.3.1;
- `<rate coefficient>` is the field with the name of the function that evaluates the rate coefficient of the corresponding reaction. For more information about rate coefficient functions see section 5.2;
- `<parameters>` is a list of comma separated values with all the parameters (if any) required for the corresponding rate coefficient function. For more information about the parameters needed for each rate coefficient functions see section 5.2;
- `<enthalpy>` is the field with the value of the reaction enthalpy (positive or negative). If this field is missing, then the reaction enthalpy is assumed to be zero; if the value of the enthalpy is to be calculated from the difference between the energies of `<products>` and `<reactants>`, then the keyword `level` should be used in this field.

  The enthalpy information is only needed for the thermal model. However, every entry of the chemistry files should follow the previous structure, even if the thermal model is not activated.

- LoKI-C ignores multiple descriptions of the same reactions (*i.e.* reactions with the same set of `<reactants>` and `<products>`), **returning a warning message**.

Before creating a chemistry file, it is recommended that the user checks out those distributed with the code (*e.g.*, `[repository folder]/Code/Input/Nitrogen/nitrogen.chem` or `[repository folder]/Code/Input/Oxygen/oxygen.chem`), so that they become familiar with the writing of the different chemical reactions already implemented in LoKI-C. As an example, Code 12 shows an extract of a chemistry file distributed with LoKI (`nitrogen_mod.chem`) that contains a simplified chemistry scheme for Nitrogen.

```
%---- electron impact excitation/deexcitation ----
e + N2(X,v=0) <-> e + N2(A3Su+)          | eedf                  |          |
...
e + N(4S) <-> e + N(2D)                  | eedf                  |          |
...

%---- e-V processes ----
e + N2(X,v=0) <-> e + N2(X,v=1)          | eedf                  |          |
...
e + N2(X,v=1:9) -> e + N2(X,v=v+1)       | nitrogenEVscaling     |          |
...

%---- electron impact dissociation ----
e + N2(X,v=0) -> e + 2N(4S)              | eedf                  |          | 2.24
e + N2(X,v=0) -> e + N(4S) + N(2D)       | eedf                  |          | 0.00

%---- electron impact ionization ----
e + N2(X,v=0) -> 2e + N2(+,X)            | eedf                  |          |
...
e + N(4S) -> 2e + N(+,gnd)               | eedf                  |          |
...

%---- electron recombination ----
e + N2(+,X) -> 2N(4S)                    | powerElectronTemp     | (11600^(-0.5))*2.80E-13*(300)^0.5, -0.5 | 5.82
...

%---- heavy species collisions ----
N2(B3Pg) + N2(X) -> N2(A3Su+) + N2(X)    | constantRateCoeff     | 0.95*3e-17                | 1.18
...
N2(a'1Su-) + N2(X) -> N2(B3Pg) + N2(X)   | constantRateCoeff     | 1.9e-19                   | 1.0
...
N(2D) + N(2P) -> N2(+,X) + e             | constantRateCoeff     | 1e-19                     |
...
N(4S) + N(4S) + N2(X) -> N2(B3Pg) + N2(X) | arrheniusGasTemp      | 8.27E-46, 500             | 2.36
...
```

```
N2(+,X) + N2(X,v=0) + N2(X) -> N4(+,X) + N2(X) | powerGasTemp       | 6.80E-41*300^1.64, -1.64    | 1.05
...
N4(+,X) + N2(X) -> N2(+,X) + N2(X,v=0) + N2(X) | expGasTemp         | 2.1e-22, 121                | -1.05
...

%---- V-V processes ----
N2(X,v=1:10) + N2(X,w=0:v-1) <-> N2(X,v=v-1) + N2(X,v=w+1)    | nitrogenMolecularVV     |         | level
N2(X,v=1:10) + N2(X,v=10) -> N2(X,v=v-1) + 2N(4S)             | nitrogenMolecularVVDis  |         |

%---- Molecular V-T processes ----
N2(X,v=1:10) + N2(X) <-> N2(X,v=v-1) + N2(X)                  | nitrogenMolecularVT     |         | level
N2(X,v=10) + N2(X) -> 2N(4S) + N2(X)                          | nitrogenMolecularVTDis  |         |

%---- Atomic V-T processes ----
N2(X,v=7:10) + N(4S) <-> N2(X,v=v-1) + N(4S)                  | nitrogenAtomicVT        |         | level
...
N2(X,v=10) + N(4S) -> 2N(4S) + N(4S)                          | nitrogenAtomicVTDis     | 11      |
...

%---- Neutral transport ----
N2(A3Su+) + wall -> N2(X,v=0)             | binaryDiff              | 1                           | 6.2
...
N(4S) + wall -> 0.5N2(X,v=0)              | wallReaction            | 1e-1, nearWallTemperature       | 9.76/2

%---- Wall vibrational deactivation ----
N2(X,v=1:10) + wall -> N2(X,v=v-1)        | wallReaction            | 0.0011, nearWallTemperature     | level

%---- Ion transport ----
N4(+,X) + wall -> 2N2(X,v=0)              | classicalAmbipolarDiff  | 1                           |
...
```

Code 12: Extract of the Nitrogen chemistry file (`Nitrogen/nitrogen_mod.chem`)

5 Auxiliary functions

This section describes several auxiliary functions used internally by LoKI-GM to perform the simulations.

5.1 Property functions

Property functions are `*.m` functions that define the properties of gases and states used in the simulations. Some property functions are distributed with LoKI, but in general these functions can be prepared by the user, according to the needs for new functionalities. In this case, the user is advised to mimic the structure of any property function already provided with the code.

The following should be considered:

- for organisation sake, all "property functions" specified in the setup file should be located in the property functions folder (`[repository folder]/LoKI-GM_26.07/Code/PropertyFunctions`; see section 2.2);
- property functions have the arguments

  (`state`, `argumentArray`, `workCond`) or

  (`gas`, `argumentArray`, `workCond`),

  where
  - `state` or `gas`: is a handle to the state/gas-object(s) to which the property is assigned. For example, when using `N2(X,v=*)` the property is assigned to all the vibrational states with the electronic ground-state of N_2;
  - `argumentArray`: is an array of arguments corresponding to the list of parameters to be used in the setup file (see below);
  - `workCond`: is a handle to the working-conditions-object of the simulation.
- property functions are called from the setup file (see section 3) using the following structure:

  `<function name>` $\underbrace{\texttt{@[list of parameters]}}_{\text{Optional}}$

  where the `list of parameters` is a list of comma-separated values with all the parameters called by the particular property function. The parameters sent to the property functions can be mapped to the values of the different working conditions previously defined in the setup file (see Code 4 and Code 5 for examples, and therein the implementation of functions `boltzmannPopulation@gasTemperature` and `generalizedTemperatureDependentCoeff@5.0e20/sqrt(300),gasTemperature,0.5`);
- property functions **MUST** be set only for those gases and states used in the simulations, *i.e.* those listed in the specified LXCat files and chemistry files .

Table 3 lists the most relevant property functions distributed with `LoKI-GM_26.07`, along with the required parameters and dependencies.

The property functions in table 3 correspond to the following physical models.

`harmonicOscillatorEnergy`

It calculates the energy of (vibrational) state j of a diatomic molecule, adopting the harmonic oscillator model

$$E_j = \frac{\hbar\omega_{\text{vib}}}{e}\left(j+\frac{1}{2}\right) \quad , \tag{10}$$

where ω_{vib} is the `harmonicFrequency` (angular frequency) of the molecule.

`morseOscillatorEnergy`

It calculates the energy of (vibrational) state j of a diatomic molecule, adopting the Morse oscillator model

$$E_j = \frac{\hbar\omega_{vib}}{e}\left(j+\frac{1}{2}\right) - \frac{\hbar\omega_{\text{vib}}\,\chi_e}{e}\left(j+\frac{1}{2}\right)^2 \quad , \tag{11}$$

where $\omega_{\text{vib}}\,\chi_e$ is the first `anharmonicFrequency` of the molecule (with χ_e the anharmonicity constant).

Table 3: List of property functions distributed with LoKI-GM_26.07

Property	Function name	Parameters [units]	Dependencies
energy	harmonicOscillatorEnergy	n/a	harmonicFrequency
	morseOscillatorEnergy	n/a	harmonicFrequency anharmonicFrequency
	rigidRotorEnergy	n/a	rotationalConstant
	rigidRotorEnergy_NH3	n/a	rotationalConstant
statisticalWeight	rotationalDegeneracy	n/a	n/a
	rotationalDegeneracy_N2	n/a	n/a
	rotationalDegeneracy_H2	n/a	n/a
	rotationalDegeneracy_NH3	n/a	n/a
population	boltzmannPopulation	Temperature [K]	energy statisticalWeight
	boltzmannRotationalPopulation_H2	Temperature [K]	energy statisticalWeight
	boltzmannPopulationRotationalCutoff	Temperature [K] J_{MAX}	energy statisticalWeight
	boltzmannPopulationVibrationalCutoff	Temperature [K] v_{MAX}	energy statisticalWeight
	treanorPopulation	Temperature0 [K] Temperature1 [K]	energy statisticalWeight
	treanorGordietsPopulation	Temperature0 [K] Temperature1 [K]	energy statisticalWeight anharmonicFrequency
reducedMobility	generalizedTemperatureDependentCoeff	Temperature [K]	n/a
reducedDiffCoeff			
...	aT^b	a [S.I. (according to T)], b [ ]	
reducedMobility	mobilityHeliumAtomicIon	Temperature [K]	n/a
...	$a/(bT^{0.5}+c)$	a, b, c [S.I. (according to T)]	
reducedDiffCoeff	diffusionHeliumAtomicIon	Temperature [K]	n/a
...	$a/(bT^{0.5}+c)(k_BT/e)$	a, b, c [S.I. (according to T)]	
reducedMobility	tableEoNDependentMobility	file with table of	n/a
...	interpolates μN for the current E/N value	μN [V^{-1} m^{-1} s^{-1}], E/N [V m^2]	
reducedDiffCoeff	temperatureDependentEinsteinDiffCoeff	Temperature [K]	reducedMobility
...	$\mu N(k_BT/e)$		
HeatCapacity	nitrogenHeatCapacity	n/a	n/a
	oxygenHeatCapacity	n/a	n/a
	oxygenAtomHeatCapacity	n/a	n/a
	ozoneHeatCapacity	n/a	n/a
ThermalConductivity	nitrogenThermalConductivity	n/a	n/a
	oxygenThermalConductivity	n/a	n/a
	oxygenAtomThermalConductivity	n/a	n/a

rigidRotorEnergy

It calculates the energy of (rotational) state J of a diatomic molecule, adopting the rigid-rotor model

$$E_J = BJ(J+1) \quad , \tag{12}$$

where B is the rotational constant of the molecule.

`rigidRotorEnergy_NH3`
it calculates the energy of (rotational) states (J, K) with $K = 0$ of NH_3, following the work of Dowling [28], which shows good agreement with the MARVEL database [29, 30]

$$t = hc/e \tag{13a}$$
$$E_J = BJ(J+1) - 8.407 \times 10^{-2}\, t(J(J+1))^2 + 2.38 \times 10^{-5}\, t(J(J+1))^3 \quad . \tag{13b}$$

These property functions should be called within the `energy` property of the setup file, for the corresponding vibrational and rotational states.
For example:
`N2(X,v=*) = harmonicOscillatorEnergy`
to set the energies of all the vibrational states with the electronic ground-state of N_2.
`N2(X,v=0,J=*) = rigidRotorEnergy`
to set the energies of all the rotational states of the vibrational state v=0, with the electronic ground-state of N_2.

`rotationalDegeneracy`
It calculates the statistical weight of the *intrinsic* partition function of a (generic) rotational state J [20]

$$g_J = 2J + 1 \quad . \tag{14}$$

`rotationalDegeneracy_N2`
It calculates the statistical weight of the *intrinsic* partition function of a rotational state J of the N_2 molecule [20]

$$g_J = 3\left[1 + \frac{1}{2}\left(1 + (-1)^J\right)\right](2J+1) \quad . \tag{15}$$

`rotationalDegeneracy_H2`
It calculates the statistical weight of the *intrinsic* partition function of a rotational state J of the H_2 molecule [20]

$$g_J = \left[2 - (-1)^J\right](2J+1) \quad . \tag{16}$$

`rotationalDegeneracy_NH3`
it calculates the statistical weight of the *intrinsic* partition function of a rotational state (J, K), with $K = 0$, of the vibrational ground-state $(v1 = 0, v2 = 0, v3 = 0, v4 = 0)$ of NH_3, considering that the state $(0, 0)$ has a double degeneracy [29]

$$g_J = 2(2J+1) \text{ for } J = 0 \tag{17a}$$
$$g_J = 2J + 1 \text{ for } J = 1, 2, 3, \ldots \quad . \tag{17b}$$

These property functions should be called within the `statisticalWeight` property of the setup file, for the corresponding rotational states (see also section 3.2).
For example:
`N2(X,v=0,J=*) = rotationalDegeneracy_N2`
to set the statistical weights of all the rotational states of the vibrational state v=0, with the electronic ground-state of N_2.

`boltzmannPopulation`
It calculates the population of state j, adopting the Boltzmann distribution

$$\xi_j = \frac{g_j \, \exp\left(-\frac{E_j}{k_B T}\right)}{\sum_{i \in \text{siblings}} g_i \, \exp\left(-\frac{E_i}{k_B T}\right)} \quad , \tag{18}$$

where T is the characteristic temperature of the distribution.

`boltzmannRotationalPopulation_H2`

It calculates the population of rotational state j of H_2, adopting Boltzmann distributions for the ortho/para configurations, with weights 25% and 75%, respectively

$$\xi_{j_{\text{even}}} = 0.25 \frac{g_{j_{\text{even}}} \exp\left(-\frac{E_{j_{\text{even}}}}{k_B T}\right)}{\sum_{i \in \text{siblings}} g_{i_{\text{even}}} \exp\left(-\frac{E_{i_{\text{even}}}}{k_B T}\right)} \tag{19a}$$

$$\xi_{j_{\text{odd}}} = 0.75 \frac{g_{j_{\text{odd}}} \exp\left(-\frac{E_{j_{\text{odd}}}}{k_B T}\right)}{\sum_{i \in \text{siblings}} g_{i_{\text{odd}}} \exp\left(-\frac{E_{i_{\text{odd}}}}{k_B T}\right)} \quad , \tag{19b}$$

where T is the characteristic temperature of the distribution.

`boltzmannPopulationRotationalCutoff`

It calculates the population of rotational state J, adopting a Boltzmann distribution truncated at level J_{MAX}

$$\xi_J = \frac{g_J \exp\left(-\frac{E_J}{k_B T}\right)}{\sum_{J'=0}^{J_{\text{MAX}}} g_{J'} \exp\left(-\frac{E_{J'}}{k_B T}\right)} \quad , \tag{20}$$

where T is the characteristic temperature of the distribution.
This distribution allows defining the populations of rotational states to be considered in the solution to the electron Boltzmann calculation, irrespectively of the full list of states involved in electron collision events (for example, appearing also in extra cross section files).

`boltzmannPopulationVibrationallCutoff`

It calculates the population of vibrational state v, adopting a Boltzmann distribution truncated at level v_{MAX}

$$\xi_v = \frac{g_v \exp\left(-\frac{E_v}{k_B T}\right)}{\sum_{v'=0}^{v_{\text{MAX}}} g_{v'} \exp\left(-\frac{E_{v'}}{k_B T}\right)} \quad , \tag{21}$$

where T is the characteristic temperature of the distribution.
This distribution allows defining the populations of vibrational states to be considered in the solution to the electron Boltzmann calculation, irrespectively of the full list of states involved in electron collision events (for example, appearing also in extra cross section files).

`treanorPopulation`

It calculates the population of state j, adopting the Treanor distribution [31, 32]

$$\xi_j = \frac{g_j \exp\left(\frac{j(E_1-E_0)-(E_j-E_0)}{k_B T_0}\right) \exp\left(-\frac{j(E_1-E_0)}{k_B T_1}\right)}{\sum_{i \in \text{siblings}} g_i \exp\left(\frac{i(E_1-E_0)-(E_i-E_0)}{k_B T_0}\right) \exp\left(-\frac{i(E_1-E_0)}{k_B T_1}\right)} \quad , \tag{22}$$

where T_0 and T_1 are the characteristic temperatures of the distribution.

`treanorGordietsPopulation`

It calculates the population of state j, adopting the Treanor-Gordiets distribution [31, 32]

$$\xi_j = \begin{cases} \dfrac{g_j \exp\left(\frac{j(E_1-E_0)-(E_j-E_0)}{k_B T_0}\right) \exp\left(-\frac{j(E_1-E_0)}{k_B T_1}\right)}{\sum_{i \in \text{siblings}} \xi_i} & , j \leq j^\star \\ \dfrac{\xi_{j^\star} \frac{j^\star}{j}}{\sum_{i \in \text{siblings}} \xi_i} & , j > j^\star \end{cases} \quad , \tag{23}$$

where T_0 and T_1 are the characteristic temperatures of the distribution and

$$j^\star \equiv \frac{1}{2}\left[1 + \frac{E_1 - E_0}{\hbar \omega_{\text{vib}} \chi_e} \frac{T_0}{T_1}\right]$$

is the vibrational level corresponding to the minimum of the Treanor distribution (with $\omega_{\text{vib}}\chi_e$ the anharmonic frequency of the oscillator characterizing the distribution).

These property functions should be called within the `population` property of the setup file, for the corresponding states, using
`boltzmannPopulation@<temperature>`
`boltzmannPopulationRotationalCutoff@<temperature>,<J_MAX>`
`treanorPopulation@<temperature0>,<temperature1>`
where `temperature,temperature0,temperature1` can be temperature values or the dynamic variables `gasTemperature` or `electronTemperature`.
For example
`N2(X,v=*) = treanorPopulation@gasTemperature,4000`
to set the population of all the vibrational states with the electronic ground-state of N_2;
`N2(X,v=0,J=*) = boltzmannPopulation@gasTemperature`
to set the population of all the rotational states of the vibrational state v=0, with the electronic ground-state of N_2.

`generalizedTemperatureDependentCoeff`
It calculates a general property (*e.g.*, `reducedMobility` or `reducedDiffCoeff`)

$$aT^b \quad , \tag{24}$$

where the normalization constant a, the temperature T and the power b should be specified as parameters to the function.

`mobilityHeliumAtomicIon`
It calculates the `reducedMobility` of the helium atomic ion [33]

$$\frac{a}{bT^{0.5}+c} \quad , \tag{25}$$

where the normalization constant a, the temperature T and the quantities b, c should be specified as parameters to the function.

`diffusionHeliumAtomicIon`
It calculates the `reducedDiffCoeff` of the helium atomic ion [33]

$$\frac{a}{bT^{0.5}+c}\frac{k_B T}{e} \quad , \tag{26}$$

where the normalization constant a, the temperature T and the quantities b, c should be specified as parameters to the function.

`tableEoNDependentMobility`
It interpolates the `reducedMobility` for the current value of the reduced electric field, from a table with filename specified as parameter to the function.

`temperatureDependentEinsteinDiffCoeff`
It calculates the `reducedDiffCoeff` from the Einstein relation

$$\mu_e N \frac{k_B T}{e} \quad , \tag{27}$$

where the temperature T should be specified as parameter to the function, and the reduced mobility $\mu_e N$ is obtained from the `workingConditions`.

These property functions should be called within the `reducedMobility` and `reducedDiffCoeff` properties of the setup file, for the corresponding states, using for example

```
reducedMobility:
   - He(2S3) = generalizedTemperatureDependentCoeff@8.6155e+19,gasTemperature,0.5
   - He(+,1S2) = mobilityHeliumAtomicIon@2.68e21,2.96e-3,3.11e-2,gasTemperature
   - O(+,gnd) = tableEoNDependentMobility@Databases/IonMobilities/O+_O2.txt
reducedDiffCoeff:
   - He(+,1S2) = diffusionHeliumAtomicIon@2.68e21,2.96e-3,3.11e-2,gasTemperature
   - O2(+,X) = temperatureDependentEinsteinDiffCoeff@gasTemperature
```

`nitrogenHeatCapacity`
It calculates the nitrogen heat capacity [34]

$$C_p(\mathrm{N_2}) = 29.1 + \frac{2494.2}{553.4\sqrt{\pi/2}} \exp\left(-2\left(\frac{T_g - 1047.4}{553.4}\right)^2\right) \quad . \tag{28}$$

`oxygenHeatCapacity` and `oxygenAtomHeatCapacity` and `ozoneHeatCapacity`
It calculates the heat capacity of O_2, O and O_3 [35]

$$C_p(\mathrm{O, O_2, O_3}) = 8.314510\left(a_1 T_g^{-2} + a_2 T_g^{-1} + a_3 + a_4 T_g + a_5 T_g^2 + a_6 T_g^3 + a_7 T_g^4\right) \quad , \tag{29}$$

where $a_1, \ldots, a_7$ are NASA polynomial coefficients taken from [35]: p. 164 for atomic oxygen; p. 166 for molecular oxygen and ozone.

These property functions should be called within the `heatCapacity` property of the setup file, for the corresponding states, using for example
`heatCapacity`:

```
- O2 = oxygenHeatCapacity
```

`nitrogenThermalConductivity`
It calculates the nitrogen thermal conductivity [34]

$$\lambda_g(\mathrm{N_2}) = (1.717 + 0.084 T_g - 1.948 \times 10^{-5} T_g^2) \times 10^{-3} \quad . \tag{30}$$

`oxygenThermalConductivity` and `oxygenAtomThermalConductivity`
It calculates the thermal conductivity of O_2 and O [36]

$$\lambda_g(\mathrm{O, O_2}) = 10^{-4} \exp\left(a_1 \ln T_g + \frac{a_2}{T_g} + \frac{a_3}{T_g^2} + a_4\right) \quad , \tag{31}$$

where $a_1, \ldots, a_4$ are transport coefficients taken from [36] p. 86.

These property functions should be called within the `thermalConductivity` property of the setup file, for the corresponding states, using for example
`thermalConductivity`:

```
- O2 = oxygenThermalConductivity
```

5.2 Rate coefficient functions

Rate coefficient functions are `'*.m'` functions that compute the rate coefficients of the reactions included in the kinetic mechanisms. Some of these functions are distributed with LoKI, implementing the most common types of rate coefficients and useful models. In general these functions can be prepared by the user, according to the needs for new functionalities. In this case, the user is advised to mimic the structure of any rate coefficient function already provided with the code.

The following should be considered:

- For organisation sake, all "rate coefficient functions" specified in the different auxiliary input files should be located in the rate coefficient functions folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions`, which is organized into one-level subfolders with self-explanatory names (see section 2.2), for example:
  - `ElectronKinetics`
  - `General`
  - specific folders with functions for gases (`Nitrogen`, `Oxygen`, ...)
  - `SurfaceKinetics`
  - `TransportIons`

Table 4: List of rate coefficient functions distributed with `LoKI-GM_26.07` corresponding to simple expressions. These functions can be found in the folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions/General`.

Function name	Expression	Parameters in order [units]
`arrhenius`	$a \exp(b/T)$	T [as b], a [S.I.], b [as T]
`arrheniusElectronTemp`	$a \exp(b/T_e)$	a [S.I.], b [eV]
`arrheniusGasTemp`	$a \exp(b/T_g)$	a [S.I.], b [K]
`constantRateCoeff`	a	a [S.I.]
`einsteinCoeffFromOscStrength`	$2\pi\varepsilon_0 e^4/(m_e h^2 c) \times (u_i - u_j)^2 (g_j/g_i) f_{ji}$	f_{ji} [S.I.]
`expElectronTemp`	$a \exp(T_e/b)$	a [S.I.], b [eV]
`expGasTemp`	$a \exp(T_g/b)$	a [S.I.], b [K]
`expTemp`	$a \exp(T/b)$	T [as b], a [S.I.], b [as T]
`modifiedArrhenius`	$aT^b \exp(c/T)$	T [as c], a [S.I.], b [ ], c [as T]
`modifiedArrheniusCharEnergyTemp`	$a{u_k}^b \exp(c/u_k)$	a [S.I. (u_k in eV)], b [ ], c [eV]
`modifiedArrheniusElectronTemp`	$aT_e^b \exp(c/T_e)$	a [S.I. (T_e in eV)], b [ ], c [eV]
`modifiedArrheniusGasTemp`	$aT_g^b \exp(c/T_g)$	a [S.I. (T_g in K)], b [ ], c [K]
`offsetArrheniusGasTemp`	$a_0 + a \exp(b/T_g)$	a_0, a [S.I.], b [K]
`orbitalExchange`	$a[(u_i - u_j)/T_g]^b/g_i \,, (u_i - u_j) < T_g$	a [S.I.], b [ ], c [K eV^{-1}]
	$a[(u_i - u_j)/T_g]^b \exp(-(u_i - u_j)c/T_g)/g_i \,, (u_i - u_j) > T_g$	a [S.I.], b [ ], c [K eV^{-1}]
`powerElectronTemp`	aT_e^b	a [S.I. (T_e in eV)], b [ ]
`powerGasCharEnergyTemp`	$aT_g^b u_k^c$	a [S.I. (u_k in eV; T_g in K)], b [ ], c [ ]
`powerGasElecTemp`	$aT_g^b T_e^c$	a [S.I. (T_e in eV; T_g in K)], b [ ], c [ ]
`powerGasTemp`	aT_g^b	a [S.I. (T_g in K)], b [ ]
`powerlnGasTemp`	$[a + b\ln(c/T_g)]T_g^d$	a, b [S.I. (T_g in K)], c [K], d [ ]
`powerTemp`	aT^b	T [any], a [S.I. (according to T)], b [ ]
`radiationImprisonmentEinsteinCoeff`	$A_{ij}\lambda_{ij}$	A_{ij} [s^{-1}]
`radiationImprisonmentOscStrength`	$2\pi\varepsilon_0 e^4/(m_e h^2 c) \times (u_i - u_j)^2 (g_j/g_i) f_{ji}\lambda_{ij}$	f_{ji} [S.I.]
`rateCoeffCopy`	K_r of `reactDescrip` r	`reactDescrip` r [ ]

  - `TransportNeutrals` .

- rate coefficient functions have the arguments (`time`, `densitiesAll`, `totalGasDensity`, `reaction`, `rateCoeffParams`, `chemistry`) where
  - `time`: is the current time at which the function is called;
  - `densitiesAll`: is an array with the density values of all the states involved in the simulations;
  - `totalGasDensity`: is the the current value of the total gas density;
  - `reaction`: is a handle to the reaction-object of the simulation;
  - `rateCoeffParams`: is an array of arguments corresponding to the list of parameters to be used in the calculation of the rate coefficient (see tables 4 and 5-9);
  - `chemistry`: is a handle to the chemistry-object of the simulation.

- rate coefficient functions are called from the auxiliary chemistry files (see section 4.3) using the following structure:

... | `<rate coefficient>` | `<parameters>` | ...

(`<rate coefficient>`: Type of rate coefficient; `<parameters>`: Parameters needed)

  where
  - `<rate coefficient>` is the field with the name of the function that computes the rate coefficient;
  - `<parameters>` is a list of comma separated values with all the parameters (if any) required for the corresponding rate coefficient function.

- rate coefficient functions **MUST** conclude with a list of dependencies `true`/`false`

  `onTime`

  `onDensities`

  `onGasTemperature`

  `onElectronKinetics`

  for example
  `dependent = struct('onTime', false, 'onDensities', false, 'onGasTemperature', false, 'onElectronKinetics', true)`

The functions supplied with the code can be divided into two categories: those corresponding to simple expressions (see table 4) and those corresponding to more elaborated implementations based on specific models (see tables 5-9).

Table 5: List of model-based rate coefficient functions distributed with `LoKI-GM_26.07` in folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions/ElectronKinetics`.

Function name	Model	Comments
`eedf`	Integrated electron-impact cross section over an eedf (either prescribed or obtained from the solution of the Boltzmann equation) (see Reference Manual [12])	The corresponding electron-impact cross section must be provided in the setup file.
`eedfCopyv0`	Evaluates the rate coefficient of a reaction involving vibrational states (either as reactants or products), by copying the rate coefficient of a similar reaction involving the corresponding parent state.	
`pseudoEedfCopy`	Evaluates a rate coefficient of an electron-impact reaction by setting it equal to the rate coefficient of another reaction using the reaction's description as input parameter	The description of the original reaction must be provided in the chemistry file.

The distribution of LoKI-GM further includes other rate coefficient functions, implemented for specific gases, for example:

- in folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions/Nitrogen`, evaluating: (i) atomic and molecular VV and VT rate coefficients between nitrogen molecules and atoms, including mechanisms leading to N_2 dissociation [38, 57]; (ii) rate coefficients for EV mechanisms in nitrogen [38, 58]. **NOTE THAT THESE FUNCTIONS ARE PROVIDED FOR DEMONSTRATION PURPOSES ONLY, WHEN RUNNING THE SIMPLIFIED NITROGEN CHEMISTRY WITH THE DEFAULT SETUP OF LoKI-GM**;
- in folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions/Oxygen`, evaluating: (i) atomic and molecular VV and VT rate coefficients between oxygen molecules and atoms, including mechanisms leading to O_2 dissociation [59, 60]; (ii) rate coefficients for electron-impact dissociation and dissociative electron attachment [59, 60].

Table 6: List of model-based rate coefficient functions distributed with `LoKI-GM_26.07` in folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions/SurfaceKinetics`.

Function name	Model	Comments
`arrheniusSurf`	$\frac{a \exp(b/T_g)}{([S]_{\text{surface}} + [F]_{\text{surface}})(A/V)}$ The rate coefficient is converted to the equivalent of a volume binary-reaction for use in a surface chemistry model.	• Parameters (in order) a [s^{-1}], b [K] • `surfaceSiteDensity` must be provided in `workingConditions`.
`binaryTransportChantrySurface`	Similar to `binaryTransportChantry` [37] (see table 8) The rate coefficient is converted to the equivalent of a volume binary-reaction for use in a surface chemistry model.	• Parameters (in order): (i) the wall recombination probability; (ii) the temperature T used to calculate the thermal velocity $\langle v \rangle$, *e.g.* `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ...; see Reference Manual [12] Note: all the wall reactions for the same species (if applicable) have to be consistent, ensuring a maximum total probability of 1 for the losses at the wall • The corresponding reduced diffusion coefficients must be provided in the setup file • `surfaceSiteDensity` must be provided in the `workingConditions`
`binaryTransportSurface`	Similar to `binaryTransport` [37] (see table 8) The rate coefficient is converted to the equivalent of a volume binary-reaction for use in a surface chemistry model.	• Parameters (in order): (i) the wall recombination probability; (ii) the temperature T used to calculate the thermal velocity $\langle v \rangle$, *e.g.* `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ...; see Reference Manual [12] Note: all the wall reactions for the same species (if applicable) have to be consistent, ensuring a maximum total probability of 1 for the losses at the wall • The corresponding reduced diffusion coefficients must be provided in the setup file • `surfaceSiteDensity` must be provided in the `workingConditions`
`multicomponentTransportChantrySurface`	Similar to `multicomponentTransportChantry` [37] (see table 8) The rate coefficient is converted to the equivalent of a volume binary-reaction for use in a surface chemistry model.	• Parameters (in order): (i) the wall recombination probability; (ii) the temperature T used to calculate the thermal velocity $\langle v \rangle$, *e.g.* `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ...; see Reference Manual [12] Note: all the wall reactions for the same species (if applicable) have to be consistent, ensuring a maximum total probability of 1 for the losses at the wall • The corresponding Lennard-Jones parameters must be provided in the setup file • `surfaceSiteDensity` must be provided in the `workingConditions`

Table 6: List of model-based rate coefficient functions distributed with `LoKI-GM_26.07` in folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions/SurfaceKinetics` (continued).

Function name	Model	Comments
`multicomponentTransportSurface`	Similar to `multicomponentTransport` [37] (see table 8) The rate coefficient is converted to the equivalent of a volume binary-reaction for use in a surface chemistry model.	• Parameters (in order): (i) the wall recombination probability; (ii) the temperature T used to calculate the thermal velocity $\langle v \rangle$, *e.g.* `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ...; see Reference Manual [12] Note: all the wall reactions for the same species (if applicable) have to be consistent, ensuring a maximum total probability of 1 for the losses at the wall • The corresponding Lennard-Jones parameters must be provided in the setup file • `surfaceSiteDensity` must be provided in the `workingConditions`
`surfaceTransport`	$\frac{k^0_{r_j} \exp\left(-\frac{E_{r_j}}{RT_w}\right)}{([F]_{\text{surface}} + [S]_{\text{surface}})(A/V)} \times \frac{3}{4}\nu_{D_j} \exp\left(-\frac{E_{D_j}}{RT_w}\right)$ Surface-diffusion rate coefficient for use in a mesoscopic surface chemistry model [38–40] (see Reference Manual [12]).	• Parameters (in order): ν_{D_j} [s^{-1}], E_{D_j} [J mol^{-1}], $k^0_{r_j}$ [ ], E_{r_j} [J mol^{-1}], and a Boolean parameter to choose the 3/4 back-diffusion correction • `surfaceSiteDensity` must be provided in the `workingConditions`
`thermalDesorption`	$\nu_{dr_j} \exp\left(-\frac{E_{dr_j}}{RT_w}\right)$ Thermal desorption rate coefficient for use in a mesoscopic surface chemistry model [38–40] (see Reference Manual [12]).	• Parameters (in order): ν_{dr_j} [s^{-1}], E_{dr_j} [J mol^{-1}]
`thermalEffusionSurface`	$\frac{k^0_{r_j} \exp\left(-\frac{E_{r_j}}{RT}\right)}{([F]_{\text{surface}} + [S]_{\text{surface}})(A/V)} \frac{1}{\tau_{r_j}}$ Adsorption rate coefficient for use in a mesoscopic surface chemistry model [38–40] (see Reference Manual [12]).	• Parameters (in order): $k^0_{r_j}$ [ ], E_{r_j} [J mol^{-1}], T [K] • The current version of this function adopts the *wall recombination* model, with $\tau_{r_j} = \tau_{\text{wall}_j} \simeq (4/\langle v_j \rangle)(V/A)$ • The temperature T is used to calculate the thermal velocity $\langle v_j \rangle$ and can be set equal to `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ... • `surfaceSiteDensity` must be provided in the `workingConditions`

Table 6: List of model-based rate coefficient functions distributed with `LoKI-GM_26.07` in folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions/SurfaceKinetics` (continued).

Function name	Model	Comments
		• Parameters (in order): $k^0_{r_j}$ [ ], E_{r_j} [J mol^{-1}], T [K] • The current version of this function adopts the *wall recombination* model, with $\tau_{r_j} = \tau_{\text{wall}_j} \simeq (4/\langle v_j \rangle)(V/A)$ • The temperature T is used to calculate the thermal velocity $\langle v_j \rangle$ and can be set equal to `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ... • `surfaceSiteDensity` must be provided in the `workingConditions`

`thermalEffusionSurface_SKO`	$\frac{k^0_{r_j} \exp\left(-\frac{E_{r_j}}{RT}\right)}{([F]_{\text{surface}} + [S]_{\text{surface}})(A/V)} \frac{1}{\tau_{r_j}}$
	Adsorption rate coefficient for use in a mesoscopic surface chemistry model [38–40] (see Reference Manual [12]), when activating a Surface Kinetics Only (SKO).
		The surface reactions calling this function should add a `volume catalyst` to the right- and left-hand sides, *e.g.*
		`O(3P) + wall_empty(F) -> wall_O(F) + O(3P)`
		`O(3P) + wall_empty(S) -> wall_O(S) + O(3P)`
		`O(3P) + wall_O(S) -> wall_empty(S) + O(3P)`
		
		For $[F_v] = 0.90$, $[S_v] = 0.05$ and $[O_s] = 0.05$, the setup file should be configured by setting
		`gasProperties:`
		`fraction:`
		`- wall_empty=0.90+0.05`
		`- wall_O=0.05`
		`stateProperties:`
		`population:`
		`- wall_empty(F)=0.90/0.95`
		`- wall_empty(S)=0.05/0.95`
		`- wall_O(S)=1.0`
		.

`wallReactionSurface`	Similar to `wallReaction` [37] (see table 8) The rate coefficient is converted to the equivalent of a volume binary-reaction for use in a surface chemistry model.	• Parameters (in order): (i) the wall recombination probability; (ii) the temperature T used to calculate the thermal velocity $\langle v \rangle$, *e.g.* `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ...; see Reference Manual [12] Note: all the wall reactions for the same species (if applicable) have to be consistent, ensuring a maximum total probability of 1 for the losses at the wall • `surfaceSiteDensity` must be provided in the `workingConditions` .

Table 7: List of model-based rate coefficient functions distributed with `LoKI-GM_26.07` in folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions/TransportIons`.

Function name	Model	Comments
`analyticalHFactorTransport`	Evaluates the transport rate coefficient of a single type of positive ions, adopting an h-factor drift-dominated model, solved analytically for plane-parallel electropositive discharges [41, 44] (see Reference Manual [12])	• Parameter: total ion collision cross section (constant value), σ (m^2) • The electron temperature must be provided in the setup file
`classicalAmbipolarDiff`	Evaluates the diffusion rate coefficient of positive ions, under the classical ambipolar diffusion approximation [41–43] (see Reference Manual [12])	• Parameter: wall recombination probability (between 0 and 1), γ [ ] • The corresponding reduced diffusion coefficient and reduced mobility must be provided in the setup file
`classicalAmbipolarDiffNegIon`	Evaluates the diffusion rate coefficient of positive ions, under the classical ambipolar diffusion approximation, considering the effect of several negative ions [41, 43, 45, 46] (see Reference Manual [12])	• The corresponding reduced diffusion coefficient and reduced mobility must be provided in the setup file
`effectiveAmbipolarDiff`	Evaluates the diffusion rate coefficient of positive ions, for a plasma with multiple positive ions and a single negative ion with low density [41, 42] (see Reference Manual [12])	• Parameter: a Boolean parameter to include/exclude the influence of the negative ion density • The corresponding reduced diffusion coefficient and reduced mobility must be provided in the setup file
`hFactorTransport`	Evaluates the transport rate coefficient of positive ions, adopting h-factor models for drift-dominated scenarios in electropositive plasmas, solved numerically at low-pressure, and for drift-diffusion scenarios, featuring the transition from electropositive to moderate or strong electronegative plasmas [41, 47–53] (see Reference Manual [12]).	• Parameters (in order): (i) the total ion collision cross section (constant value), σ (m^2); (ii) a Boolean parameter to include/exclude a low pressure correction for h_L; (iii) a Boolean parameter to include/exclude a fit for α_S from Thorsteinsson and Gudmundsson [51] • The electron and the gas temperatures must be provided in the setup file
`QGMTransport`	Evaluates the transport rate coefficient of positive ions, for a plasma with multiple positive and negative ions (assumed singly-charged) with similar temperatures ($T_+ \simeq T_-$), moderate electronegativities and very low mobilities ($\mu_+ \simeq \mu_- \ll \mu_e$). This transport model is described in [54] and is adopted by the Quantemol Global Model (QGM) [41, 55]; (see Reference Manual [12]).	• Parameters (in order): (i) the wall recombination probability (between 0 and 1) of the positive ion, γ_j [ ]; (ii) a Boolean parameter to include/exclude the influence of the ions (positive and negative) in the calculation of the multi-component diffusion coefficient; (iii) a string (**`transport`** / **`diffusion`**) to select the full/simplified version of the model, considering the effects of the multicomponent ambipolar transport and the thermal losses at the wall or considering only the effects of the ambipolar transport, using the binary diffusion coefficient as free diffusion coefficient of the positive ion species • The electron and the gas temperatures must be provided in the setup file

Table 8: List of model-based rate coefficient functions distributed with `LoKI-GM_26.07` in folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions/TransportNeutrals`.

Function name	Model	Comments
`binaryDiff`	Evaluates the diffusion rate coefficient of neutral species interacting with the wall, adopting a binary diffusion model where the species diffuse in the gas (neglecting the effects of multicomponent transport), assuming a density profile that is zero at the walls with a certain wall-recombination probability [41] (see Reference Manual [12]).	• Parameter: wall recombination probability (between 0 and 1), γ [ ] • The corresponding reduced diffusion coefficients must be provided in the setup file
`binaryDiffusion`	Evaluates the diffusion rate coefficient of neutral species interacting with the wall, adopting a binary diffusion model where the species diffuse in the gas (neglecting the effects of multicomponent transport), assuming a density profile that is zero at the walls and equal probability for each wall recombination channel [41] (see Reference Manual [12]).	• The corresponding reduced diffusion coefficients must be provided in the setup file
`binaryTransport`	Evaluates the diffusion rate coefficient of a particular neutral species interacting with the wall, adopting a binary diffusion model where the species diffuse in the gas (neglecting the effects of multicomponent transport), taking into account both the binary diffusion time and the wall reaction time assuming a flux reaching the wall with a certain wall recombination/deactivation probability [41] (see Reference Manual [12]).	• Parameters (in order): (i) the wall recombination probability; (ii) the temperature T used to calculate the thermal velocity $\langle v \rangle$, *e.g.* `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ...; (see Reference Manual [12] Note: all the wall reactions for the same species (if applicable) have to be consistent, ensuring a maximum total probability of 1 for the losses at the wall • The corresponding reduced diffusion coefficients must be provided in the setup file
`binaryTransportChantry`	Evaluates the diffusion rate coefficient of a particular neutral species interacting with the wall, adopting a binary diffusion model where the species diffuse in the gas (neglecting the effects of multicomponent transport), taking into account both the binary diffusion time and the wall reaction time assuming a flux reaching the wall with a certain wall recombination/deactivation probability [41]. This implementation of the binary transport uses the "heuristic" formula provided by Chantry [37] for the effective diffusion length (see Reference Manual [12]).	• Parameters (in order): (i) the wall recombination probability; (ii) the temperature T used to calculate the thermal velocity $\langle v \rangle$, *e.g.* `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ...; (see Reference Manual [12] Note: all the wall reactions for the same species (if applicable) have to be consistent, ensuring a maximum total probability of 1 for the losses at the wall • The corresponding reduced diffusion coefficients must be provided in the setup file

Table 8: List of model-based rate coefficient functions distributed with `LoKI-GM_26.07` in folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions/TransportNeutrals` (continued).

Function name	Model	Comments
`multicomponentDiffusion`	Evaluates the diffusion rate coefficient of a particular neutral species interacting with the wall, adopting multicomponent diffusion (using Wilke's formula [56]) and neglecting the wall reaction time, assuming a density profile that is zero at the walls and equal probability for each wall-recombination channel [37, 41] (see Reference Manual [12]).	• The corresponding Lennard-Jones parameters must be provided in the setup file
`multicomponentTransport`	Evaluates the transport rate coefficient of a particular neutral species interacting with the wall, taking into account both the multicomponent diffusion time (using Wilke's formula [56]) and the wall reaction time, assuming a flux reaching the wall with a certain wall recombination/deactivation probability [37,41] (see Reference Manual [12]).	• Parameters (in order): (i) the wall recombination probability; (ii) the temperature T used to calculate the thermal velocity $\langle v \rangle$, *e.g.* `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ...; (see Reference Manual [12] Note: all the wall reactions for the same species (if applicable) have to be consistent, ensuring a maximum total probability of 1 for the losses at the wall • The corresponding Lennard-Jones parameters must be provided in the setup file
`multicomponentTransportChantry`	Evaluates the transport rate coefficient of a particular neutral species interacting with the wall, taking into account both the multicomponent diffusion time (using Wilke's formula [56]) and the wall reaction time, assuming a flux reaching the wall with a certain wall recombination/deactivation probability [41]. This implementation of the binary transport uses the "heuristic" formula provided by Chantry [37] for the effective diffusion length (see Reference Manual [12]).	• Parameters (in order): (i) the wall recombination probability; (ii) the temperature T used to calculate the thermal velocity $\langle v \rangle$, *e.g.* `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ...; (see Reference Manual [12] Note: all the wall reactions for the same species (if applicable) have to be consistent, ensuring a maximum total probability of 1 for the losses at the wall • The corresponding Lennard-Jones parameters must be provided in the setup file
`wallReaction`	Evaluates the transport rate coefficient of a particular species interacting with the wall, assuming a flux reaching the wall with a certain wall recombination/deactivation probability, and neglecting the diffusion transport [37,41] (see Reference Manual [12]).	• Parameters (in order): (i) the wall recombination probability; (ii) the temperature T used to calculate the thermal velocity $\langle v \rangle$, *e.g.* `wallTemperature`, `nearWallTemperature`, `gasTemperature`, ...; (see Reference Manual [12] Note: all the wall reactions for the same species (if applicable) have to be consistent, ensuring a maximum total probability of 1 for the losses at the wall

Table 9: List of model-based rate coefficient functions distributed with `LoKI-GM_26.07` in folder `[repository folder]/LoKI-GM_26.07/Code/RateCoeffFunctions/Others`.

Function name	Model	Comments
`inFlow`	Evaluates the (generalized) rate coefficient (in $\mathrm{m^{-3}\,s^{-1}}$) for the inlet flow of species assuming conservation of atoms in the gas/plasma mixture. The inlet flow of species j is calculated as $\nu_{\mathrm{inflow}} n_j(\mathrm{m^{-3}s^{-1}}) = \frac{(dV/dt)_j}{V_{\mathrm{chamber}}} n_j$ with $\left(\frac{dV}{dt}\right)_j(\mathrm{m^3 s^{-1}}) = \frac{Q(\mathrm{sccm})10^{-6}}{60}\frac{p_{\mathrm{PTN}}}{p}\frac{T_g}{T_{\mathrm{PTN}}}$ and $T_{\mathrm{PTN}} = 273.15$ K, $p_{\mathrm{PTN}} = 1.013 \times 10^5$ Pa (see Reference Manual [12]).	• The inlet flow, Q [sccm], must be provided in the setup file • The dimensions of the reactor chamber, R [m] and L [m], must be provided in the setup file • The populations of the j-species, $\delta_j = n_j/N$ [ ], must be provided in the setup file
`outFlow`	Evaluates the rate coefficient (in $\mathrm{s^{-1}}$) for the outlet flow of species assuming conservation of atoms in the gas/-plasma mixture. When chosen the `ensureIsobaric` model, the rate coefficient ν_{outflow}, assumed equal for all volume states, is calculated by imposing the isobaric condition $dp/dt = 0$ to the particle conservation equation, yielding $\nu_{\mathrm{outflow}}(\mathrm{s^{-1}}) = \frac{1}{N}\sum_j \left(S_j^{\mathrm{chem}} + S_j^{\mathrm{transp}}\right) + \frac{1}{T_g}\frac{dT_g}{dt}$ with S_j^{chem} and S_j^{transp} the source terms due to chemical reactions and transport losses, respectively. For other flow models, the rate coefficient, assumed equal for all volume states, is calculated as $\nu_{\mathrm{outflow}}(\mathrm{s^{-1}}) = \frac{Q_{\mathrm{outflow}}(\mathrm{sccm})10^{-6}}{60}\frac{p_{\mathrm{PTN}}(\mathrm{Pa})}{k_B T_{\mathrm{PTN}}(\mathrm{K})} \times \frac{1}{V_{\mathrm{chamber}}(p(t)/k_B T_g(t))}$, with $p(t)$ and $T_g(t)$ the time-dependent gas pressure and temperature, respectively (see Reference Manual [12]).	• The outlet flow, Q_{outflow} [sccm], or the corresponding outflow model, must be provided in the setup file • The dimensions of the reactor chamber, R [m] and L [m], must be provided in the setup file • The time-dependent total gas density $N(t)$, must be provided as input to the function

5.3 Other auxiliary functions

In folder `OtherAuxFunctions` the user can find `*.m` functions

- for evaluating the time derivative of the EEDF at a given time t. In the current distribution of LoKI, this function is `eedfTimeDerivative.m` and is not optimized for standalone LoKI-B calculations;
- for calculating working conditions that can evolve during the simulations. In the current distribution of LoKI, this functionality is for calculating the `reducedField` in the case of LoKI-B calculations for step electric fields (function `reducedFieldRamp.m`) or electric field pulses (function `reducedFieldPulse.m`);
- for obtaining the reduced electric field resulting from the imposition of a time-dependent discharge current (functions `reducedFieldFromCurrentFile.m` and `getEoNFromCurrent.m`)

In general these functions can be prepared by the user, according to the needs for new functionalities. In this case, the user is advised to mimic the structure of any auxiliary function already provided with the code.

5.4 Utilities

In folder `Utilities` the user can find scripts

- for plotting two different EEDFs in the same MATLAB® graph. In the current distribution of LoKI, this function is `compareEEDFs.m`;
- for plotting the errors associated with the convergence cycles from the log-file. In the current distribution of LoKI, this function is `graphical_log.m`;
- for interactively assist the modification of LXCat **text-files**, for compliance with the LoKI-B parser. In the current distribution of LoKI, this function is `LXCat2LoKI.m`.

6 Changelog (until 2026/07/07)

This is probably not very interesting for the ordinary user. But since you have made it this far, please take a moment to read a brief history of the code since its first release.

This section documents code changes up to 2026/07/07. For subsequent updates, please refer to the LoKI-GM GitHub repository
https://github.com/LoKI-Suite/LoKI-GM
which contains the latest information on code development.

- `LoKI-GM_26.07` (2026/07/07):
  - Release of `LoKI-GM_26.07` as open-source tool, comprising updated versions of LoKI-B and LoKI-C
  - IMPORTANT NEW FEATURE: Extend simulation types to quasi-stationary simulations, replacing the electron kinetics by lookup tables, under either the LFA or LEA approximations
  - NEW FEATURE: Refactor `Chemistry.solve` by extracting steady-state and post-discharge logic and by including pulsed-simulations logic
  - IMPORTANT NEW FEATURE: I/O support for multiple simulations with LoKI-B, including simultaneous variations of several parameters: E/N (or T_e), T_g, ω, p and n_e. The output support extends to TXT files (full), GUI (partial) and HDF5 (partial)
  - NEW FEATURE: Improves environment cleanup for performance
  - NEW FEATURE: add automatic EEDF plot refresh when anisotropy is toggled
  - NEW FEATURE: added support for checking mass conservation in reactions (with addition of flag `checkChemFilesStoichiometry` in the setup file)
  - IMPORTANT NEW FEATURE: automatic handling of input/output flow by the code (i) to ensure constant pressure; (ii) to allow for "flow barriers" (factors between 0-1) specifically defined for each individual species
  - IMPORTANT NEW FEATURE: introduction of `electronDensity`, `dischargeCurrent` and `dischargePowerDensity` as new parameters under `workingConditions` and `convergenceParameter`
  - IMPORTANT NEW FEATURE: redefinition of iterative cycles: pressure, neutrality, global
  - NEW FEATURE: improvement of function `iterateOverParameter` (i) to correct the linear extrapolation; (ii) to add the possibilities of not limiting the linear extrapolation and to deactivate the bissection method (these changes are essential for a quick convergence in n_e); (iii) to avoid negative parameters when random generation is used
  - IMPORTANT NEW FEATURE: inclusion of HDF5 support for output
  - NEW FEATURE: new `output` parameters: inclusion of `dataFormat`; renaming of `dataFiles` as `dataSets`
  - NEW FEATURE: gas heating source terms (i) added the possibility of calculating the gas heating using the Joule power (from the electron kinetics) as sole source term; (ii) inclusion, as source term and for states not defined in the chemistry, of the power transferred from vibrations and rotations, as calculated by the electron kinetics (it includes the CAR description). For consistency, if vibrational / rotational states are defined in the chemistry, the power calculated by the electron kinetics is not used, as it is already included in the state-to-state description
  - NEW FEATURE: inclusion of additional property functions and general revision of these functions
  - NEW FEATURE: inclusion of additional rate coefficient functions and general revision of these functions
  - UPDATE: improvement in documentation: (i) general revision; (ii) separation of the *User Manual* into a *User Guide* and a *Reference Manual*; (iii) generation of `HTML` documentation
  - UPDATE: reset position of GUI for larger-screen devices
  - UPDATE: New organization (i) `Utilities` folder moved into `Code` folder; (ii) functions were organized among folders `OtherAuxFunctions` and `Utilities`
  - UPDATE: in GUI that now shows positive/negative errors in chemistry cycles

- UPDATE: general optimization of convergence criteria (default tolerances and relative errors)
- UPDATE: general improvement of output TXT files and GUI (particularly for chemistry parameters)
- UPDATE: better use of I/O regarding chemistry, by using the `chemistry` object as parameter, particularly in `rateCoeffFunctions`
- UPDATE: general revision of the `chemistry` class organization
- UPDATE: improvement and revision of systematic self-diagnostic tests
- UPDATE: in thermal model (i) avoids concatenation of "empty" surface masses, thus fixing an error when the thermal model is activated and surface species are considered in the kinetics (`massArray` is used in thermal conductivity); (ii) separates the IDs of surface, transport and volume reactions, thus allowing to consider also surface processes with a fractional heating "fw"; (iii) updates the gas temperature only in the global cycle
- UPDATE: to speedup `postDischarge` calculations: `dependent.onElectronKinetics` only if `eedf` is being solved
- UPDATE: chemistry data for oxygen, CO_2 and nitrogen
- UPDATE: in function `thermalEffusionSurface function`, added the possibility of selecting different temperatures (gas, wall, ...) for the model
- UPDATE: in LXCat cross section files (TXT and JSON) for several gases
- BUG FIX: added exception for *atomless* gases in the `getAtoms` class method
- BUG FIX: when using external boundary condition in the thermal model
- BUG FIX: correct evaluation of `reactionRates` on the final time, for reactions with `gas` as catalyst
- BUG FIX: corrections to handle a HF case with the thermal model activated (updating the reduced excitation frequency)
- BUG FIX: allow for volume catalyst species when calculating the thermal velocity, for the calling by `effectiveSurfaceTransport`
- BUG FIX: solved error in function `radiationImprisonmentEinsteinCoeff`: the expression to evaluate the absorption coefficient was multiplied by the velocity, instead of divided
- BUG FIX: in GUI, to plot the iteration parameter T_e in case `PrescribedEedf`
- BUG FIX: to allow LXCat extra files to be saved as output
- BUG FIX: EEDF evolution during post-discharge should be deactivated by default
- A few minor changes to improve stability and performance of the code.

- `LoKI-B_v25.10.1` (2025/09/25):
  - IMPORTANT NEW FEATURE: initial support in JSON format, for auxiliary LXCat files and input files.
  - IMPORTANT NEW FEATURE: definition of elastic collisions extended to electronic/vibrational/rotational states, without any restriction other than avoid duplicated or missing information.
  - IMPORTANT NEW FEATURE: added support for output in HDF5 format, with changes in some fields of the input file. For more information see section 3.5.
  - NEW FEATURE: generalizing the writing of thresholds in cross section files using scientific notation with uppercase (E) descriptors (before, only lowercase (e) descriptors were supported).
  - NEW FEATURE: automatic indexing to maximum power-balance error in the convergence of e-e collisions.
  - NEW FEATURE: added property function to calculate the Boltzmann population of orto/para rotational states of H_2.
  - BUG FIX: throw an error when effective cross sections are provided for several electronic states' siblings of the same gas.
  - BUG FIX: correction of function `Setup.m` to meet changes in MATLAB's function `ind2sub`, which no longer supports scalar input for array size (after R2024a).
  - BUG FIX: multiplication of the `prescribedEedf` expression by the "shape" parameter was missing.

  - BUG FIX: correction of (i) the discretization of the electron density spatial-growth operator; (ii) evaluation of the electron mobility in the case of spatial growth, consistent with (i) and with its evaluation in the `evaluateSwarmParameters` function; (iii) evaluation of the power lost due to the electron density spatial growth, consistent with (i) and with the calculation of the power using swarm parameters, as evaluated in the `evaluateSwarmParameters` function.
  - BUG FIX: solved error in the function that solves e-e collisions. When evaluating the A matrix, both old and new values of the matrix elements were used to impose detailed balance.
  - BUG FIX: corrections in the GUI related to `prescribedEedf`.
  - BUG FIX: solved error preventing LXCat extra files to be saved as output.
  - BUG FIX: solved error in the checking of the availability of elastic cross sections.
  - UPDATE: in cross section files for N-atoms, including superelastic mechanisms from N(2P,2D).
  - A few minor changes to improve stability and performance of the code.

- `LoKI_v3.1.0` (2022/11/04):
  - Forth release of the code with newer versions of both the Boltzmann and Chemistry modules (`LoKI-B_v2.2.0` and `LoKI-C_v3.1.0`)
  - NEW FEATURE: added general saving of all input files as output.
  - NEW FEATURE: added support for one-level subfolder organisation of source files.
  - NEW FEATURE: first implementation of logging system.
  - NEW (hidden) FEATURE: first implementation of the coupling between LoKI-B and LoKI-C for post-discharge simulations (to be verified/extended for stability improvement and towards a full coupling between the tools, in future releases).
  - A few minor changes to improve stability and performance of the code.

- `LoKI-C_v3.1.0` (2022/11/04):
  - NEW FEATURE: added new transport rate coefficient function `classicalAmbipolarDiffNegIon`.
  - NEW FEATURE: detailed balance calculations have been extended to reverse mechanisms other than the default ones (eedf and two-body volume heavy-species reactions).
  - NEW FEATURE: new implementation of the thermal module, considering different boundary conditions (see Reference Manual [12]), and adding support for the `GUI` and `Output` classes.
  - NEW FEATURE: information on the steady-state rate coefficients of the chemistry is now included in the `rateCoefficients.txt` output file.
  - NEW FEATURE: users can select the temperature adopted in the rate coefficient functions of transport models where the thermal velocity is evaluated.
  - NEW FEATURE: added errors when reaching the maximum number of iterations for the chemistry cycles.
  - NEW FEATURE: added new tab to the GUI with information on the input of the chemistry reactions.
  - NEW FEATURE: added the possibility of selecting lin/log scale for both axis on the chemistry-related graphs.
  - BUG FIX: the code was not prepared to consider negative charges when evaluating the electron density (imposing quasi-neutrality) during post-discharge calculations.
  - BUG FIX: added checks for (partially) empty reactions (*i.e.* reactions without reactants and/or products), for which the code now throws an error message. Accordingly, the calling of the `inFlow` / `outFlow` rate coefficient functions now uses `e -> X` and `X -> e`, respectively (workaround to be solved in future releases).
  - BUG FIX: proper normalisation of populations for empty gases.
  - BUG FIX: improve `iterateOverParameter` function to avoid stalling.
  - A few minor changes to improve stability and performance of the code.

- `LoKI-B_v2.2.0` (2022/11/04):

- NEW FEATURE: general implementation of anisotropic e-impact collisions.
- NEW FEATURE: proper implementation of hf simulations for prescribed eedf .
- BUG FIX: added checking to avoid e-impact collisions types non-supported by LXCat.
- BUG FIX: improve the evaluation of power in electron-electron collisions.
- BUG FIX: solved error in the temporal growth operator, when using the time-integration algorithm.
- A few minor changes to improve stability and performance of the code.

- `LoKI_v3.0.0` (2022/03/04):
  - Third release of the code with newer versions of both the Boltzmann and Chemistry modules (`LoKI-B_v2.1.0` and `LoKI-C_v3.0.0`)
  - BUG FIX: solved error in the population property functions when using `electronTemperature` as parameter.
  - BUG FIX: solved a problem in the displaying of text in the GUI for UNIX machines.
  - Removed some deprecated functions and files.
  - A few minor changes to improve stability and performance of the code.

- `LoKI-C_v3.0.0` (2022/03/04):
  - NEW FEATURE: first implementation of surface kinetics (including a revision of transport functions)
  - BUG FIX: solved a problem with the normalisation of vibrational distributions during output for both GUI and text files.
  - BUG FIX: `wallTemperature` is now used when available in the calculation of thermal velocity for transport models.
  - BUG FIX: solved small error in the evaluation of the power transferred from electrons to gas temperature in the thermal module.
  - BUG FIX: solved small error in the units presented in output files.
  - BUG FIX: for HF simulations the chemistry module now receives and uses the DC mobility for transport calculations.
  - Thermal model now only accounts for radial losses.
  - improved performance of the chemistry integration following suggestions by Luis Marques.
  - A few minor changes to improve stability and performance of the code.

- `LoKI-B_v2.1.0` (2022/03/04):
  - NEW FEATURE: the DC reduced mobility is always evaluated and the (complex) HF mobility is only provided in the case of oscillating fields.
  - NEW FEATURE: added the possibility of selecting lin/log scale for both axis on the graphs of the different swarm parameters. For the power graph only the xaxis can be toggled between lin/log scale.
  - BUG FIX: solved important error in the e-e collision (`mixingDirectSolutions` scheme only) operator related with the units of the reduced field
  - BUG FIX: now attachment collisions are not considered for the evaluation of a reduced field from power balance when using `PrescribedEedf`.
  - BUG FIX: activated the renormalisation of the eedf for the calculation of the eedf derivative.
  - BUG FIX: solved an inconsistency in the evaluation of the reference power in `PrescribedEedf` with respect to `Boltzmann` class.
  - Added the possibility of providing thresholds with scientific notation in LXCat files.
  - A few minor changes to improve stability and performance of the code.

- `LoKI-B_v2.0.1` (2021/09/20):
  - BUG FIX: solved error in the clipping of negative values of an elastic cross section (when evaluated from an effective one) to zero.

- `LoKI_v2.0.0` (2021/02/14):
  - Second release of the code with newer versions of both the Boltzmann and chemistry modules (`LoKI-B_v2.0.0` and `LoKI-C_v2.0.0`)
  - BUG FIX: Solved an error in the evaluation of an elastic cross section from an effective one that occurs for some initialisation conditions when both modules are activated.
  - New system of property functions implemented LoKI-wide to accommodate necessities of the LoKI-C thermal module (see sections 3.2, 3.3 and 5.1).
  - A few minor changes to improve stability and performance of the code.
- `LoKI-C_v2.0.0` (2021/02/14):
  - NEW FEATURE: First implementation of the thermal module (self-consistent calculation of the gas temperature, (see Reference Manual [12]).
  - NEW FEATURE: Added the possibility of saving the time-dependent densities of species.
  - NEW FEATURE: First implementation of a post-discharge description.
  - BUG FIX: Solved some errors related to the multicomponent transport rate coefficient functions.
  - BUG FIX: Solved an error in the orientation of some matrices in the `multicomponentTransportChantry` rate coefficient function.
  - BUG FIX: Solved some errors in certain rate coefficient functions related with the dimensions of the discharge.
  - A few minor changes to improve stability and performance of the code.
- `LoKI-B_v2.0.0` (2021/02/14):
  - NEW FEATURE: Added possibility to perform pulsed simulations (see section 3 and Reference Manual [12])
  - NEW FEATURE: Added two new parameters calculated on output, the energy diffusion coefficient and the energy mobility.
  - BUG FIX: Solved small error in the implementation of the CC-CAR operator (see Reference Manual [12]).
  - BUG FIX: Solved an error in the evaluation of both the Townsend coefficient and the attachment coefficient (the code now accounts for more than one of such collisions per gas).
  - BUG FIX: Solved an inconsistency between the code and the documentation. The code now evaluates the real part of the HF mobility for HF simulations (see Reference Manual [12]).
  - BUG FIX: Solved some inconsistencies in the evaluation of the power from the field when non lineal operators where activated resulting in a wrong evaluation of the power balance in some cases.
  - Removed deprecated class `Maxwellian`.
  - A few minor changes to improve stability and performance of the code.
- `LoKI_v1.0.0` (2019/03/20):
  - First stable release of the code with newer versions of both the Boltzmann and chemistry modules (`LoKI-B_v1.0.0` and `LoKI-C_v1.0.0`)
  - BUG FIX: fixed an error in the output of the eedf (and first anisotropy) when the Boltzmann and chemistry modules are activated.
  - BUG FIX: fixed a problem that may occur during the update of "non-standard" gas mixtures between the chemistry and Boltzmann calculations.
  - A few minor changes to improve stability and performance of the code.
- `LoKI-C_v1.0.0` (2019/03/19):
  - NEW FEATURE: Added support for reaction enthalpies in the ".chem" files in preparation for the thermal module. For more information see section 4.3.
  - NEW FEATURE: Added the first implementation of a generalised transport theory for neutral species. See section 5.2.

- NEW FEATURE: Added support for "gas stabilised reactions" in ".chem" files. For more information see section 2.3.1
- BUG FIX: fixed a problem that prevented in most cases the use of accelerated schemes for the convergence of parameters (in particular for the quasineutrality cycle). Now the convergence, and consequently the code, is much faster.
- BUG FIX: fixed an error that causes the final populations obtained in the chemistry not to be shown/saved properly in some cases.
- BUG FIX: fixed an error for which when the maximum number of iterations for the quasineutrality cycle are reached a new evaluation of the electron kinetics was performed. This error produced a mismatch in the output.
- BUG FIX: fixed error in the output of the EEDF when the chemistry were activated.
- BUG FIX: fixed several errors in the implementation of the detailed balance for heavy species reactions.
- A few minor changes to improve stability and performance of the code.

- `LoKI-B_v1.0.0` (2019/03/20):
  - NEW FEATURE: The user can now use a time integration algorithm to solve the non-linear operators. For more information see section 3.2.
  - NEW FEATURE: Added attachments collisions as non-conservative mechanisms.
  - NEW FEATURE: Added power transfer plot to the GUI.
  - NEW FEATURE: Added evaluation and output of the elastic/effective collision rate coefficients.
  - NEW FEATURE: Added attachment collisions as non-conservative mechanism (when they are present).
  - NEW FEATURE: Added the possibility for the user to change the scale of the EEDF graph in the GUI (linear/log).
  - NEW FEATURE: Added evaluation of the first anisotropy
  - IMPORTANT CHANGE: The reference power is now considered as the total gained power from all possible sources.
  - IMPORTANT CHANGE: The configuration of the electron kinetics on the setup file has been slightly modified in order to accommodate new features. For more information see section 3.2.
  - IMPORTANT CHANGE: Changed the "Maxwellian" EEDF type by "prescribedEedf" (generalised expression with Maxwellian/Druyvesteyn as particular cases). For more information see section 3.2.
  - BUG FIX: fixed a problem that caused the results of the first job of a simulation to be saved outside its subfolder.
  - BUG FIX: fixed small error in the morse oscillator energy formula.
  - A few minor changes to improve stability and performance of the code.

- `LoKI_v0.1.0` (2018/03/26):
  - First release of the code (beta) comprising `LoKI-B_v0.6.3` and `LoKI-C_v0.1.0`

- `LoKI-C_v0.1.0` (2018/03/26):
  - First release of the code (beta)

- `LoKI-B_v0.6.3` (2018/03/26):
  - BUG FIXED: corrected a problem with the Klein-Rosseland expression in the evaluation of superelastic cross sections. The bug only affected the evaluation of an elastic cross section from an effective one, and not the actual Boltzmann calculations. However, results obtained with previous versions of the code may be greatly affected (when using "effective" cross sections and superelastic mechanisms) hence it is recommended to repeat calculations with this new version.
  - BUG FIXED: fixed a problem with the detailed balance in the e-e collisions. This bug only introduced a small error that affected convergence of the non-linear algorithm.

- `LoKI-B_v0.6.2` (2018/01/18):

- BUG FIXED: corrected some expressions used in the evaluation of the swarm parameters, for the cases when the electron density growth is activated.
- BUG FIXED: fixed a problem with the Maxwellian type of EEDF, for which the code used to crash when trying to output (GUI or file) the power losses due to the electron density growth.

- `LoKI-B_v0.6.1` (2018/01/16):
  - BUG FIXED: previously the Spitzer integrals (e-e collision operator) were updated inside the routines with the operators for the electron-density growth. Now the update is done outside these routines, and convergence on both operators is completely isolated.
  - BUG FIXED: previously, different EEDFs were being used to evaluate different parts of the e-e collision operator. Now, the convergence of the algorithm has been improved.
  - The global cycle for the non-linear operators of the Boltzmann equation is now testing the changes in the EEDF only as convergence criterion.
  - Slight improvement in the warning messages, in the cases where the non-linear operators of the Boltzmann equation do not converge.
  - A few minor changes to improve stability and performance of the code.

- `LoKI-B_v0.6.0` (2018/01/13):
  - NEW FEATURE: The user can now select non-conservative ionisation operators ("one takes all", "equal sharing" or "using single differential cross section") for the Boltzmann equation as well as two types of growth models ("spatial" or "temporal") for the electron number density. For more information, see section 3.2 and Reference Manual [12].
  - NEW FEATURE: The user can now include the electron-electron collision operator in the Boltzmann calculations. For more information, see section 3.2 and Reference Manual [12].
  - IMPORTANT CHANGE: Changes in the way the power balance is evaluated. The evaluation of the "reference power" is now fairer. The power transferred to the elastic and the CAR channels is now separated into gains and losses, and the reference power is obtained as the channel from which the electrons gain more energy.
  - BUG FIXED: fixed bug in the State.find static method, that caused errors when finding all the electronic states of a certain gas.
  - BUG FIXED: fixed bug introduced in commit 813e38f (Nov 9, 2017), that prevented the user to deactivate the output of the code.
  - A few minor changes to improve stability and performance of the code.

- `LoKI-B_v0.5.0` (2017/12/14):
  - NEW FEATURE: The electron drift velocity is now evaluated by the code. This information can be seen in the "Swarm parameters" and "Look-up-table" tabs of the GUI, as well as in the corresponding output files. (User Requested)
  - NEW FEATURE: "Extra" electron-scattering cross sections, *i.e.* cross sections that are not taken into account when solving the electron Boltzmann equation, can now be added. The corresponding rate coefficients are obtained by integrating these cross sections over the calculated EEDF. For more information, see section 3.2.
  - NEW FEATURE: The power balance is now detailed in a per-gas basis. (User Requested)
  - BUG FIXED: The output of the electron "Ionization" and "Attachment" was not being done properly.
  - IMPORTANT CHANGE: the only comment character allowed in the input files is now %, for performance reasons. The # character is NO LONGER RECOGNISED.
  - IMPORTANT CHANGE: A few changes in the way the properties of the setup file are organised. For more information, see section 2.4.
  - A few changes where introduced to improve the performance of the code:
    - Improved performance of the Collision.find static method, by avoiding implicit values in the loop.
    - Improved performance of the output.saveEedf method, by vectorizing a loop (almost 13x speedup)

  - A lot of LoKI-C code has been introduced and several changes in the structure of the code has been made in order to accommodate and integrate the chemistry module. However, LoKI-C is still not available (Coming soon!)
  - A few minor changes to improve stability and performance of the code.

- `LoKI-B_v0.4.0` (2017/10/17):
  - NEW FEATURE: The user can now prescribe a Maxwellian EEDF. For more information, see section 3.2.
  - NEW FEATURE: The user can now specify the "refresh frequency" of the GUI, in order to speedup simulations. For more information, see section 3.4.
  - Structural changes in the calculations and aesthetical changes in the GUI, in order to speedup calculations. A huge difference is observed, when compared with previous versions.
  - BUG FIXED: the electron temperature was being calculated with the characteristic energy instead of the mean energy.
  - BUG FIXED: solved bug causing the GUI to display the same rate coefficient for direct and inverse processes.
  - A few minor changes to improve stability and performance of the code.

- `LoKI-B_v0.3.0` (2017/09/08):
  - NEW FEATURE: The user can now specify the populations used for the evaluation of the elastic momentum-transfer cross section, from the effective. This feature must be used with care. For more information, see section 3.2.
  - Added graphs in the GUI, for the reduced first Townsend and attachment coefficients vs. E/N.
  - Included the character "-" in the names of electronic states (important for some N2 states, and not supported before).
  - BUG FIXED: fixed error in the evaluation of the elastic momentum-transfer cross section from the effective. The bug affected molecular gases only.
  - BUG FIXED: fixed a completely harmless error related with the parsing of collisions. Electrons where not properly accounted for.
  - A few minor changes to improve stability and performance of the code.

- `LoKI-B_v0.2.0` (2017/06/09):
  - NEW FEATURE: added support for the continuous approximation for rotations (CAR). For more information, see section 3.2 and Reference Manual [12].
  - BUG FIXED: the evaluation of the elastic momentum-transfer cross section from the effective is now properly done. Previously, there were some errors in the evaluation of the populations of rotational states, as well as with the inclusion of superelastic mechanisms. The problem was not affecting the simulations in which an elastic cross section was provided by the user.
  - A few minor changes to improve stability and performance of the code.

- `LoKI-B_v0.1.1` (2017/04/10):
  - BUG FIXED: solved error in the calculation of the reduced first Townsend coefficient. The reduced field was being used in Townsend instead of SI units, in the evaluation of the coefficient.
  - A few minor changes to improve stability and performance of the code.

- `LoKI-B_v0.1.0` (2017/04/05):
  - First release of the code (beta).

References

[1] Tejero A, Guerra V, Gonçalves D, Lino da Silva M, Marques L, Pinhão N, Pintassilgo C D and Alves L L 2019 *Plasma Sources Sci. Technol.* **28** 043001

[2] Tejero A, Guerra V, Pinhão N, Pintassilgo C D and Alves L L 2021 *Plasma Sources Sci. Technol.* **30** 065008

[3] https://www.ipfn.tecnico.ulisboa.pt/nprime/ (last access October 14, 2026)

[4] Guerra V and Loureiro J 1997 *Plasma Sources Sci. Technol.* **6** 373-385

[5] Alves L L 2007 *Plasma Sources Sci. Technol.* **16** 557-569

[6] https://www.tue.nl/en/research/research-groups/elementary-processes-in-gas-discharges (last access October 14, 2026)

[7] https://www.mathworks.com/products/matlab.html (last access October 14, 2026)

[8] https://isocpp.org/ (last access October 14, 2026)

[9] https://eigen.tuxfamily.org/ (last access October 14, 2026)

[10] Alves L L, Tejero-Del-Caz A, Marques L, Pereira P, Pinhão N, Pintassilgo C D, Silva T, Viegas P and Guerra V 2026 arXiv preprint, https://arxiv.org/abs/2607.27234. Manuscript submitted for publication to *Plasma Sources Sci. Technol.* (under review)

[11] Alves L L, Tejero-Del-Caz A, Marques L, Pereira P, Pinhão N, Pintassilgo C D, Silva T, Viegas P and Guerra V, *The LisbOn Kinetics Global Model*, version v26.07, July 2026, DOI: https://doi.org/10.57859/ulisboa-istipfn.000062, URL: https://github.com/LoKI-Suite/LoKI-GM (last access October 14, 2026)

[12] Alves L L and Tejero A 2026 *The LisbOn Kinetics Global Model: Reference Manual*

[13] https://git-scm.com/ (last access October 14, 2026)

[14] https://lxcat.net/ (last access October 14, 2026)

[15] https://demo.lxcat.net/ (last access October 14, 2026)

[16] https://en.wikipedia.org/wiki/Off-side_rule

[17] Opal C, Peterson W and Beaty E 1971 *J. Chem. Phys.* **55** 4100–4106

[18] Kim Y-K 1995 *Atomic and Molecular Processes in Fusion Edge Plasmas* (Springer Science, Business Media, New York)

[19] Vialetto L, Ben Moussa A, van Dijk J, Longo S, Diomede P, Guerra V and Alves L L 2021 *Plasma Sources Sci. Technol.* **30** 075001

[20] Ridenti M A, Alves L L, Guerra V and Amorim J 2015 *Plasma Sources Sci. Technol.* **24** 035002

[21] Klein O and Rosseland S 1921 *Z. Phys.* **4** 46-51

[22] www.mathworks.com/help/matlab/ref/odeset.html (last access October 14, 2026)

[23] https://www.hdfgroup.org/solutions/hdf5/

[24] https://myhdf5.hdfgroup.org/
http://www.h5py.org/, https://docs.h5py.org

[25] https://github.com/HDFGroup/hdf-compass

[26] Pitchford L C, Alves L L, Bartschat K, Biagi S F, Bordage M C, Phelps A V, Ferreira C M, Hagelaar G J M, Morgan W L, Pancheshnyi S, Puech V, Stauffer A and Zatsarinny O 2013 *J. Phys. D: Appl. Phys.* **46** 334001

[27] Alves L L 2014 *J. Phys. Conf. Series* **565** 012007

[28] Dowling J M 1968 *J. Mol. Spectrosc.* **27** 527

[29] Al Derzi A R, Furtenbacher T, Tennyson J, Yurchenko S N and Császár A G 2015 *J. Quant. Spectrosc. Radiat. Transf.* **161** 117

[30] Furtenbacher T, Coles P A, Tennyson J, Yurchenko S N, Yu S, Drouin B, Tóbiás R and Császár A G 2020 *J. Quant. Spectrosc. Radiat. Transf.* **251** 107027

[31] Treanor C E, Rich J W and Rehm R G 1968 *J. Chem. Phys.* **48** 1798

[32] Gordiets B, Mamedov S and Shelepin L 1975 *Sov. Phys. - JETP* **42** 237

[33] Santos M *et al.* 2014 *J. Phys. D: Appl. Phys* **47** 265201

[34] Pintassilgo C D *et al.* 2014 *Plasma Sources Sci. Technol.* **23** 025006

[35] McBride B J, Zehe M J and Gordon S 2002 *NASA Glenn Coefficients for Calculating Thermodynamic Properties of Individual Species*, NASA/TP-2002-211556

[36] McBride B J, Gordon S and Reno M A 1993 *Coefficients for Calculating Thermodynamic and Transport Properties of Individual Species*, NASA TM-4513

[37] Chantry P J 1987 *J. Appl. Phys.* **62** 1141-1148

[38] Guerra V, Tejero-del-Caz A, Pintassilgo C D and Alves L L 2019 *Plasma Sources Sci. Technol.* **28** 073001

[39] Guerra V 2007 *IEEE Trans. Plasma Sci.* **35** 1397

[40] Marinov D, Teixeira C and Guerra V 2017 *Plasma Processes Polym.* **14** 1600175

[41] Alves L L and Tejero-del-Caz A 2023 *Plasma Sources Sci. Technol.* **32** 054003

[42] Coche P, Guerra V and Alves L L 2016 *J. Phys. D: Appl. Phys.* **49** 235207

[43] Rogoff G L 1985 *J. Phys. D: Appl. Phys.* **18** 1533-45

[44] Czarnetzki U and Alves L L 2022 *Rev. Mod. Plasma Phys.* **6** 31

[45] Ferreira C M, Gousset G and Touzeau M 1988 *J. Phys. D: Appl. Phys.* **21** 1403-13

[46] Guerra V and Loureiro J 1999 *Plasma Sources Sci. Technol.* **8** 110

[47] Godyak V 1986 *ed V D Associates (Fall Church, VA: Delphic Associates)*

[48] Lichtenberg A J, Vahedi V, Lieberman M A and Rognlien T 1994 *J. Appl. Phys.* **75** 2339

[49] Lee C and Lieberman M A 1995 *J. Vac. Sci. Technol. A* **13** 368

[50] Kim S, Lieberman M A, Lichtenberg A J and Gudmundsson J T 2006 *J. Vac. Sci. Technol. A* **24** 2025

[51] Thorsteinsson E G and Gudmundsson J T 2010 *Plasma Sources Sci. Technol.* **19** 015001

[52] Thorsteinsson E G and Gudmundsson J T 2010 *Plasma Sources Sci. Technol.* **19** 055008

[53] Chabert P 2016 *Plasma Sources Sci. Technol.* **25** 025010

[54] Stoffels E, Stoffels W W, Vender D, Haverlag M, Kroesen G M W, and de Hoog F J 1995 *Contrib. Plasma Phys.* **35** 331

[55] Tennyson J *et al.* 2022 *Plasma Sources Sci. Technol.* **31** 095020

[56] Wilke C R 1950 *Chem. Eng. Prog.* **46** 95

[57] Alves L L, Marques L, Pintassilgo C D, Wattieaux G, Es-sebbar E, Berndt J, Kovacević E, Carrasco N, Boufendi L and Cernogora G 2012 *Plasma Sources Sci. Technol.* **21** 45008

[58] Bourdon A and Vervisch P 2000 *J. Thermophys. Heat Transf.* **14** 489

[59] Esposito F, Armenise I, Capitta G and Capitelli M 2008 *Chem. Phys.* **351** 91

[60] Annusova A, Marinov D, Booth J-P, Sirse N, Lino da Silva M, Lopez B and Guerra V 2018 *Plasma Sources Sci. Technol.* **27** 045006